

電力効率 とGPU v7

東京大学大学院 情報理工学系研究科 創造情報学専攻
塩谷 亮太
shioya@ci.i.u-tokyo.ac.jp

本日の話題：Graphics Processing Unit (GPU)

■ GPU

- もともとはグラフィックを高速に処理するためにあった

■ GP-GPU (General Purpose computing on GPU)

- グラフィック以外に、汎用の計算に GPU を使用する使い方
 - 大量の単純な処理の繰り返しが得意
 - 行列積 → 機械学習, 科学技術計算
 - 仮想通貨のマイニング
- ## ■ この講義では GP-GPU での使用を前提として説明
- 以降で「GPU」と言った場合, GP-GPU での使用を仮定
 - グラフィックのための機能には基本触れない

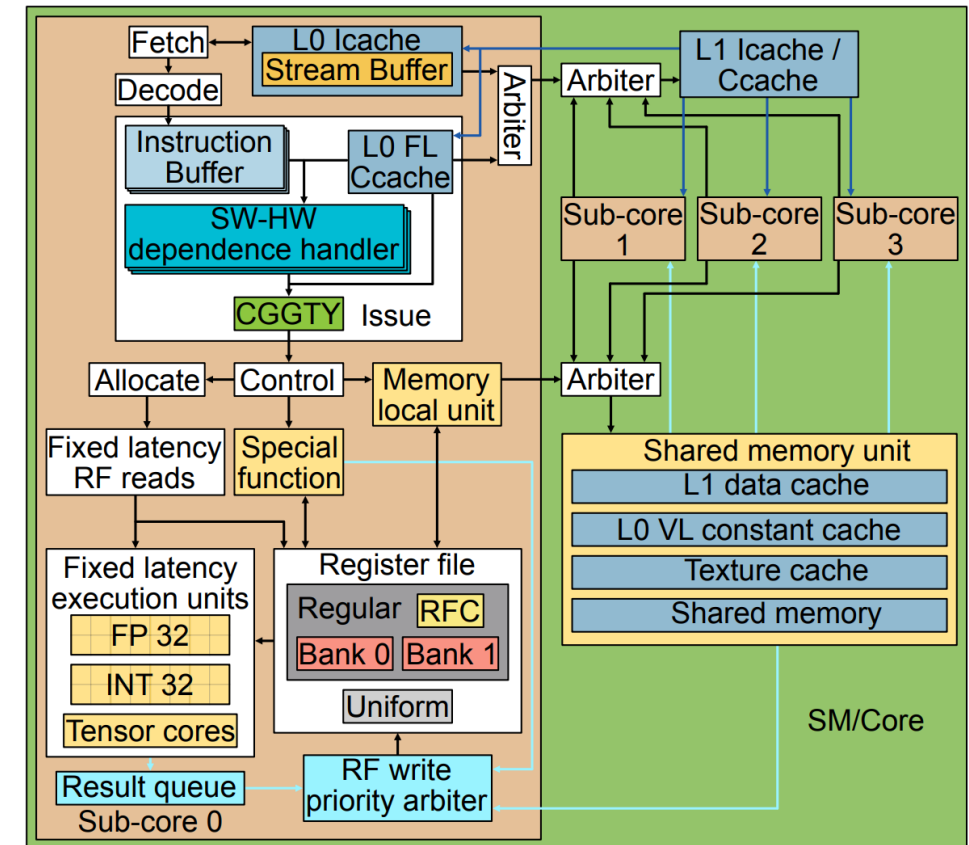
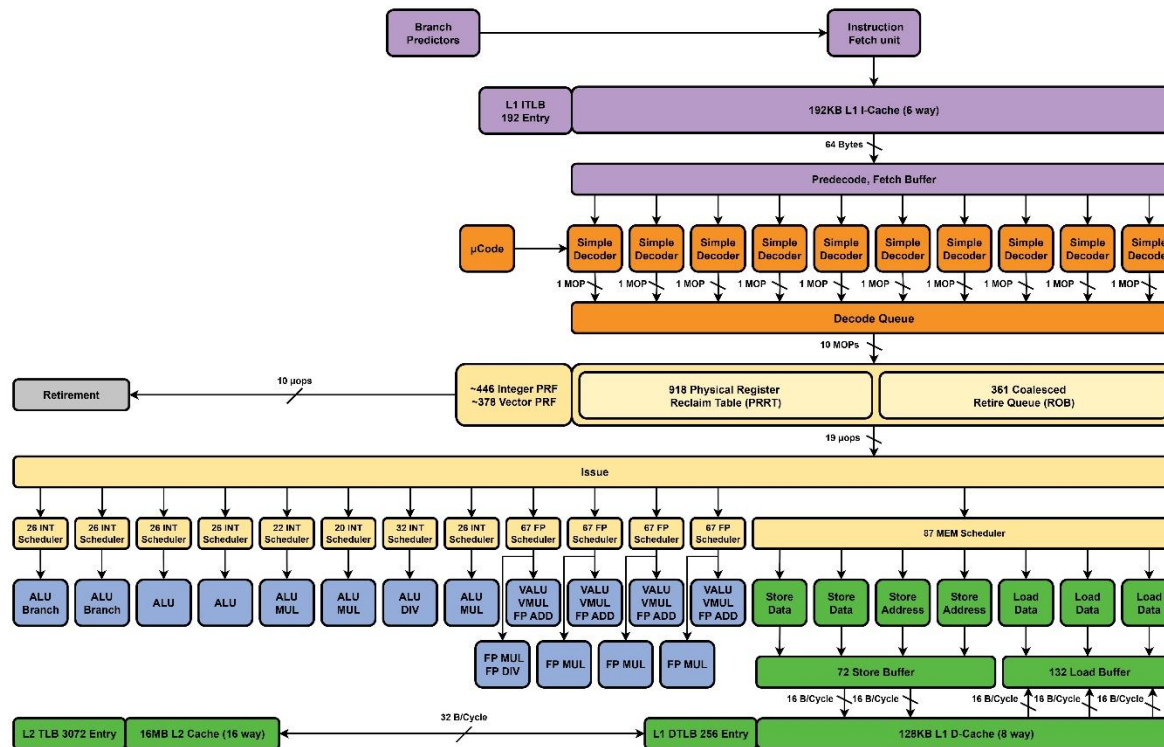
目的

- 以下を大ざっぱにでも理解してもらう
 - GPU は CPU とは何が違うのか？
 - なぜ GPU やアクセラレータは, AI において高速で効率が良いのか？

モジュール表す箱が色々描かれているが、ターゲットアプリが構造に与えている影響を説明したい

A18 P

Microarchitecture Block Diagram



- Cardyak's Microarchitecture Cheat Sheet より
https://docs.google.com/spreadsheets/d/18ln8SKIGRK5_6NymgdB9oLbTJCFwx0iFI-vUs6WFyuE/edit?gid=1076260513#gid=1076260513
- Rodrigo Huerta et al., Dissecting and Modeling the Architecture of Modern GPU Cores より <https://dl.acm.org/doi/10.1145/3725843.3756041>

目的と目標

- 目的：以下を大ざっぱにでも理解してもらおう
 - GPU は CPU とは何が違うのか？
 - なぜ GPU やアクセラレータは、AI で高速で効率が良いのか？
- 「GPU は CPU より単純な小さいコアをよりたくさん積んでいるため性能が高い」
 - しかし…
 - なぜ GPU では小さく単純にできる？
 - そもそも「小さい」とか「単純」とはどういうこと？
- これらの疑問に答えられるようにしたい

なぜ GPU などハードウェアの仕組みを学ぶのか？

■ 理解してほしいこと

- なにが得意なのか
- なぜその形になっているのか
- どうして効率が良いのか

■ 理解すると できること

- CPU と GPU の役割分担がみえる
- 製品の速度・電力効率を桁があってるぐらいのレベルで見積もれる
- 仕組みがわかるとより高速・高効率なソフトウェアが書ける

■ 直感を身につける

- 明らかに GPU (or CPU) でやるべきではない処理をやらない
- 新ハードの変なマーケティングに振り回されない

想定する聴衆と説明の方針

■ 想定：

- プログラムはある程度書いたことがある
- 半導体やアーキテクチャの事は基本あまり知らない

■ 方針：

- なるべく専門用語や概念を廃して，身近な例で説明したい
 - 水とカレー屋さんの話が続きます
 - （返ってややこしくなっているかもしれない
- 特定の製品などに着目するよりは，その土台にある部分を説明する
 - 将来に渡って，大きな基本はかわらない所の仕組みを言いたい

もくじ

1. 復習（長い）：
 1. 半導体における消費電力や効率
 2. CPU の基本
2. GPU のアーキテクチャの意味
3. アクセラレータ

土台になる部分を詳しく復習

半導体における消費電力や効率

現代のコンピュータの速度

- 速度は電力（とそれによって生じる熱）によってかなり律速されている
 - チップに供給できる電力はピン数などから一定
 - 排熱にも物理的な限界がある
- だいたい数百～1000 ワット程度を供給するのが限界
- 計算を行うために必要なエネルギーが、速度を決める重要な尺度となる
 - 同じ処理を行うのに必要なエネルギーが減れば、時間あたりの処理数が増える

消費電力や効率

- 回路の量と消費電力は（概ね）比例するが、なぜか？
- なぜ計算により電力が消費されるのか？
 - なぜ「情報」を処理すると電力が消費される？
 - 消費される電力の量はどのように決まる？
- これらを考える上で半導体の仕組みを復習

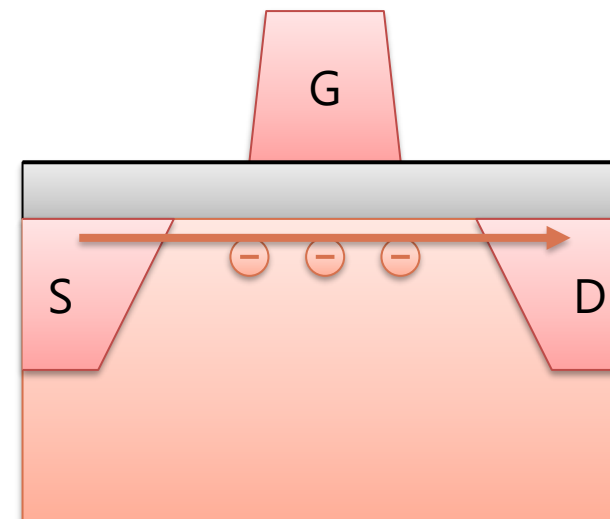
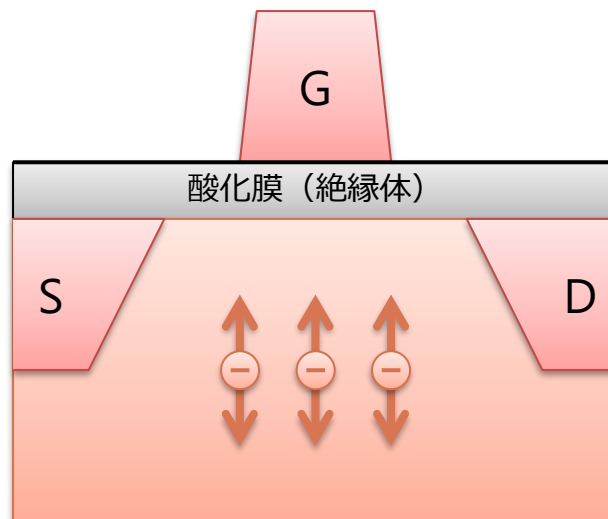
MOS FET

■ MOS: metal-oxide-semiconductor

- 上から, 金属, 酸化膜, 半導体のサンドイッチ構造
- 酸化膜を挟んで平行板コンデンサ様の構造が構成されている
 - 最近は酸化膜ではない, より誘電率が高い素材が使われる事も多い

■ 電界効果トランジスタ (FET: Field-Effect Transistor)

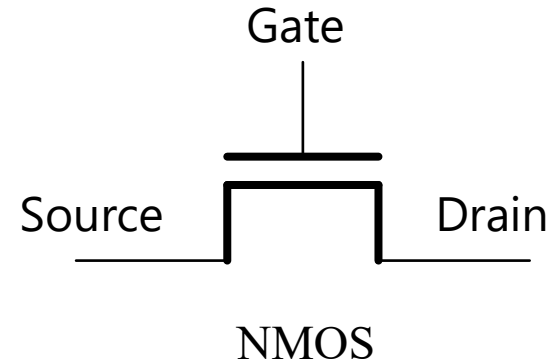
- 電界で電子を動かしてスイッチングする (NMOS の場合)
 1. G に電圧をかけてコンデンサに充電
 2. 電子の層 (チャネル) 酸化膜下にできて, S と D が繋がり ON に



MOS には NMOS と PMOS の2つがある

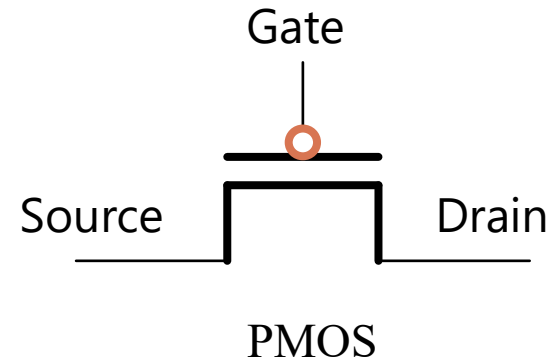
■ Negative (N)

- 電子がキャリア
- 低電位しか通せない
- 接地側に配置



■ Positive (P)

- 正孔 (hole) がキャリア
- 高電位しか通せない
- 電源側に配置
- Gate に○がついている

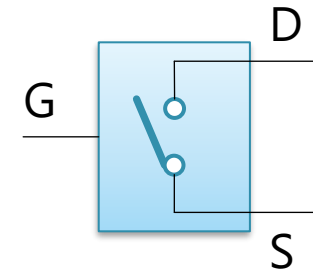
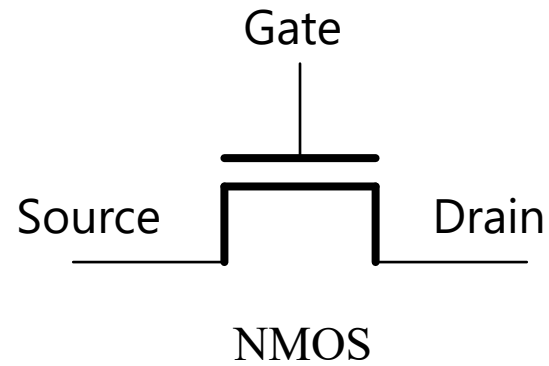


- この2つを相補的に組み合わせるので CMOS という

入力で ON/OFF が切り替わるスイッチと考える事ができる

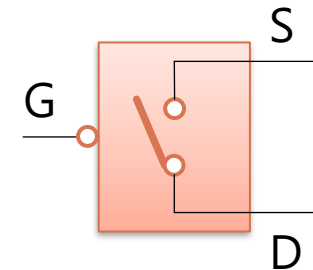
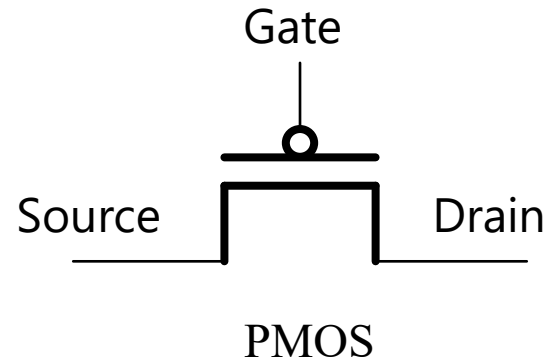
■ Negative

- Gate (G) を 1 にすると ON になる



■ Positive

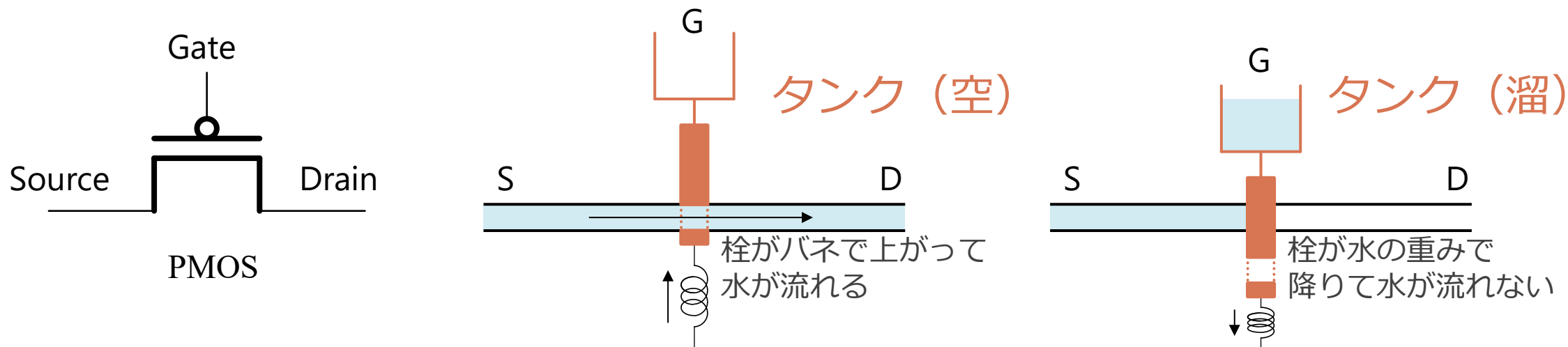
- Gate (G) を 0 にすると ON になる



だがしかし

- いきなり MOS とか言われても、何もわからないのではないかと
 - 消費電力はコンデンサへの充放電により説明されるが、コンデンサが何かとか憶えてない（or 知らない）のでは
- もうちょっと直感的な例を使ってイメージを掴みたい
 - ここでは水の流れに例えて説明
 - 技術的に正確ではないが、かなりイメージは一致する
 - 一次近似としては良くあっているはず
 - 消費電力のイメージもかなり伝えやすい
- 回路の量（大きさ）と、電力の関係のイメージを掴んでほしい

水の流れて例えることにする



■ 水流タンクシステム：

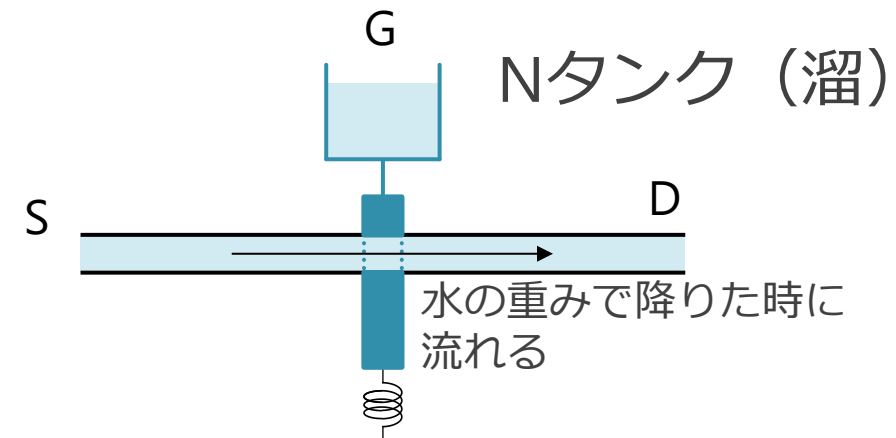
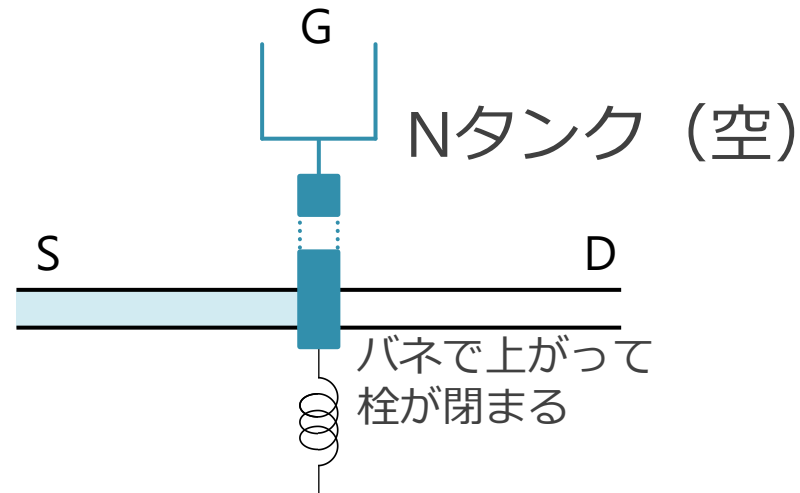
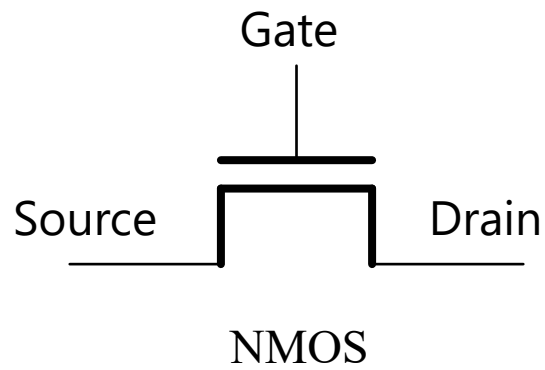
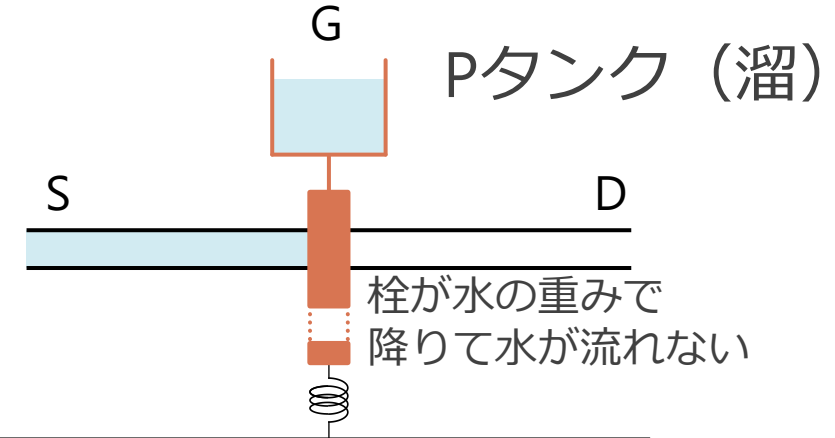
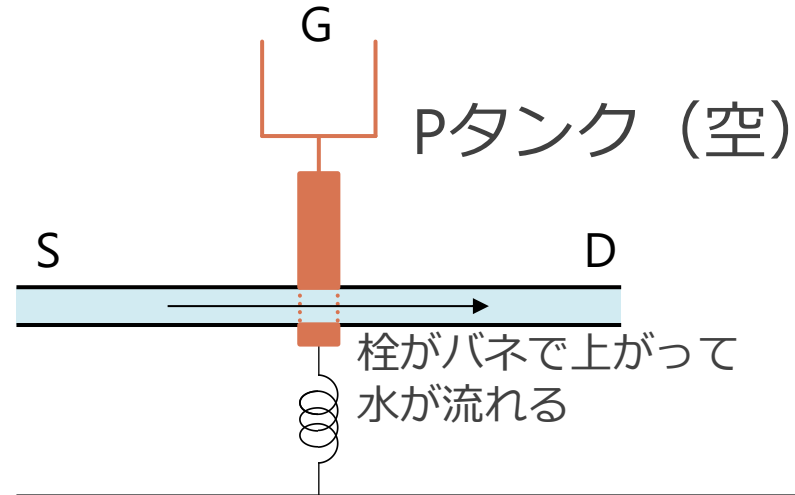
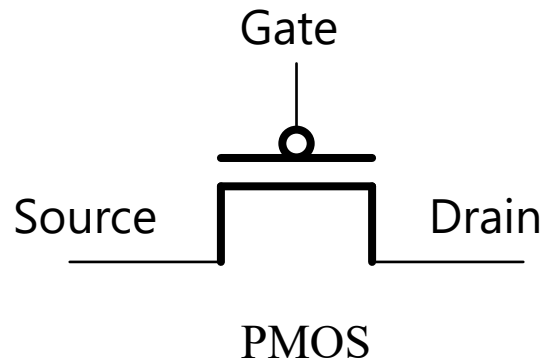
- 水が流れるパイプとタンク+栓を考える
- タンクに水がたまっているかどうかで連動して動く栓を持つ
 - 栓には水が通れる部分があって、水流を制御できる

■ PMOS と同様の挙動：

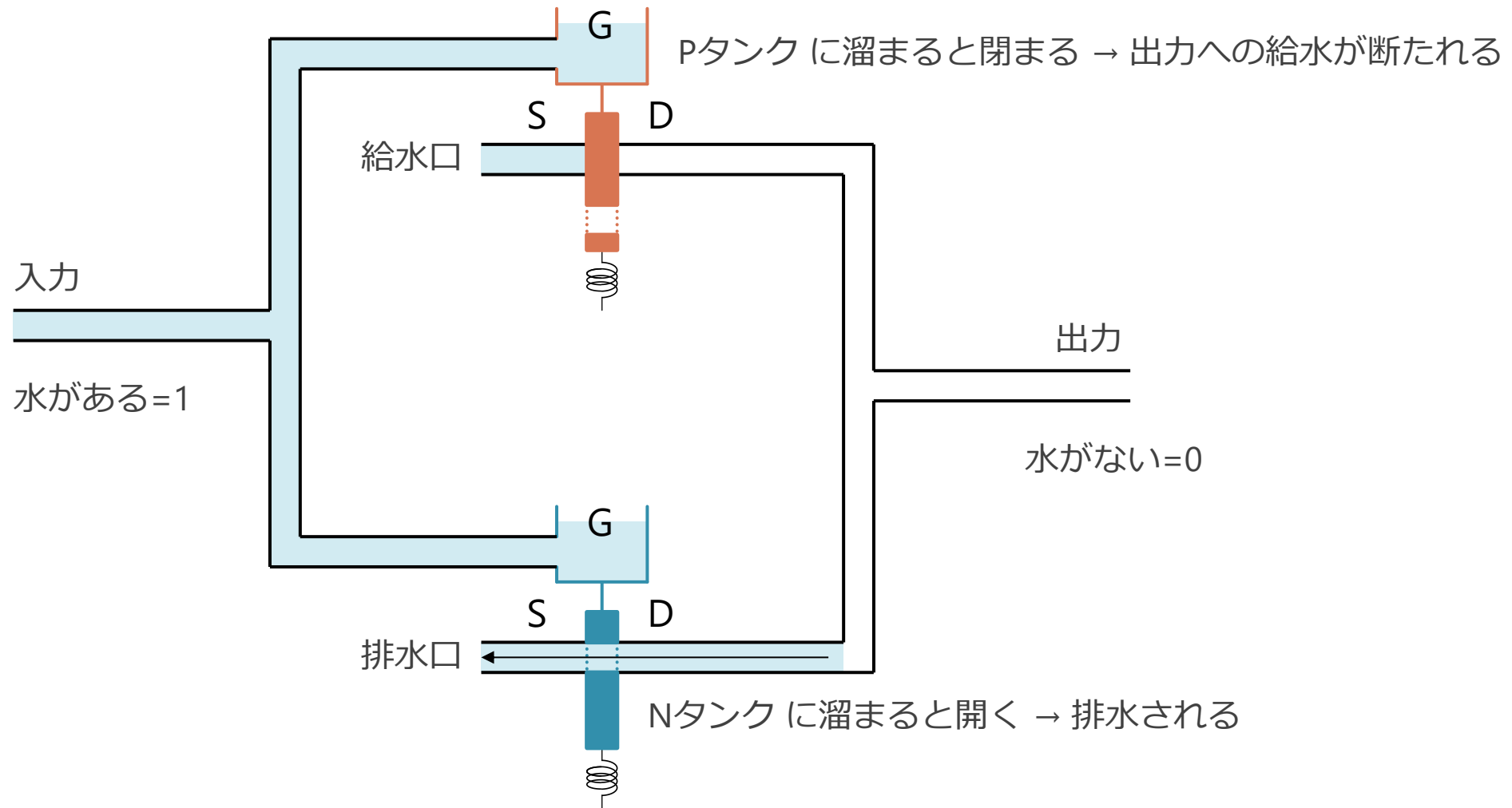
- 「タンクが空だと、S から D に水が流れる」
- 「電荷が Gate から抜けて空だと、S から D に電荷が流れる」

タンクは2種類ある (PMOS と NMOS に相当)

タンクに水が溜まっている時に、流れるか遮断されるかが逆になっている

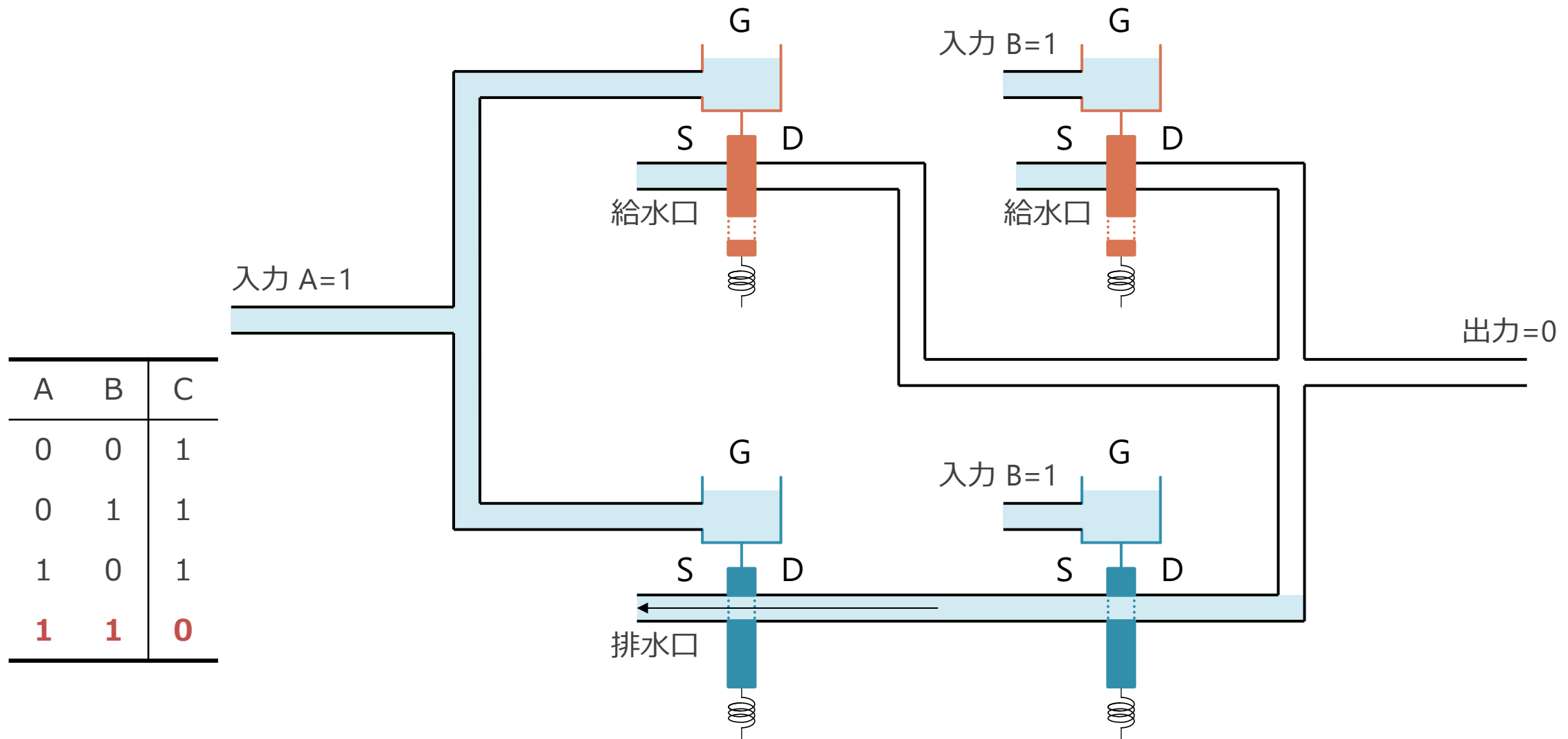


P タンクと Nタンクで NOT ゲートが作れる



- 給水口には常に一定の圧力で水が供給されている (=電源)
- 入力側の水の有無により, 出力側の水の有無が反転 → NOT ゲートの動作

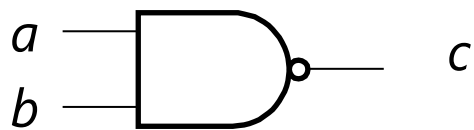
上側を並列繋ぎ、下側を直列繋ぎにすると NAND になる



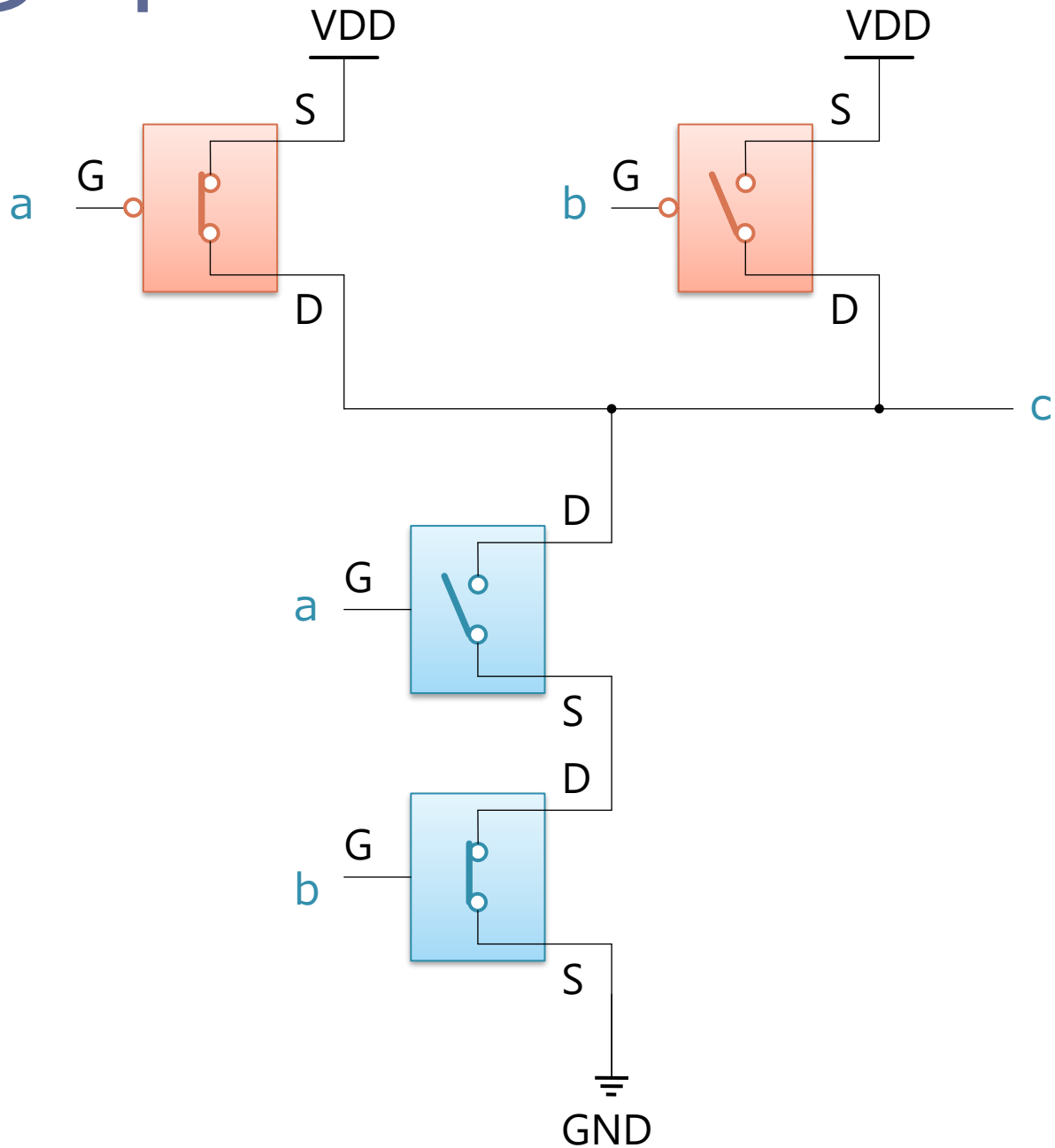
- 下側の2つのNタンクは、直列に繋がってるので双方溜まった時のみ排水が有効 → NAND

スイッチで書いたNAND ゲート

- 右のように接続すると NAND ゲートが作れる



a	b	c
0	0	1
0	1	1
1	0	1
1	1	0

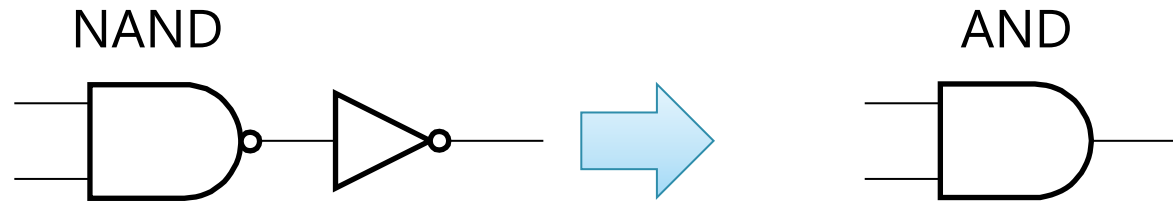


ここからしばらく余談

- ここまで：
 - スイッチ的なものがあると NOT と NAND が作れる
- なんらかの入力に従って ON/OFF できるスイッチがあれば, それでコンピュータが作れる

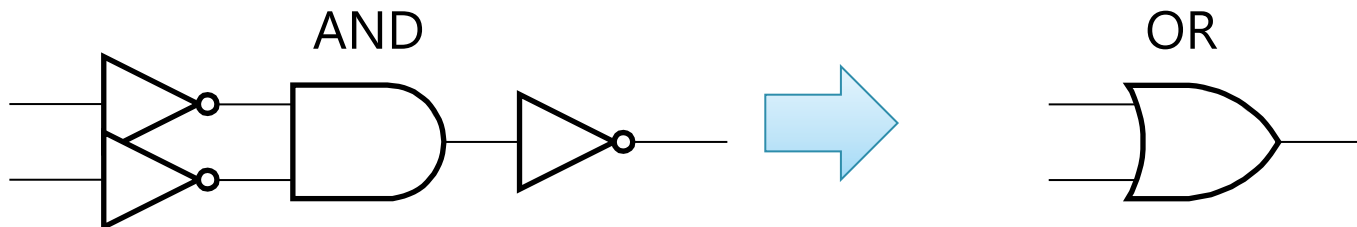
NOT と NAND から AND と OR を作る

- NAND の出力に NOT をつけると AND が作れる



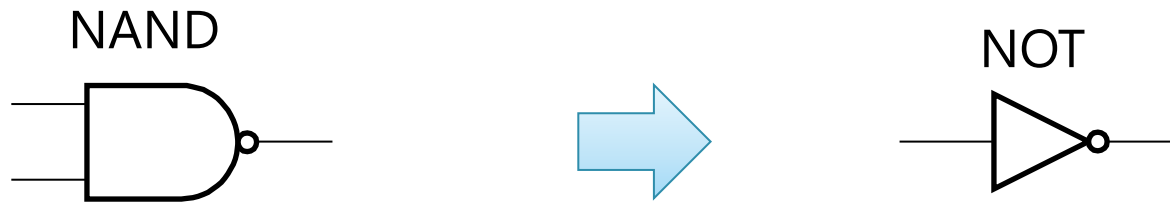
- AND の入力と出力に NOT をつけると OR が作れる

- ド・モルガンの定理 : $\text{NOT}(a \text{ OR } b) = \text{NOT}(a) \text{ AND } \text{NOT}(b)$
- 両辺に NOT を適用 : $a \text{ OR } b = \text{NOT}(\text{NOT}(a) \text{ AND } \text{NOT}(b))$



NAND が作ればそれだけでも良い

- NAND の 2 つの入力に同じものを入れれば NOT になる



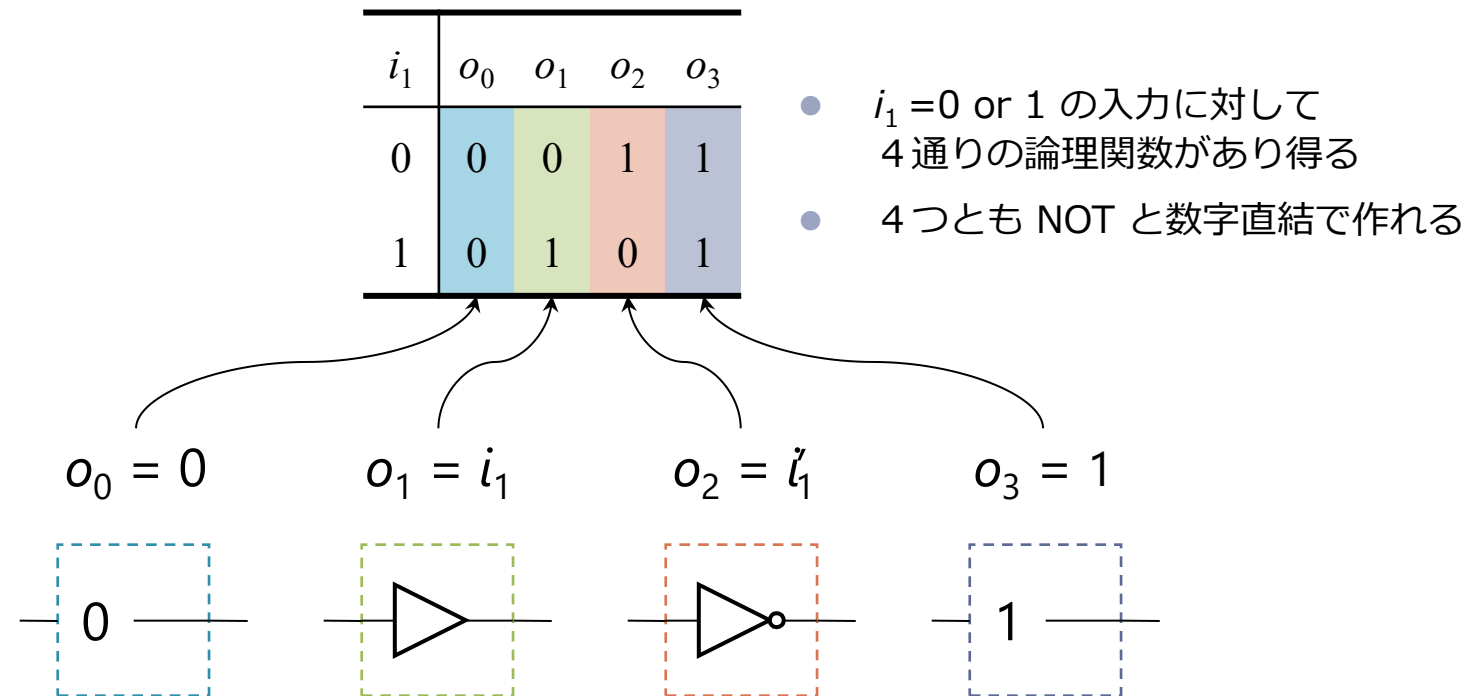
完全性 (Completeness, 完備性)

- AND, OR, NOT を組み合わせると任意の論理関数が作れる
- 完全集合 (Complete Set) :
 - その組み合わせによって、すべての論理関数を表現できる論理関数の集合
- 完全集合の例
 - {AND, OR, NOT}
 - {AND, NOT}
 - {OR, NOT}
 - {NAND}
 - {NOR}

完全性の証明 {AND, OR, NOT}

■ 数学的帰納法：

1. 1入力の論理関数は {AND, OR, NOT} の組合せで表現できる
2. n 入力の関数を {AND, OR, NOT} の組合せで表現できたと仮定して, $(n + 1)$ 入力の関数が表現できることをいう

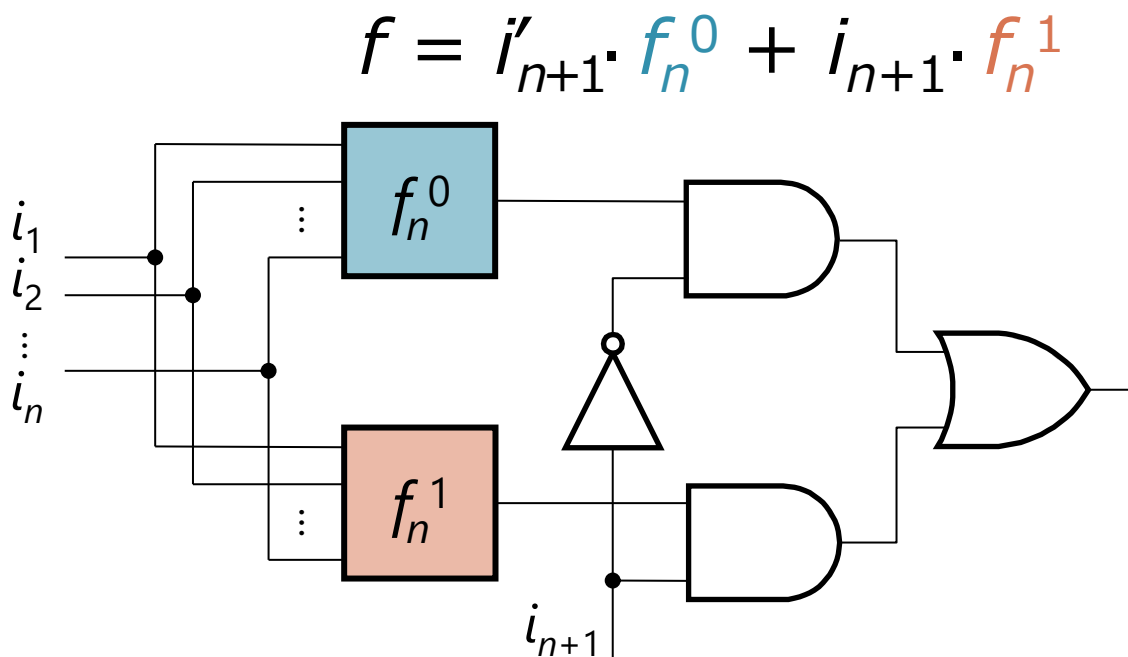


完全性の証明 {AND, OR, NOT}

■ 数学的帰納法：

1. 1入力の論理関数は {AND, OR, NOT} の組合せで表現できる
2. n 入力の関数を {AND, OR, NOT} の組合せで表現できたと仮定して,
($n + 1$) 入力の関数が表現できることをいう

i_{n+1}	i_n	...	i_2	i_1	o
0	0	...	0	0	
	0	...	0	1	
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
	1	...	1	1	
1	0	...	0	0	
	0	...	0	1	
	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
	1	...	1	1	



- 任意の論理関数 f_n^0, f_n^1 が表現できるので、それらの結果を i_{n+1} を使って選択
- この選択部分は AND/OR/NOT で作れる

スイッチからコンピュータを作る

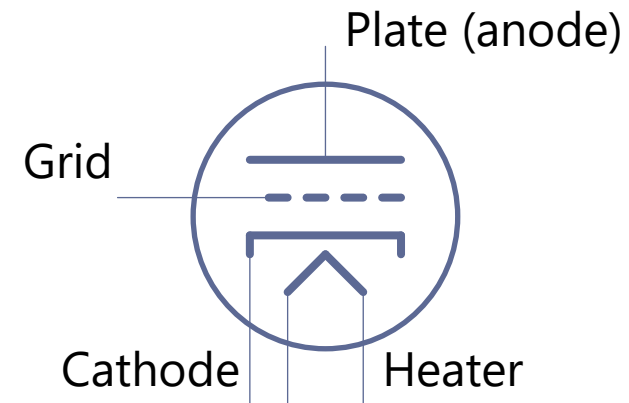
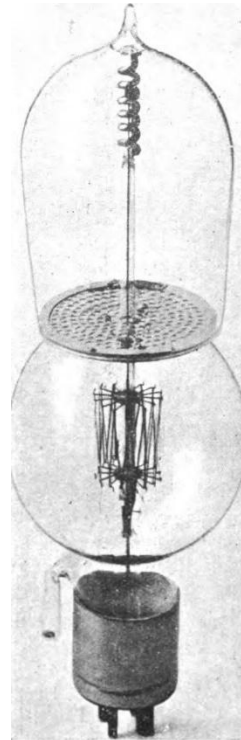
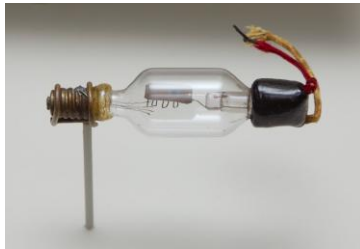
- スイッチから NOT, AND, OR を作る手順
 1. スイッチを組み合わせて NOT と NAND を作る
 2. NOT と NAND を組み合わせて AND と OR を作る
- NOT, AND, OR からコンピュータを作る手順
 1. NOT, AND, OR を使うと任意の論理回路が作れる
 2. 任意の論理回路を使うと CPU の部品が作れる
 - PC, レジスタ, 演算器, メモリ...
 3. それらを組み合わせるとコンピュータができる

つまり,

- なんらかの入力に従って ON/OFF できるスイッチがあれば,
それでコンピュータが作れる
 - 具体的な回路の組み方は, 回路の種別ごとに違う事もある
 - NAND さえ出来ればこっちのもの

真空管（三極管）

- 電球にカソードとグリッド, プレートなどの電極を追加したもの



画像は <https://en.wikipedia.org/wiki/Triode> より
De Forest Audion tube と Lieben-Reisz tube
1900 年過ぎの発明

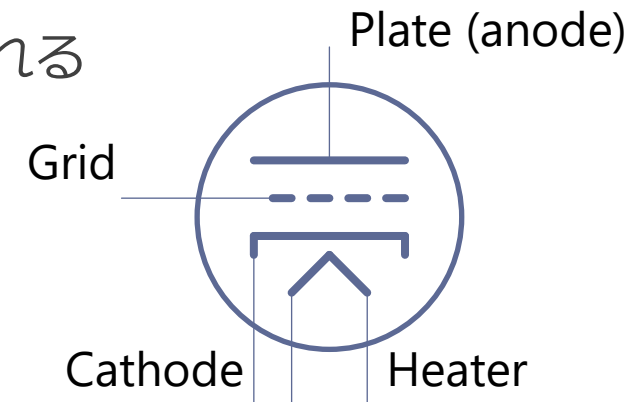
真空管の動作

■ 動作：

1. Plate に正の電圧を，Cathode に負の電圧をかける
2. Heater（電球のフィラメント）に電源を接続
3. 暖められた Cathode が電子を放出
4. Plate に電子が吸い寄せられる
 - 電子が移動する = それらの間に電流が流れる

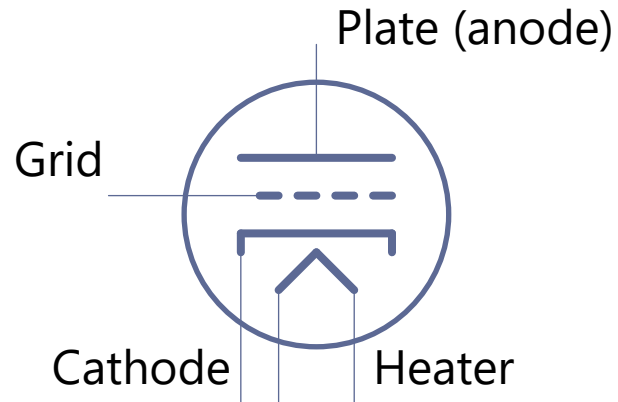
■ スイッチング

- なにもしなければ Grid は網なので電子はすき間を通過
- Grid に負の電圧をかける
- 負同士ではじきあうので，電子が上にいけなくなり電流が OFF

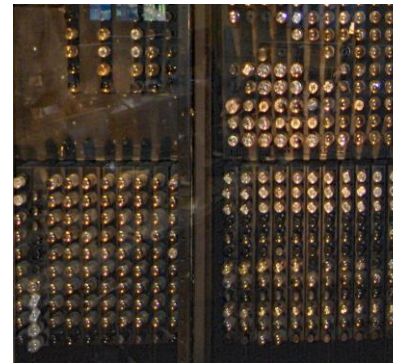
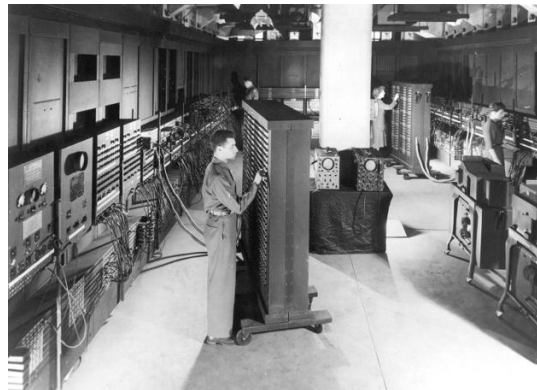


真空管

- Grid にかける電圧で, Cathode と Plate を ON/OFF できる
 - つまり, これでもコンピュータが作れる



- ENIAC (1945)
 - 世界で最初の汎用コンピュータ
 - 18,000 個の真空管で出来ている



レッドストーン回路

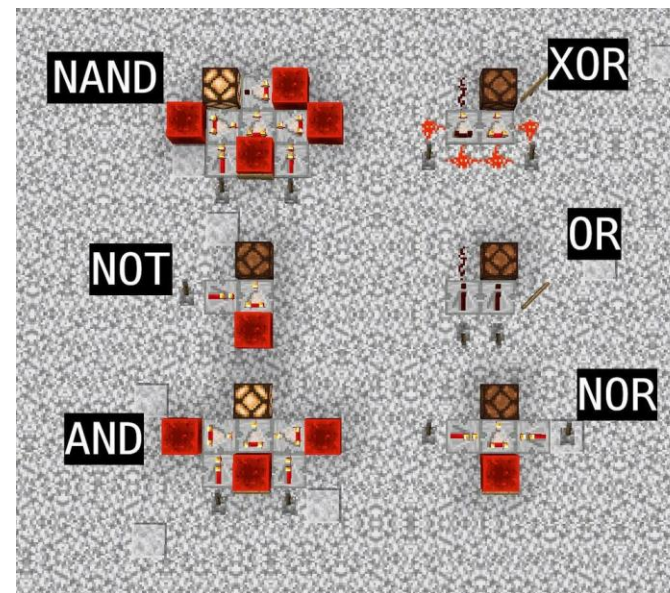
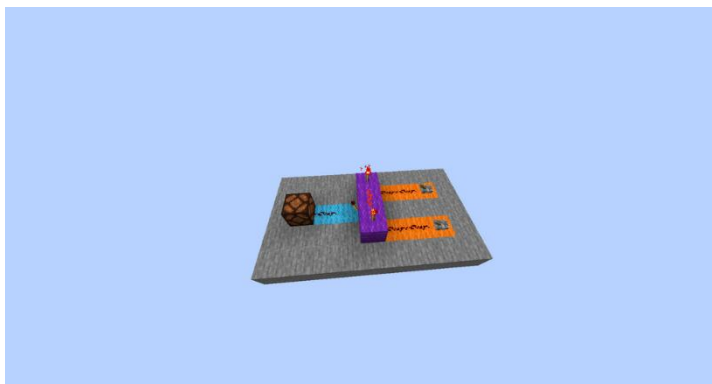
- Minecraft というゲームの中で作られている回路
 - <https://www.minecraft.net/ja-jp>
 - もともとは箱庭でブロックをおいて色々つくって遊ぶゲーム



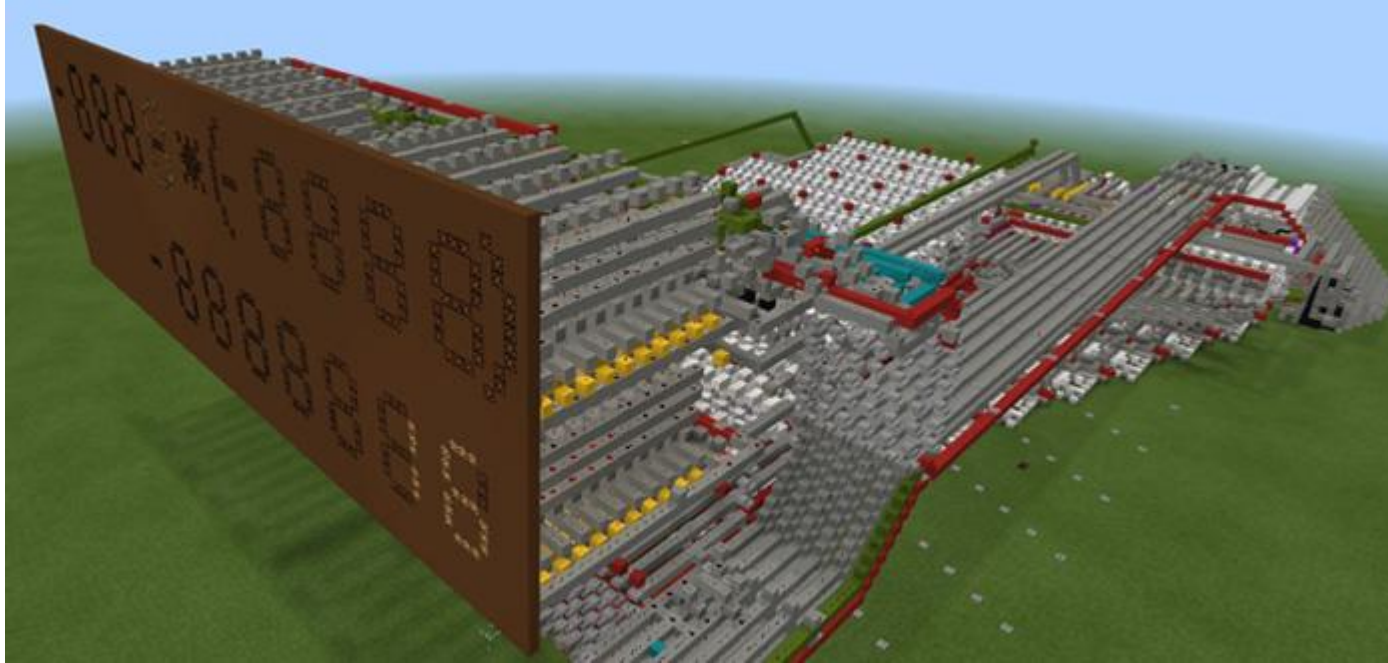
レッドストーン回路

- 「レッドストーン」というゲーム内の要素を利用
 - 有効と無効の状態を持つ
 - スイッチと扉を繋げて、遠くから開けたり閉じたりとかする
- これも入力で出力を制御出来ていることに誰かが気づいた

AND ゲート
右のスイッチを2つとも入れると
左側が赤くなる



入力で出力を制御出来ればコンピュータが作れる

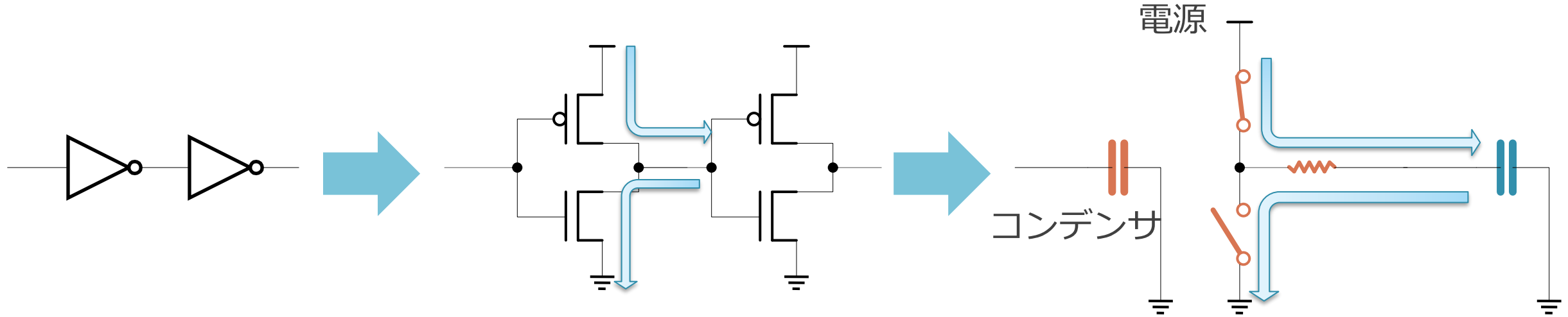


画像は <https://mcpedl.com/raffis-calculator-2-map/> より
四則演算ができる計算機・・・らしい

もくじ

1. CMOS と水流システム
2. スイッチがあればコンピュータが作れる
3. **CMOS と水流システムの消費エネルギー**

CMOS 回路におけるスイッチングの消費エネルギー

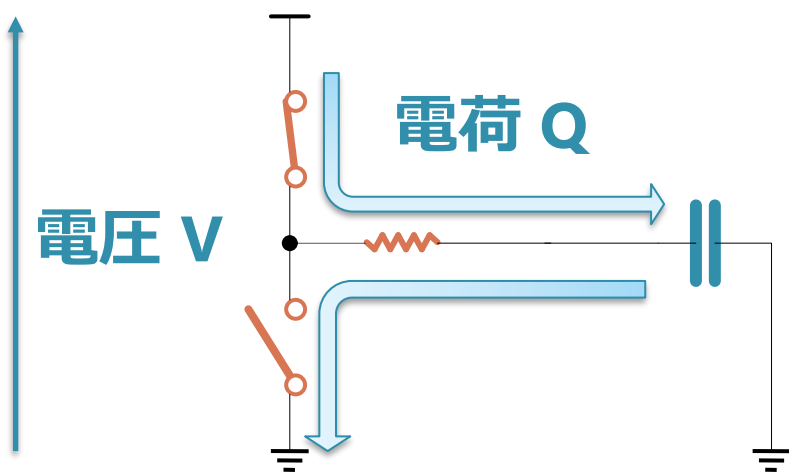


- CMOS 回路は、コンデンサに連動したスイッチとみなせる
(MOS 構造のゲートのところがコンデンサになってる)
 - コンデンサが充電状態：下のスイッチがON
 - コンデンサが放電状態：上のスイッチがON
- 主に計算で消費されるエネルギー：
 - スイッチを ON/OFF するためのコンデンサへの充放電で消費

これも水流で考える

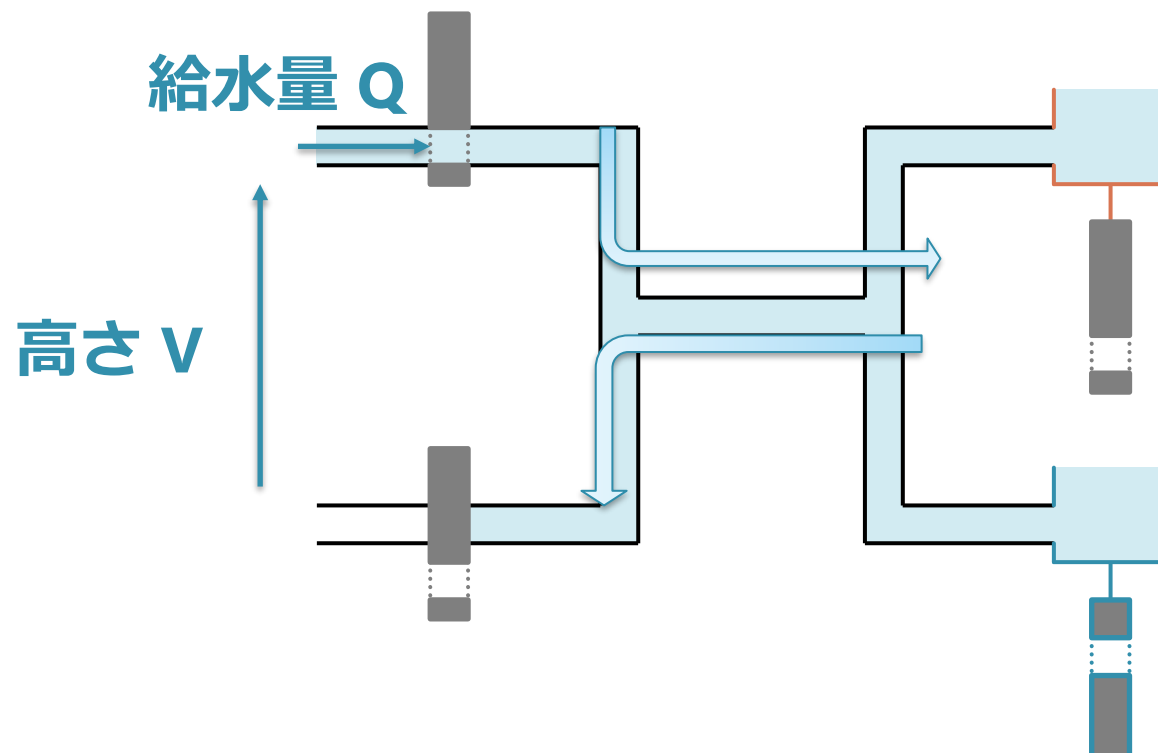
■ 電流を流して失われるエネルギー

- コンデンサに溜まる
電荷の量 : Q
- 電圧 : V
- 消費エネルギー : $Q \times V$

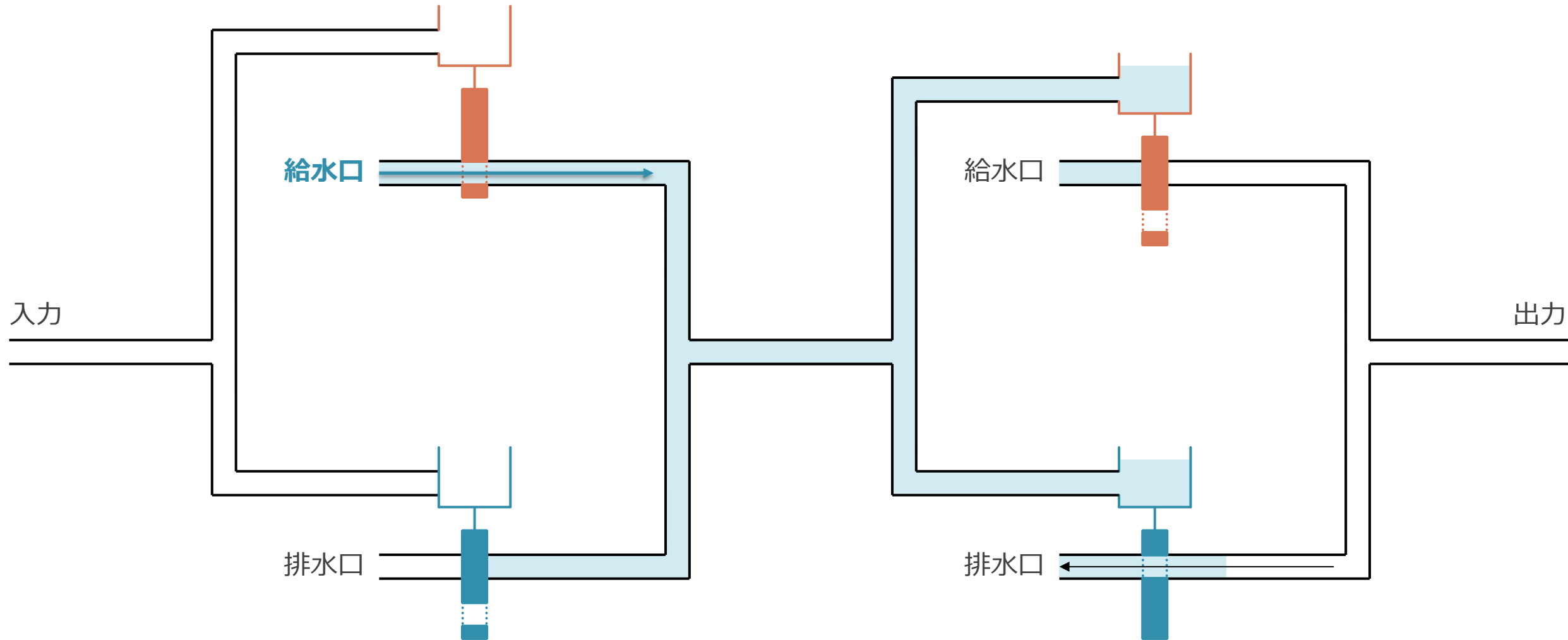


■ 水を高いところから流すとその分位置エネルギーが失われる

- 水の総量 : Q
- 排水口から給水口高さ : V
- エネルギー : $Q \times V$

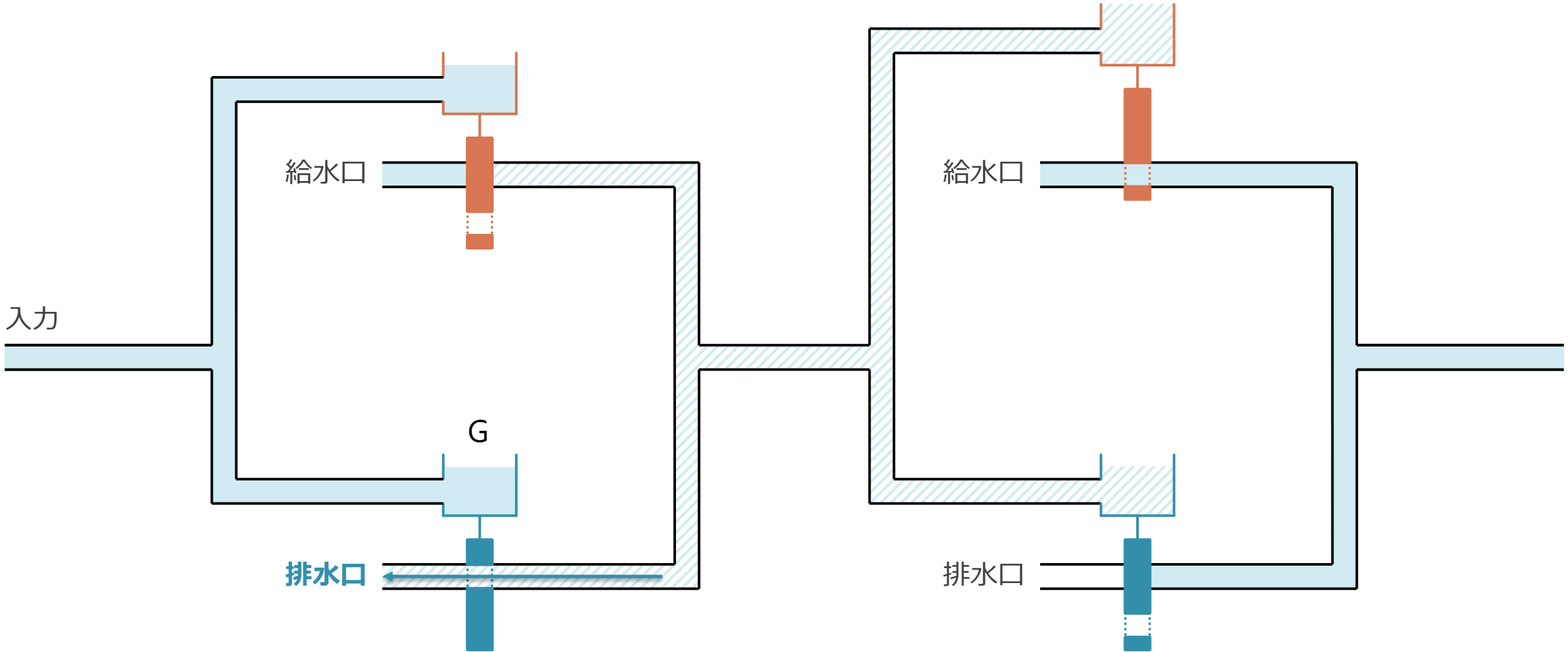


NOT ゲートを 2 段重ねた例：入力が 0 の場合



- 左上の給水口から，右側 2 つのタンクに給水される

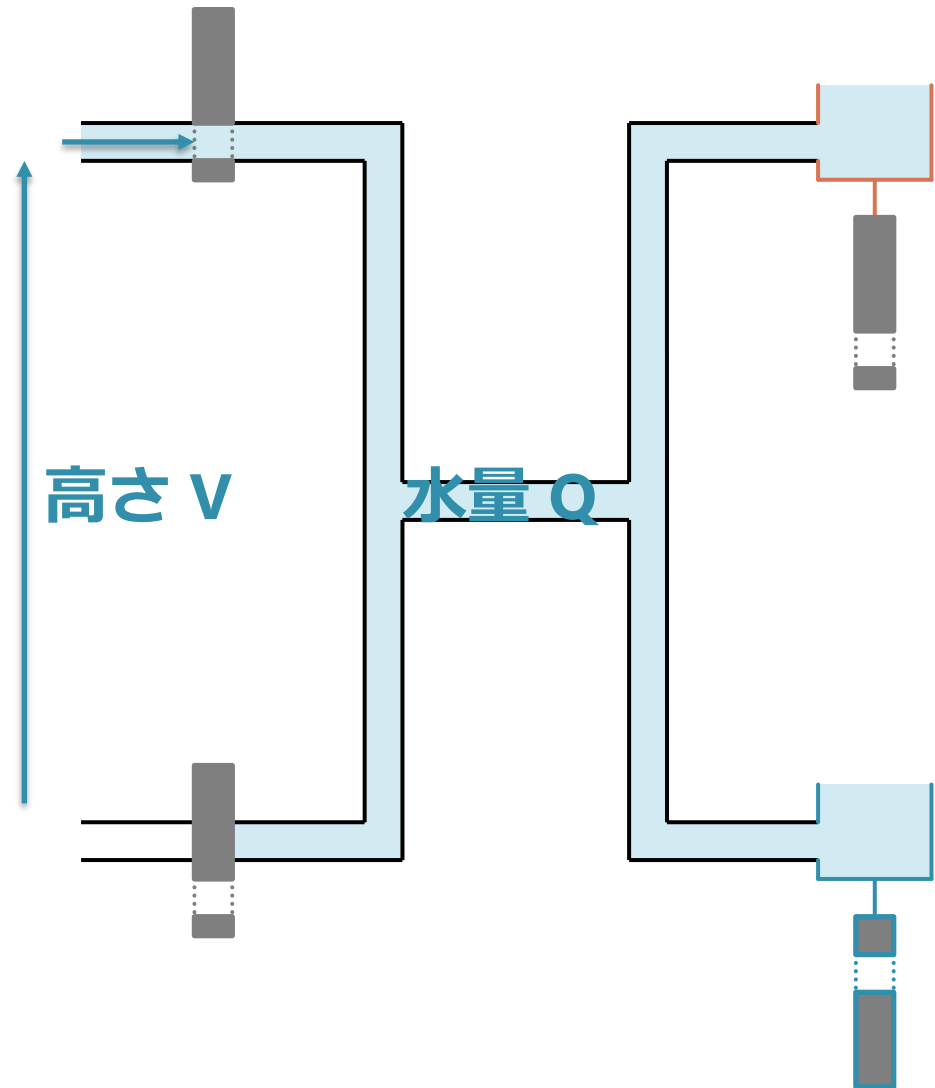
入力が 0 → 1 に遷移すると、
中央と右側タンクの水（斜線部分）が排水される



- 入力に水がくると、右側の2つのタンクにあった水が排水される

つまり、スイッチの切り替えが起きると・・・

- スイッチングが起きると・・・
 - 高さ V の給水源から水量 Q が供給
 - 総量 Q の水がタンクとパイプに入って抜ける
- 入力の変化に伴い、水の出し入れが生じるとエネルギーが消費される
 - 電荷が電源から GND に流れるのも同じ
- 水を高いところから流すとその分位置エネルギーが失われる
 - タンクにたまった水の総量： Q
 - 排水口から給水口高さ： V
 - エネルギー： $Q \times V$



補足：スイッチの ON/OFF 以外による消費エネルギー

- その他に、リーク電流と呼ばれるものもある
 - トランジスタを OFF にしていても、流れ続けてしまう電流
- 分類：
 - 充放電によるもの：動的（dynamic）消費電力
 - リークによるもの：静的（static）消費電力
- リークは、栓が完璧ではないのでチロチロ水が漏れているイメージ

水流システムのアナロジーから説明できること

■ 消費エネルギー：

- 回路：全 PMOS/NMOS のコンデンサに溜まる電荷量 Q と、電圧 V によって決まる
- 水流：全タンクにたまる総水量 Q と、高さ V によって決まる

■ 回路量に比例して消費エネルギーは増える：

- 水流：全タンクに溜まる総水量はタンクの数に比例する

■ 配線も電力を消費する：

- 回路：配線が持つ寄生容量に充放電が起きる
- 水流：パイプにも水は多少溜まる

■ 半導体化が微細化すると電力や速度も向上する：

- 回路：コンデンサに電荷をためる量と時間が減る
- 水流：タンクが小さくなるので、水が貯まる時間と量が減る

これらの話は、基本的には半導体技術が進んでも変わらない

図は IEEE INTERNATIONAL ROADMAP FOR DEVICES AND SYSTEMS (IRDS) EXECUTIVE SUMMARY 2022 より

モデル名						
YEAR OF PRODUCTION	2022	2025	2028	2031	2034	2037
Logic industry "Node Range" Labeling	G48M24	G44M20	G42M16	G40M16 T2	G38M16 T4	G38M16 T6
Fine-pitch 3D integration scheme	Stacking	Stacking	Stacking	3DVLSI	3DVLSI	3DVLSI
Logic device structure options	finFET LGAA	LGAA	LGAA CFET-SRAM	LGAA-3D CFET-SRAM	LGAA-3D CFET-SRAM	LGAA-3D CFET-SRAM
Platform device for logic	finFET	LGAA	LGAA CFET-SRAM	LGAA-3D CFET-SRAM-3D	LGAA-3D CFET-SRAM-3D	LGAA-3D CFET-SRAM-3D
実際のサイズ						
LOGIC DEVICE GROUND RULES						
Mx pitch (nm)	32	24	20	16	16	16
M1 pitch (nm)	32	23	21	20	19	19
M0 pitch (nm)	24	20	16	16	16	16
Gate pitch (nm)	48	45	42	40	38	38
Lg: Gate Length - HP (nm)	16	14	12	12	12	12
Lg: Gate Length - HD (nm)	18	14	12	12	12	12
Channel overlap ratio - two-sided	0.20	0.20	0.20	0.20	0.20	0.20
Spacer width (nm)	6	6	5	5	4	4
Spacer k value	3.5	3.3	3.0	3.0	2.7	2.7
Contact CD (nm) - finFET, LGAA	20	19	20	18	18	18
Device architecture key ground rules						
Device lateral pitch (nm)	24	26	24	24	23	23
Device height (nm)	48	52	48	64	60	56
FinFET Fin width (nm)	5.0					
Footprint drive efficiency - finFET	4.21					
Lateral GAA vertical pitch (nm)		18.0	16.0	16.0	15.0	14.0
Lateral GAA (nanosheet) thickness (nm)		6.0	6.0	6.0	5.0	4.0
Number of vertically stacked nanosheets on one device		3	3	4	4	4
LGAA width (nm) - HP		30	30	20	15	15
LGAA width (nm) - HD		15	10	10	6	6
LGAA width (nm) - SRAM		7	6	6	6	6
Footprint drive efficiency - lateral GAA - HP		4.41	4.50	5.47	5.00	4.75
Device effective width (nm) - HP	101.0	216.0	216.0	208.0	160.0	152.0
Device effective width (nm) - HD	101.0	126.0	96.0	128.0	88.0	80.0
PN separation width (nm)	45	40	20	15	15	10

- CMOS の作り方は今後色々変わっていく
 - 色々な3次元構造が導入される予定
 - チップ上にトランジスタを積み上げる事で数を増やす
- 作りが変わっても，電力や速度の考え方は基本的に変わらない

Notes: Mx—Tight-pitch routing metal interconnect IDM—independent device manufacturer FinFET—fin field-effect transistor LGAA—lateral gate all around EUV—extreme ultraviolet NA—numerical aperture Ge—germanium SiGe—silicon germanium RMG—replacement metal gate VLSI—very large-scale integration W2W—wafer to wafer D2W—die to wafer Mem-on-Logic—memory on logic

Figure ES28

Devices will continue to aggressively scale in the next 5 years

(余談 1) 微細化による半導体世代の「定義の進化」

- 半導体の世代を表す「2nm」などの言葉は「進化」してきた
 - 最初はちゃんと物理サイズを表していた
 - いろいろあって、前の世代の数字に単に 0.7 をかけたものになった
 - 世代の数字が予定通り進まないで格好がつかないため
 - 注意深く見ると企業は「2nm」とは言わず「2N」「2 ナノ世代」等と表記している
- 「現在の半導体は 2nm 程度のサイズであり、これは原子数個分で～」のような表現
 - 「2nm の半導体」は、実際には別にどこも 2nm ではない
 - ただのモデルナンバーなので、そこまで縮んでない
 - 物理的にはまだ 14nm ぐらい
 - 原子サイズの限界まではまだある程度余地がある

図と説明は IEEE INTERNATIONAL ROADMAP FOR DEVICES AND SYSTEMS (IRDS) EXECUTIVE SUMMARY 2022 より。以下は日本語版。

https://irds.ieee.org/images/files/pdf/2022/2022IRDS_ExecutiveSummary_JP.pdf

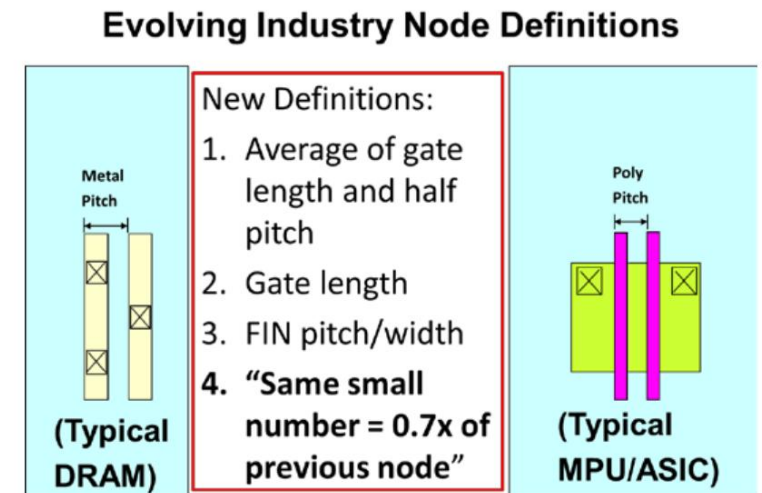


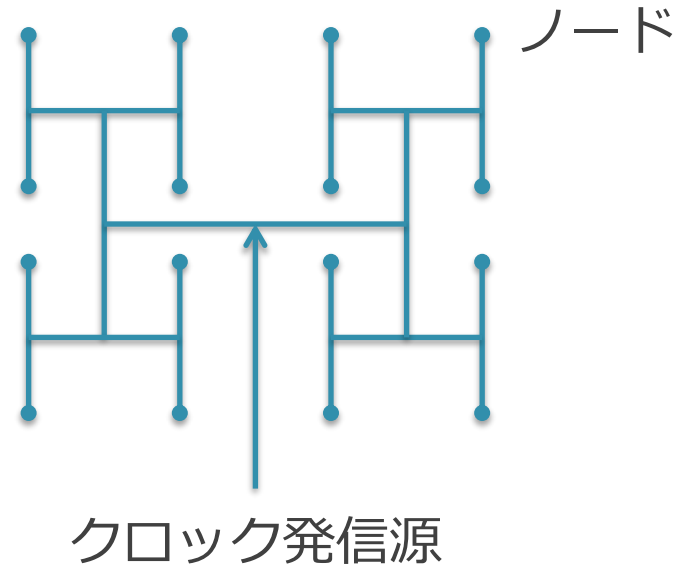
Figure ES26

Industry “adaptation” of technology node definition

(余談2) クロックによる消費エネルギー

- クロック信号：
 - 回路の動作の同期をとるための信号で，回路全体に配られる
 - したがって，クロック供給のための配線は長大
 - 配線も寄生コンデンサを作るので，そこで充放電が起きる
- クロック信号線では，配線の総延長がすごいことになる
 - すべての部分が単一のクロック発信源まで接続されている
 - クロック発信源から各部まで配線長が等しくなるように配置
- ただ同期を取るためだけに，チップ内の電力の数割が消費されている
 - クロック信号線全体に電荷を入れて抜くを繰り返している

クロックの配線方法の例：H-TREE



■ H-TREE

- ◇ クロック発信源から各ノードへの配線長が全て等しくなる
- ◇ しかしその分、物理的に近いノードであっても遠回りになる

H-TREE による配線の例 : IBM Power PC

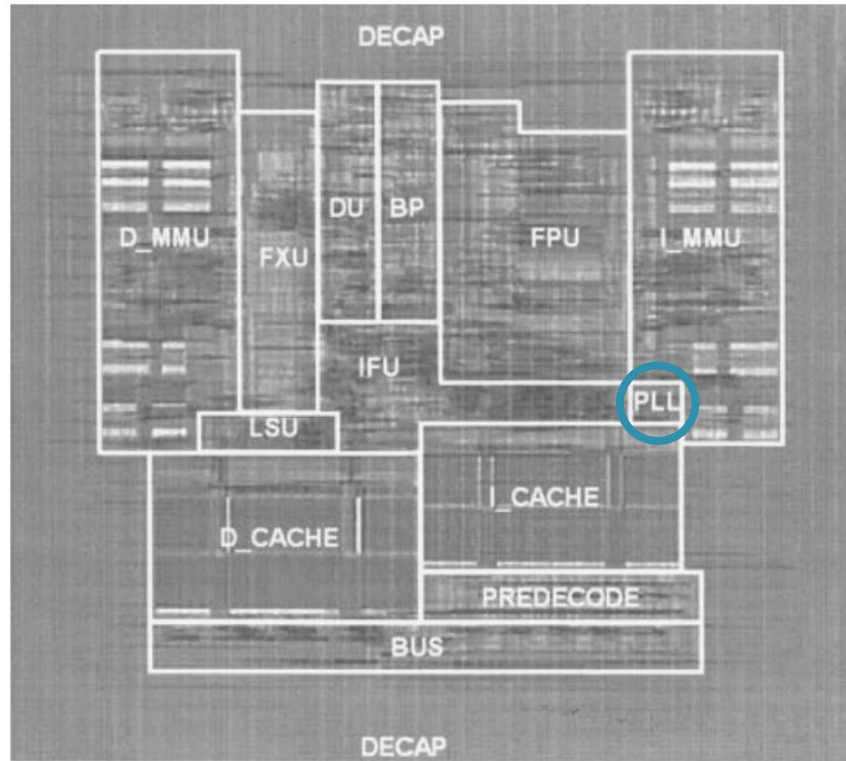


Figure 5.4.2: Die micrograph and floorplan.

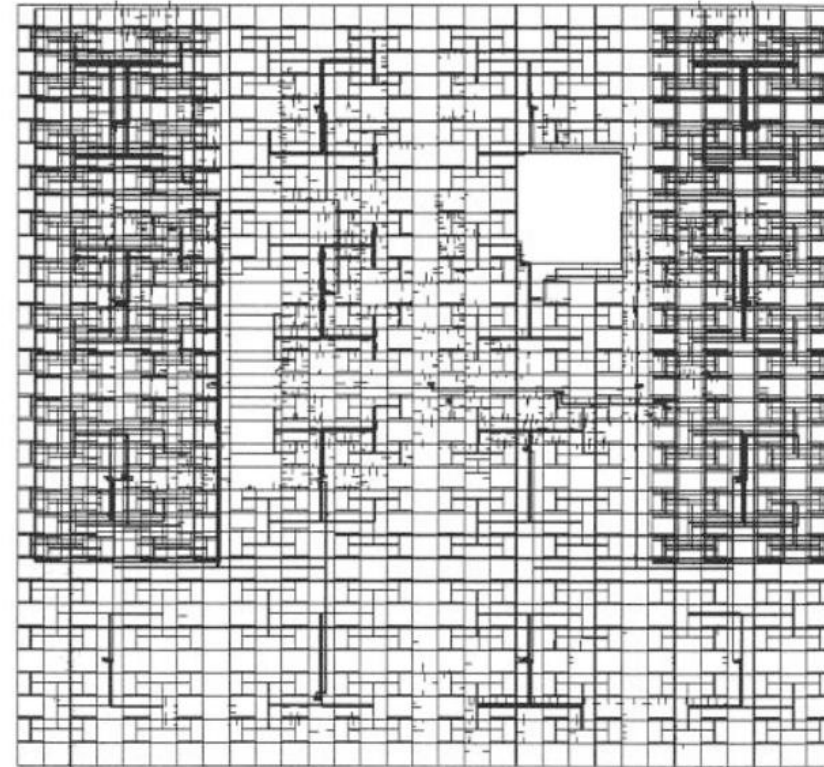


Figure 5.4.7: Clock distribution.

- P. Hofstee, N. Aoki, D. Boerstler, P. Coulman¹, S. Dhong, B. Flachs, N. Kojima, O. Kwon, K. Lee, D. Meltzer², K. Nowka, J. Park, J. Peter, S. Posluszny, M. Shapiro³, J. Silberman², O. Takahashi, B. Weinberger, MP 5.4 A 1GHz Single-Issue 64b PowerPC Processor, ISSCC 2000 より

図は以下より：

KIM, Youngchan; KIM, Taewhan. Algorithm for synthesis and exploration of clock spines. In: 2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC). IEEE, 2017. p. 263-268.

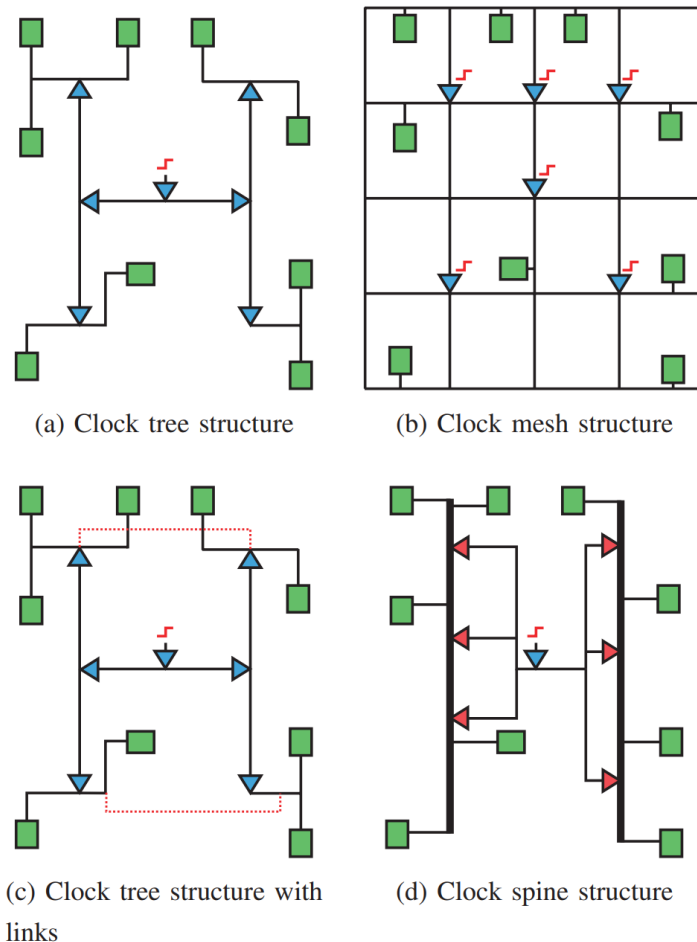
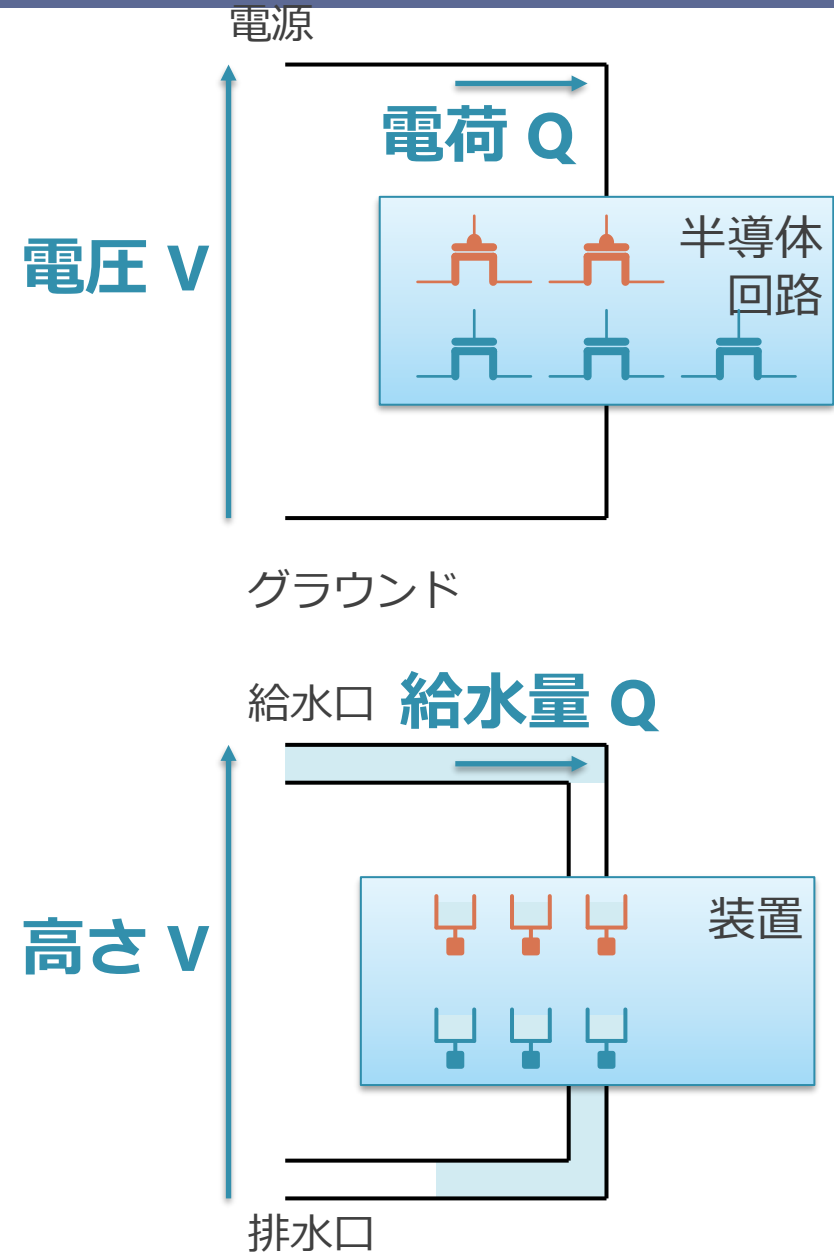


Fig. 1. Two extreme clock network structures are shown in (a) and (b). Two intermediate clock network structures are shown in (c) and (d). Clock spine structure in (d) delivers clock signal from clock source to clock sinks through top-level clock tree, clock spines, and stubs. At least two spine buffers have to be placed on each clock spine to maintain the tolerance to clock skew variability.

- skew を無くすためには、クロックネットワーク全体で電圧のズレがなければ良い
- clock grid：メッシュ状の配線を使う
 - ノード同士を密に縦横で繋いでやれば電位差が生まれにくい
- clock spine：太い配線を幹にして、そこから配る
 - 太い線の中では差が生まれにくいので、そこから配る

ここまでのまとめ

- CMOS 半導体の消費電力：
 - 電荷の充放電と電圧により主に決まる
 - システムに流した水の量と高さのイメージ
- 消費電力は回路の大きさに比例する
 - 充放電される電荷の量は，回路の量に比例
 - 水タンクの総数や総量が大きくなるイメージ
- 消費電力は動作周波数にも比例する
 - 充放電回数が比例するから
 - 水を入れて抜く回数が増えるとエネルギーが増えるイメージ



CPU の基本

コンピュータ

- 今主流のものは、みな基本的には「ノイマン型アーキテクチャ」に従う
 - John von Neumann (1903年 - 1957年)
 - (写真は Wikipedia より)



コンピュータ

- 「プログラム = 命令列 = 数字の列」に従って計算をする機械
 - （「ノイマン型アーキテクチャ」
- CPU を使うコンピュータの構成要素：
 - CPU
 - 演算器
 - レジスタ
 - PC
 - メモリ（データを記憶するもの）

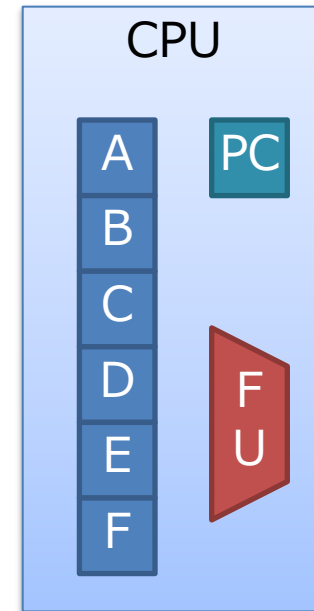
CPU

■ コンピュータの心臓部

- メモリから命令を読み出し，計算する

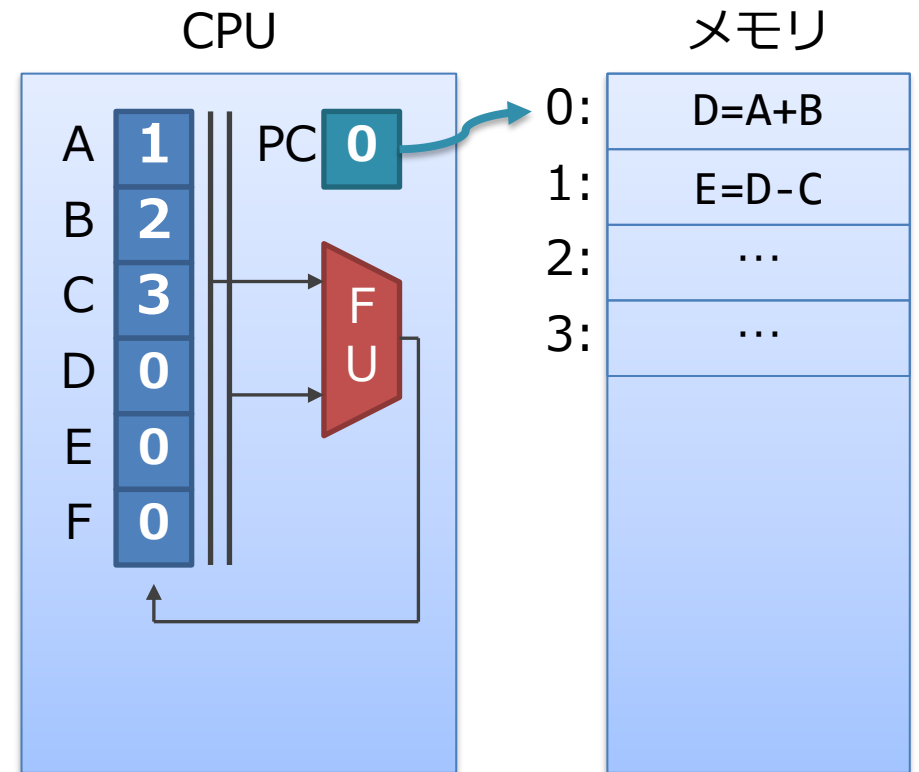
■ 構成要素：

- 演算器 (FU: Functional Unit)
 - 加算器や AND 演算器など
 - 指示された種類の演算を行う
- レジスタ・ファイル (右図では A,B,C...)
 - データを記憶し，位置を指定して読み書きする
 - CPU の演算は，このレジスタ上でのみ行う
- PC (Program Counter)
 - 現在見ている命令のアドレスを記憶している場所



CPU の動作

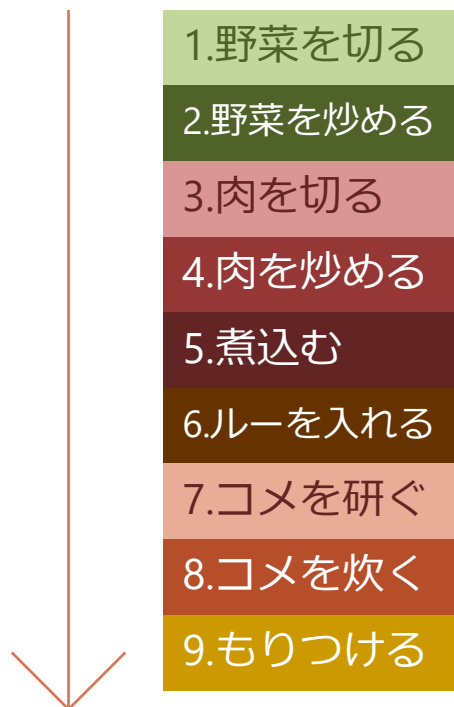
- PC (Program Counter) :
 - 現在処理する命令のアドレスを保持
- おおざっぱな命令の処理 :
 1. PC が指すアドレスのメモリから読む
 2. 読んできた命令に応じて処理をする
 3. PC を更新 (数字をたす)
 4. 1. にもどる



CPU の動作は料理に似ている

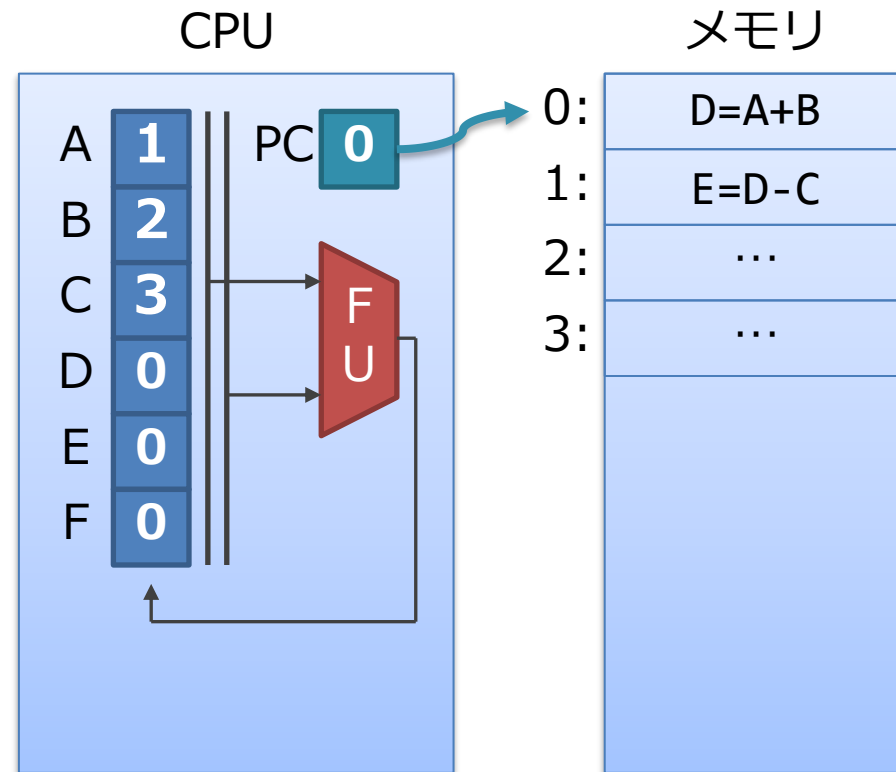
- レシピをみながら料理をするのに似ている
 - レシピの各手順が命令
 - 「今何個目の手順を見てるか」, を憶えているのが PC
 - ひとつひとつ手順を取り出して, 指示に従って処理

カレーのレシピ



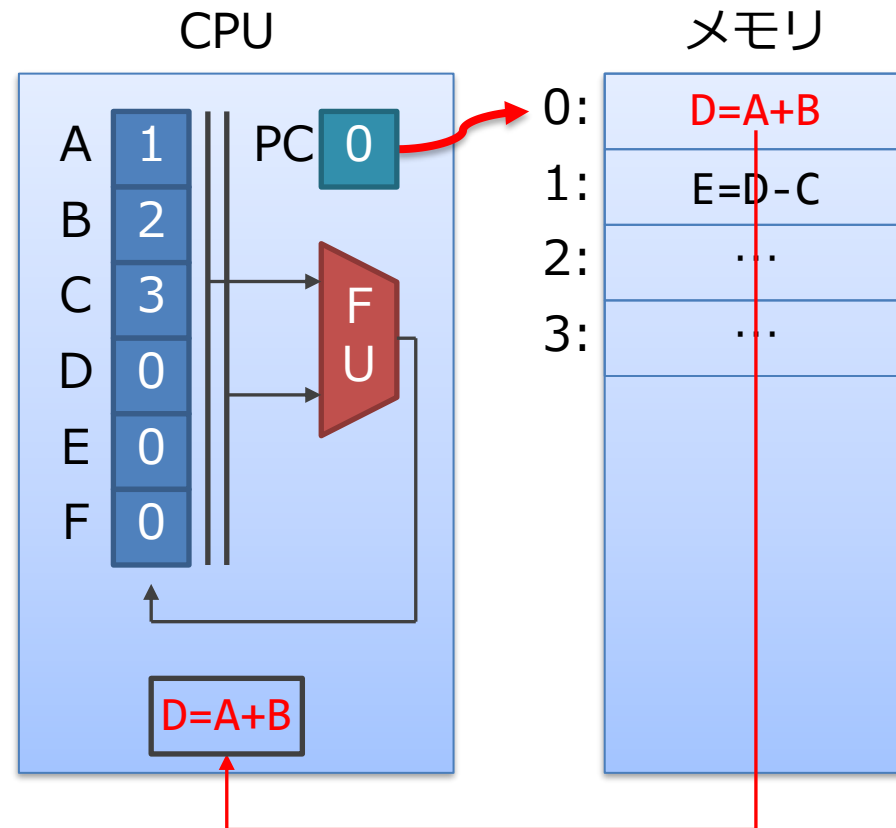
0. 初期状態

- PC はアドレス 0 を指している
- レジスタの初期値は1,2,3...
- メモリには, 以下の命令がある
 - 0 番地に $D=A+B$
 - 1 番地に $E=D-C$



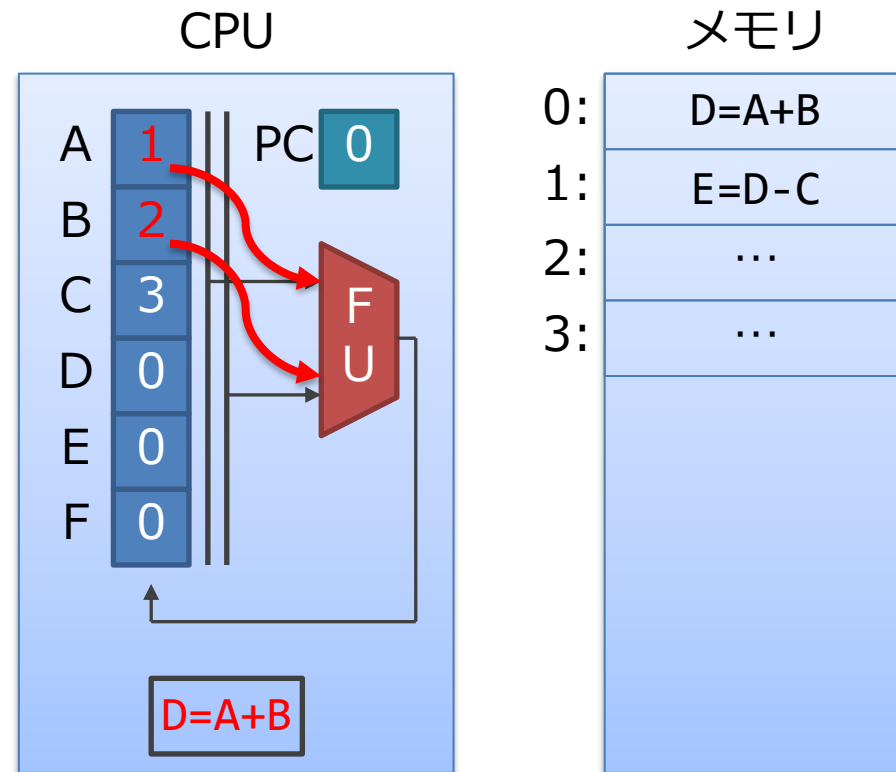
1. 命令の読み出し（フェッチ）

1. PC が指している命令の番地を読む
 1. 今はアドレス 0 を指している
2. 内容である $D=A+B$ が得られる
3. CPU 内にもってくる



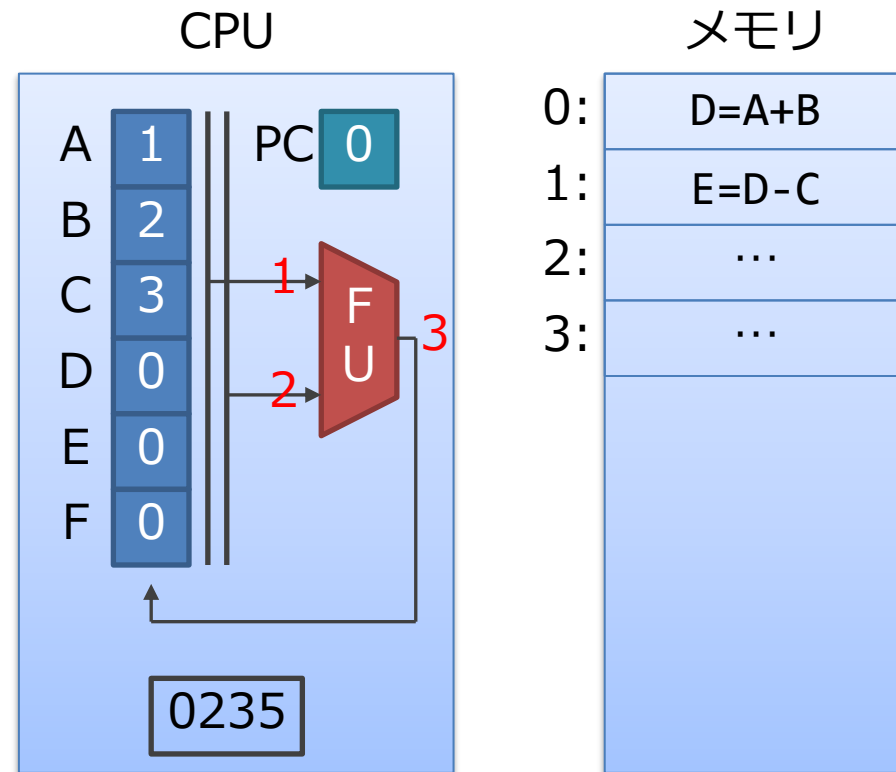
2. レジスタ読み出し

1. 入力の A と B をレジスタから読みだす
2. 中身である 1 と 2 が取れる



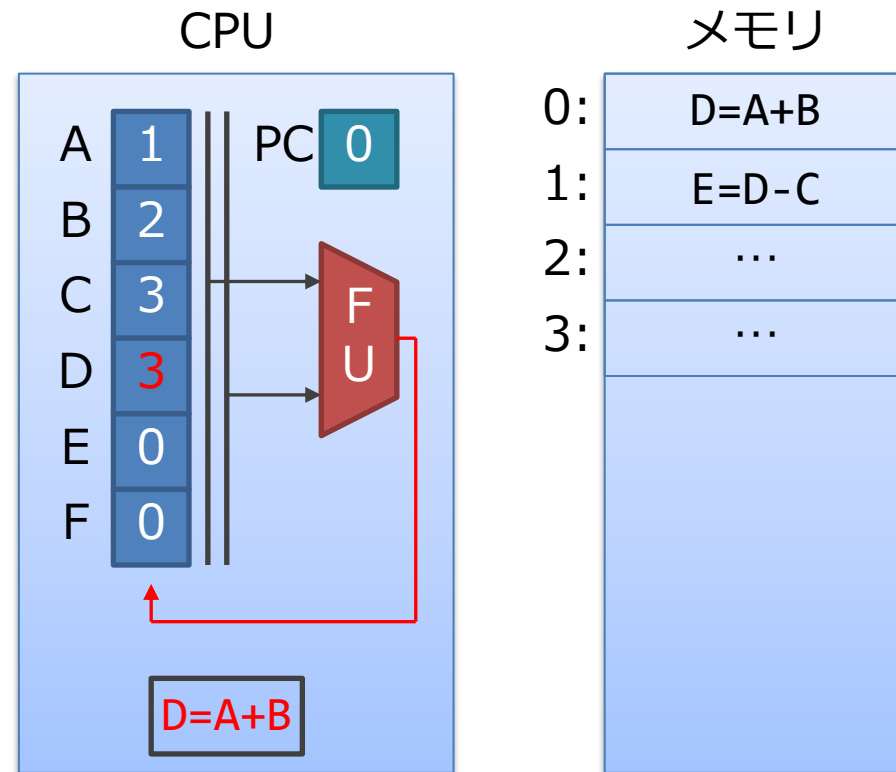
3. 演算の実行

1. 演算器 (FU) で, 足し算をする
 - $1 + 2 = 3$



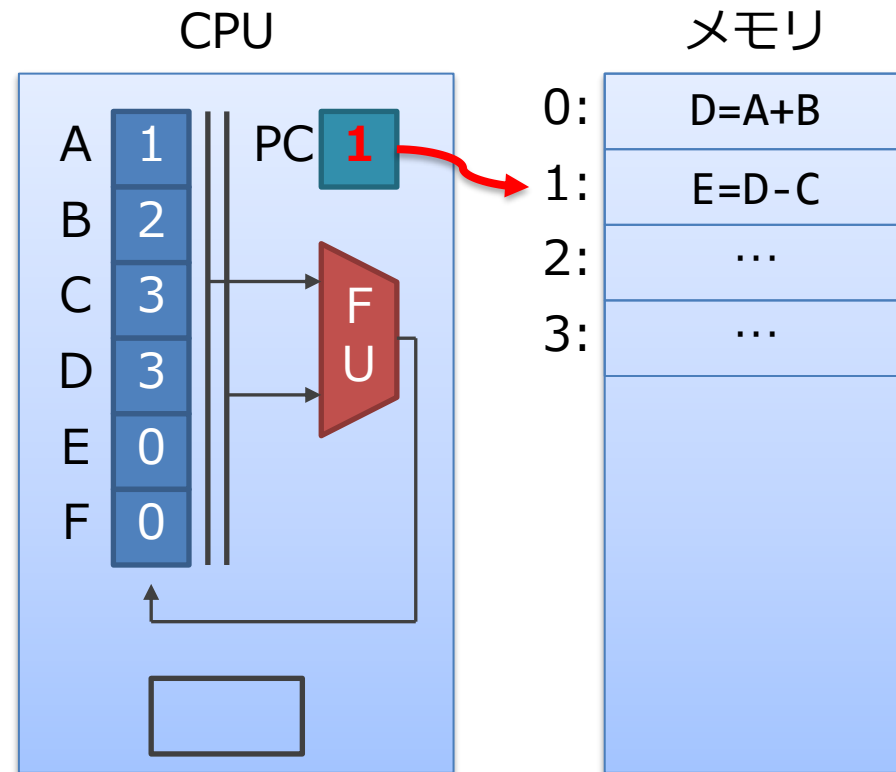
4. レジスタへの結果の書き戻し

1. D に結果の3を書き込む



5. 次の命令へ

1. PC に + 1 をする
2. 1. の命令の読み出しに戻る

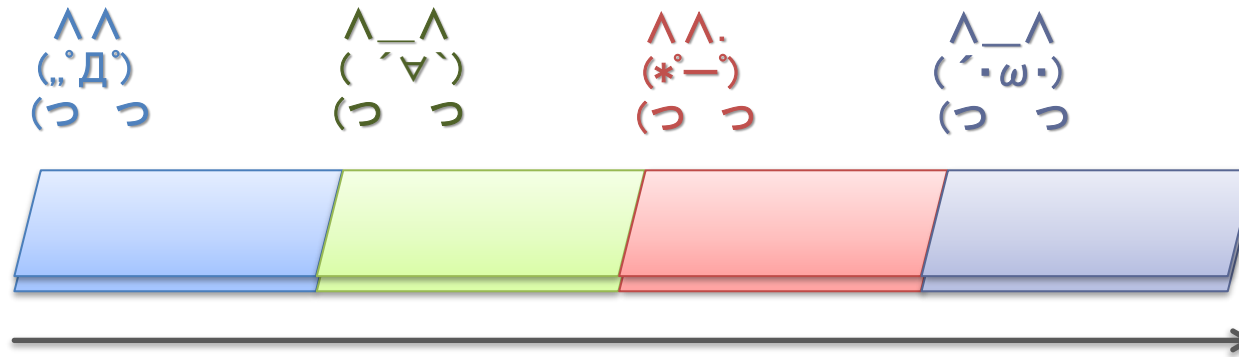


もくじ

- CPU の基本
- 命令パイプライン

導入：工場のラインを考える

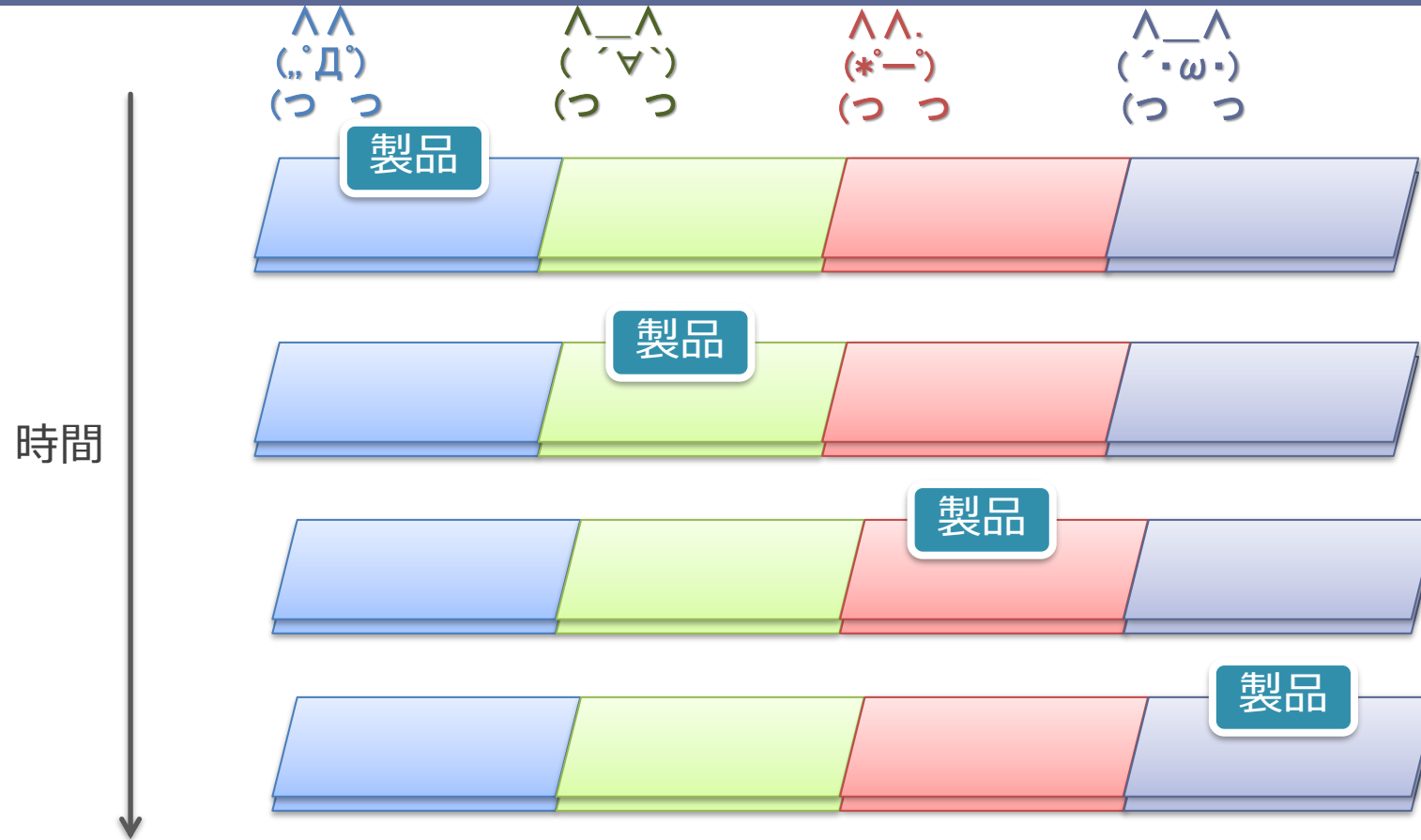
パワポだとアニメーションします



- ベルトコンベアのラインの上を製品が流れていく
 - 4 人の人が、それぞれの工程の作業をおこなって完成
- 上のように1つしか製品をながさないで、
 - 各人は他の人が作業している間はヒマ

導入：工場のラインを考える

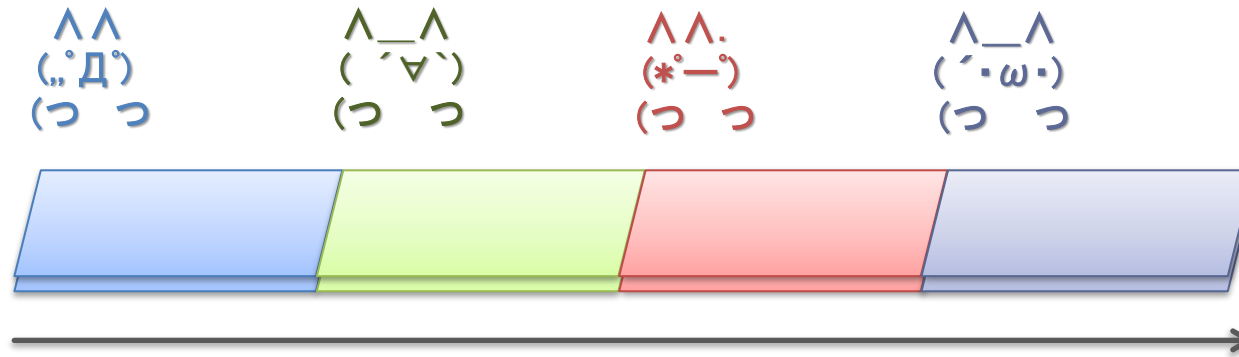
PDF 用のパラパラマンガ版



- ベルトコンベアのラインの上を製品が流れていく
 - 4 人の人が、それぞれの工程の作業をおこなって完成
- 上のように1つしか製品をながさないで、
 - 各人は他の人が作業している間はヒマ

導入：工場のラインを考える

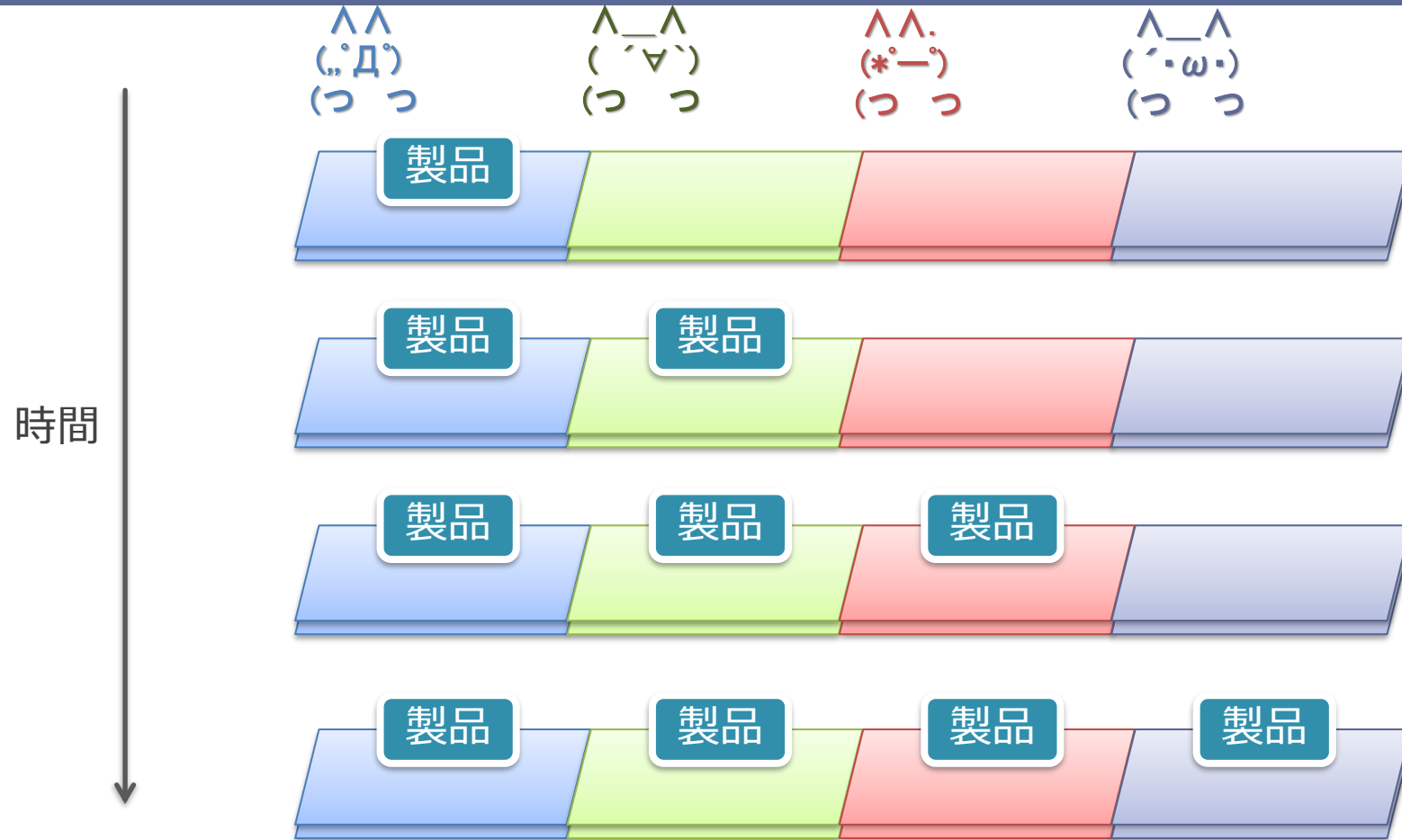
パワポだとアニメーションします



- 実際の工場：複数の製品を同時に流す
 - 各工程を並列して処理することによりスループットを向上
 - さっきの4倍の速度で製品ができあがっていく
- これが **命令パイプライン** の基本的な考え方

導入：工場のラインを考える

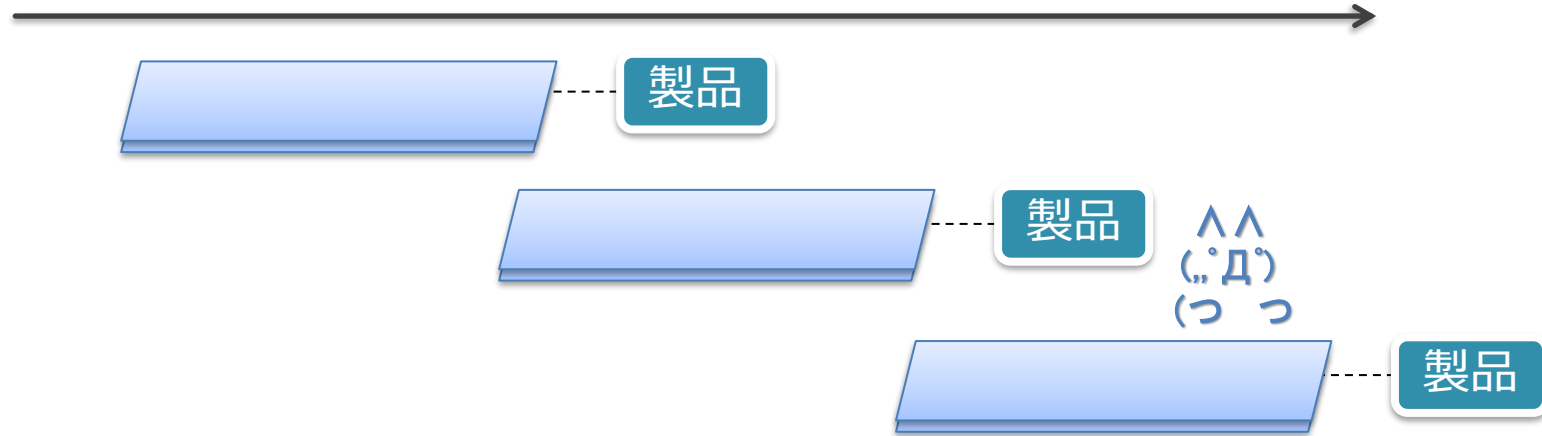
PDF 用のパラパラマンガ版



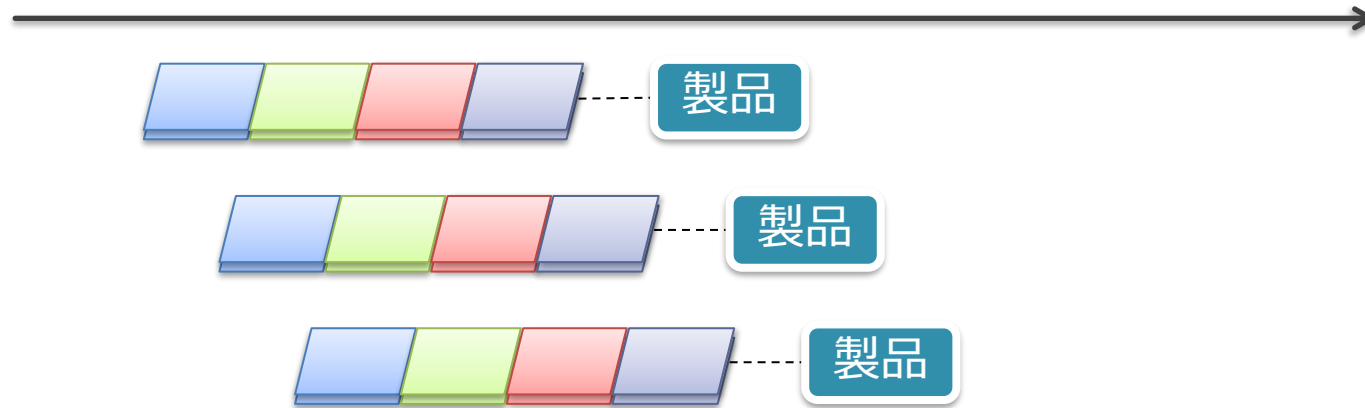
- 実際の工場：複数の製品を同時に流す
 - 各工程を並列して処理することによりスループットを向上
 - さっきの4倍の速度で製品ができあがっていく
- これが **命令パイプライン**

パイプライン化による性能向上

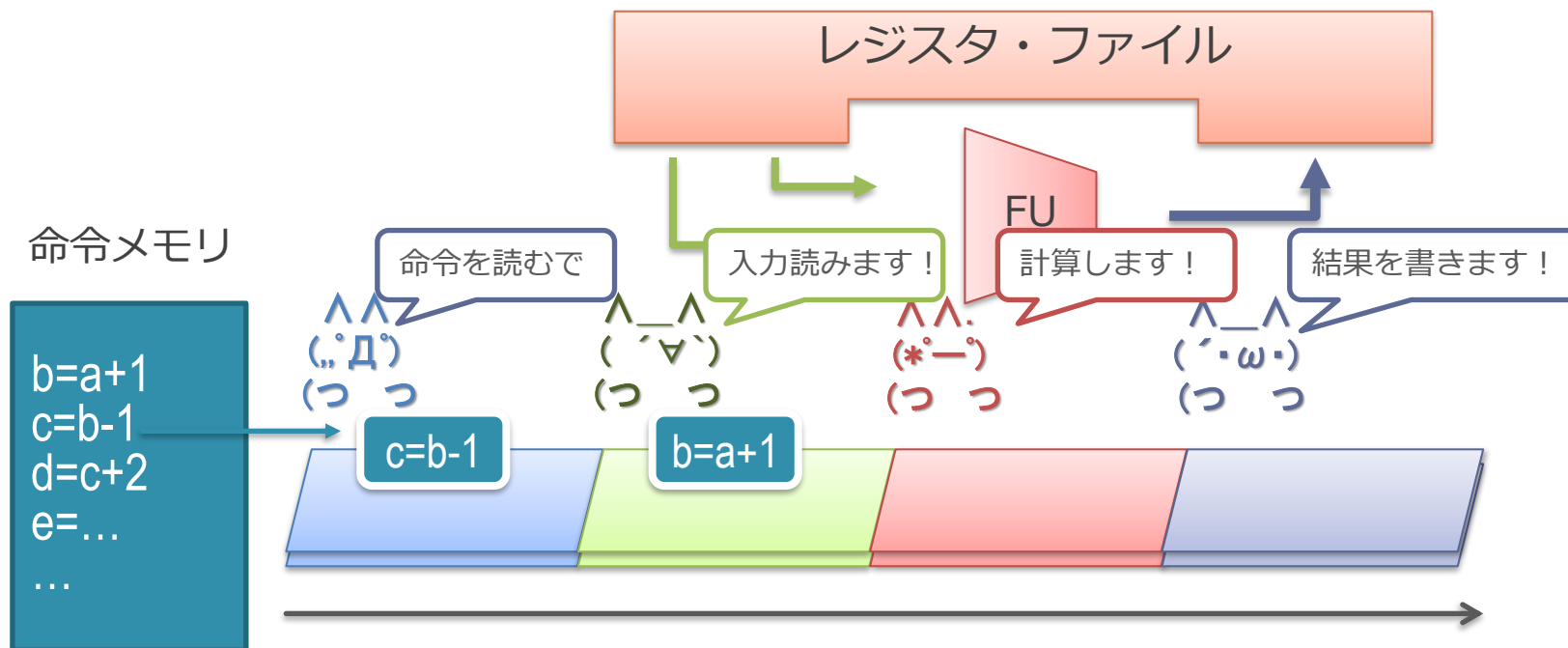
パイプライン化しない場合



パイプライン化した場合



命令パイプライン



- (.,.D.) が命令をメモリから読み出す
- ('▽`) がレジスタ・ファイルから入力の値を読み出す
- (*°ー°) が演算器 (FU) で計算する
- (´・ω・) がレジスタ・ファイルに結果を書き込む

ここまでの振り返りと方向性

ここまでのまとめ

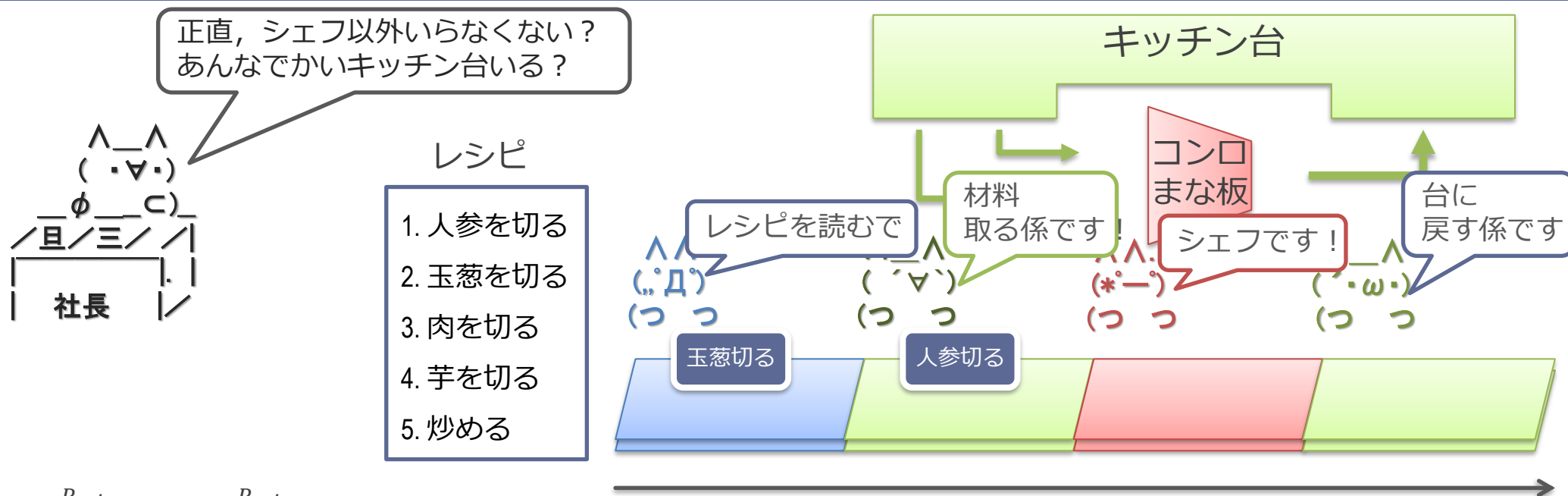
1. 消費電力は回路の大きさに比例する
 - 水タンクの総量が大きくなるイメージ
 2. CPU の動作
 - 命令を読んで, レジスタを読んで, 計算し, レジスタに戻す
 - 流れ作業によって効率を上げている
- これらをもとに, エネルギー効率とアーキテクチャを見ていく

CPU や GPU などのエネルギー効率とは

牧野先生の Principles of High-Performance Processor Design より

- 効率 $\eta = \frac{P_{min}}{P_{actual}} = \frac{P_{min}}{P_{min} + P_{overhead}}$
- P_{min} = そのアプリケーション内の「演算」を行うのに最低必要なエネルギー
 - たとえば行列積であれば、乗算や加算そのものに必要なエネルギー
- P_{actual} = 実際にチップで消費されたエネルギー
 - 命令を読み出したり、レジスタを読み書きしたりがのってくる
- 効率を上げるのに取り得る方針：
 - $P_{overhead}$ を減らす
 - （全体のエネルギーを減らすなら、 P_{min} 自体を減らす方向もある（付録）

カレー屋さんパイプラインで考える



■ 効率 $\eta = \frac{P_{min}}{P_{actual}} = \frac{P_{min}}{P_{min} + P_{overhead}}$

- P_{min} = 材料の原価とシェフの人件費 (絶対カレーを作るのに必要)
- $P_{overhead}$ = キッチン台, 他の係の人 (必ずしもなくても)

■ 効率を上げるのに取り得る方針:

- $P_{overhead}$ を減らす = シェフ以外をなんとかしてクビにするか減らす
- (P_{min} を減らす = 別の方法としてカレー自体の具材を減らして質を下げると, 効率 η は上がらないがトータルコストは減る)

GPU のアーキテクチャ

GPU が対象とする処理

- 大量の単純な処理の繰り返し
 - 並列処理できる部分が容易にわかる
 - 典型的な例：行列積

並列処理できる部分が自明なループの例

■ ループで書いた行列積のコード

```
for (k = 0; k < 1024; k++) // 最外周ループ
    for (j = 0; j < 1024; j++)
        for (i = 0; i < 1024; i++)
            a[j][i] += b[j][k] * c[k][i];
```

■ ループ内の乗算や加算は、明らかに並列にできる

- ループの各周回の乗算を並列にやって、別々に足し合わせていける
- (浮動小数点の場合、厳密には多少結果が変わる)

対象とする処理の違い

- CPU :
 - 単一の連続した処理
- GPU :
 - 大量の単純な処理の繰り返し
- ここではカレー屋さんで説明

単一の連続した処理= 1つのカレーを速く作る場合

■ 1つのカレーを並列処理で高速に作るのは難しい

- 「2.野菜を炒める」は
「1.野菜を切る」を先にやる必要がある
- 「2.野菜を炒める」と
「3.肉を切る」は並列にできる

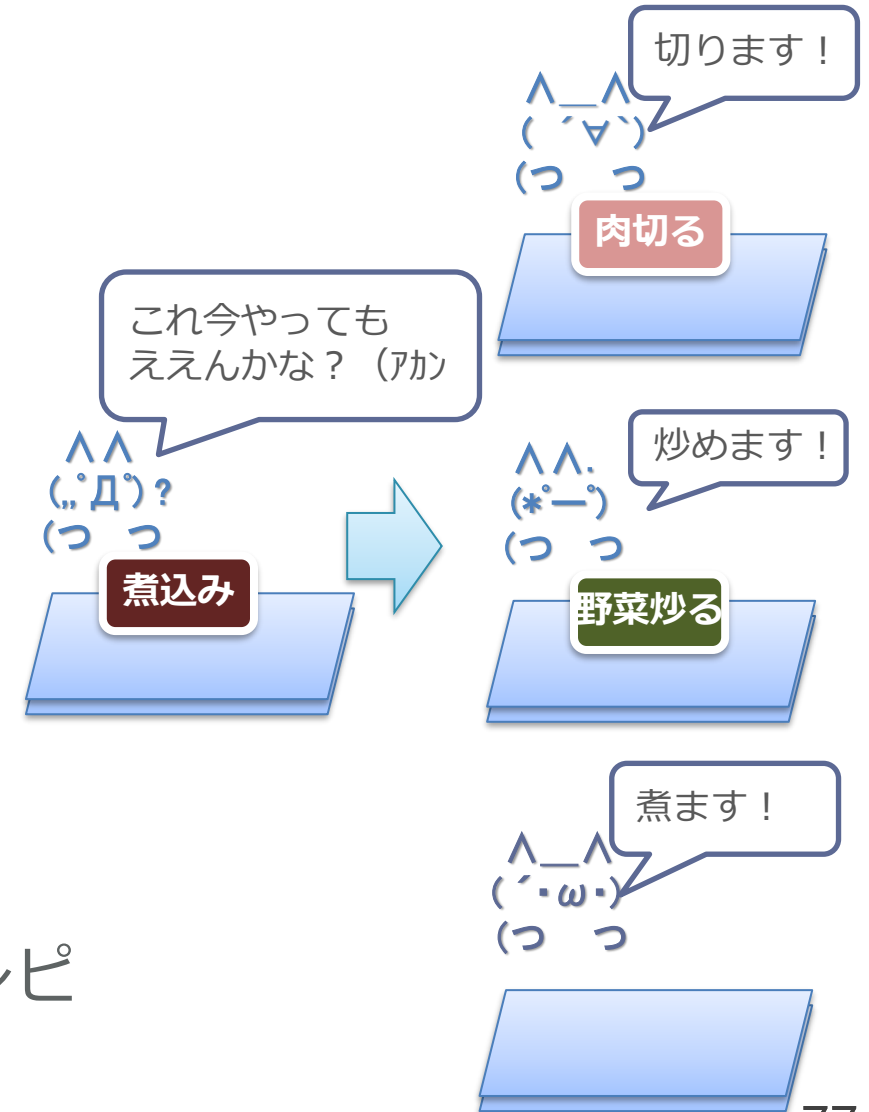
■ 先頭の人が見極めて仕事を割り振れば ある程度できなくはない

- 右側に3倍の人員を割いているが、
3倍の速さにはならない
- 依存があるので、先に並列処理できない
ことが多い

■ CPU はこういうプログラムがターゲット

- 1.野菜を切る
- 2.野菜を炒める
- 3.肉を切る
- 4.肉を炒める
- 5.煮込む
- 6.ルーを入れる
- 7.コメを研ぐ
- 8.コメを炊く
- 9.盛りつける

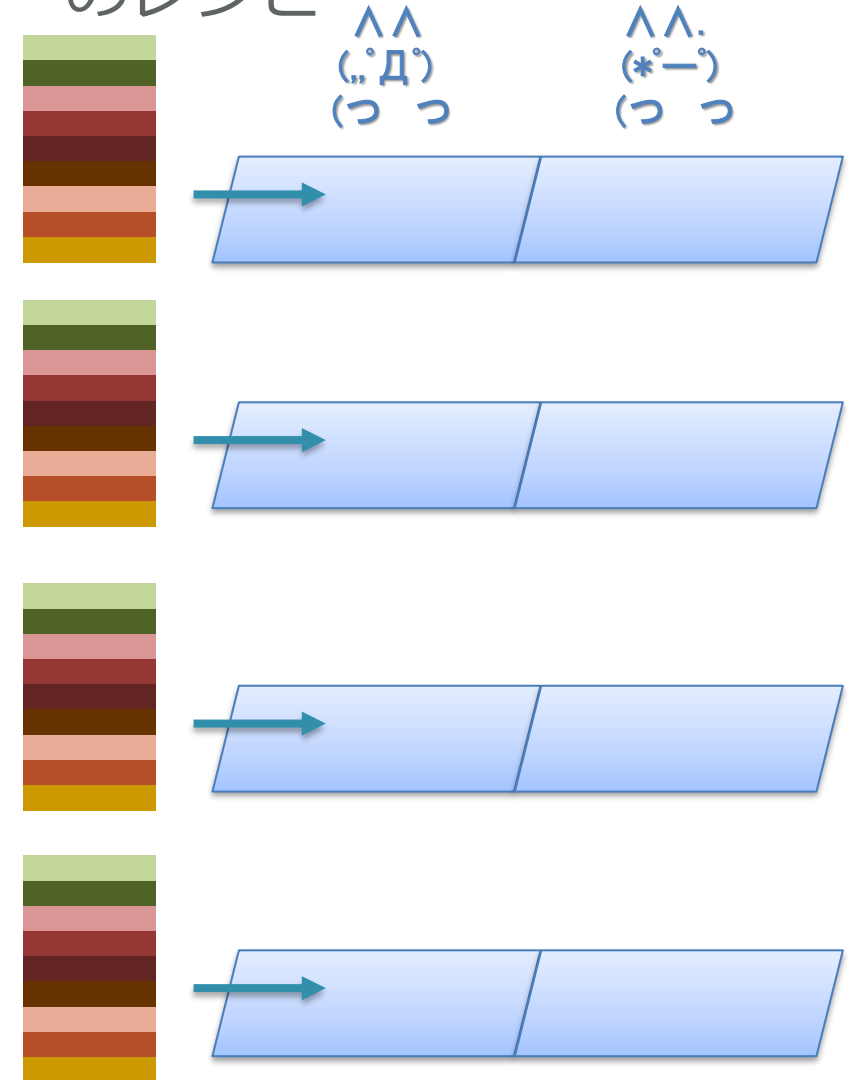
カレーのレシピ



大量の単純な処理の繰り返し = カレーをたくさん作る場合

- 単に並べてみんなで同じ処理をすれば速くなる
 - ループの各周をみんなでやってくださいと言うだけ
 - 振り分けとか考える必要もない
- 並べた分だけ速くなる（すごい）
 - 右だと4倍速い
- GPU はそう言うことが出来るプログラムがターゲット
 - もともとグラフィック処理では、画面上の多数のピクセルの処理がこれに該当
 - ピクセル間の処理は基本的に依存がない
 - 画面の上の方と下の方は並列にやっても問題ない

カレーのレシピ



ループのマルチスレッド化による並列実行

- ループで書いた行列積のコード

```
for (k = 0; k < 1024; k++) // 最外周ループ
    for (j = 0; j < 1024; j++)
        for (i = 0; i < 1024; i++)
            a[j][i] += b[j][k] * c[k][i];
```

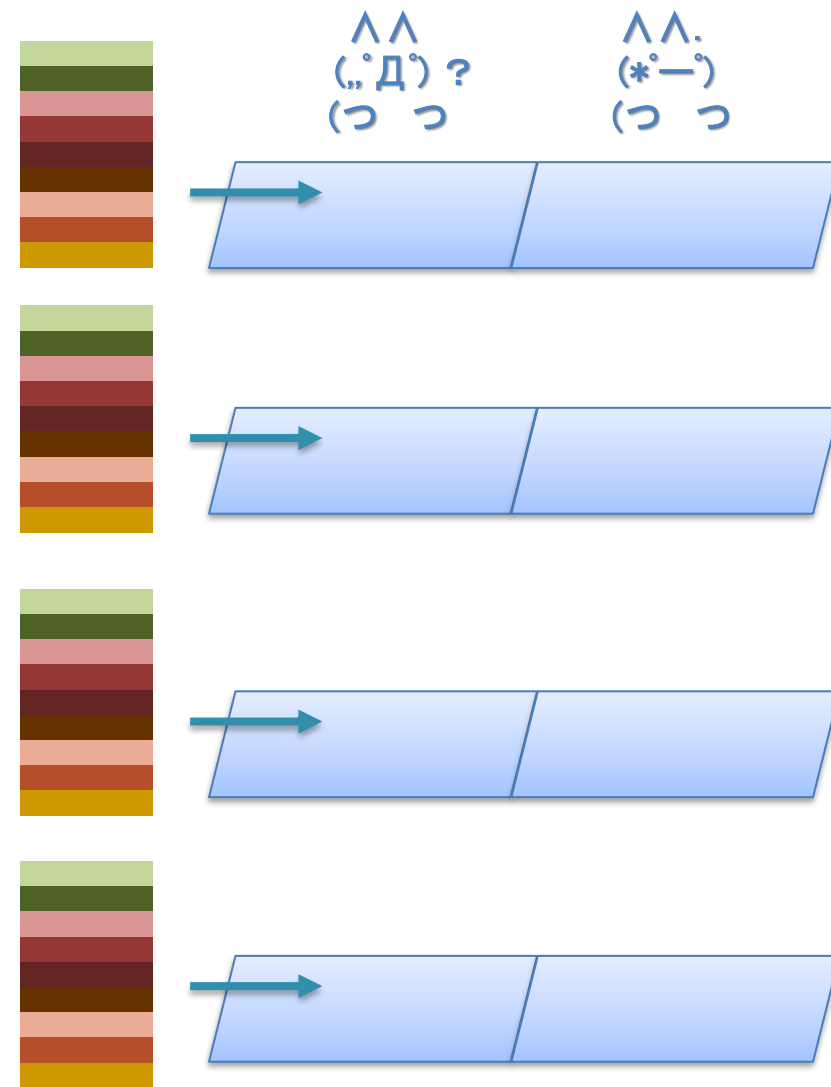
- 最外周ループ部分を複数のスレッドとしてマルチスレッド化

// func が 1024 個起動されて並列に実行される
// 0-1023 がスレッド ID として渡される

```
func(thread_id) {
    k = thread_id;
    for (j = 0; j < 1024; j++) // ここは同じ
        for (i = 0; i < 1024; i++)
            a[j][i] += b[j][k] * c[k][i];
}
```

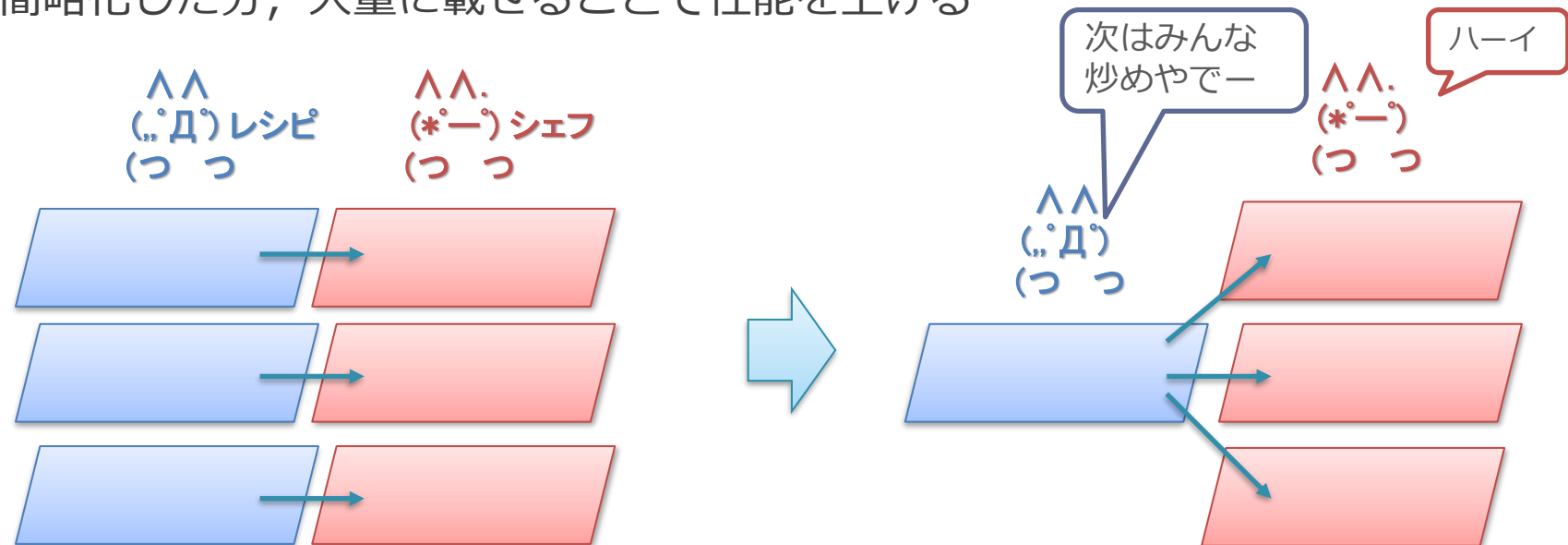
マルチスレッド化による並列実行

- マルチスレッド化による並列実行による高速化
 - 先ほどの例では 1024 スレッドが同時実行される
- マルチコア CPU でも同じなのでは？
 - マルチコア CPU = 複数の CPU を束ねたもの
 - 複数の CPU でそれぞれでスレッドを並列に実行できる



マルチコアとの違い：回路量あたりの性能の向上

- GPU は同じ処理をするスレッドを多数走らせることに特化
 - 行列積の例では，全スレッドが「同じ処理」をしている
 - 正確には，演算（乗算と加算）は同じで，値が異なる
- 方針：みんな同じ処理をしているので，それを利用して回路を簡略化
 - みんな同じ処理をしている = 制御回路を共有できる
 - レシピ自体や読む係は1人で良く，みんなに同じ指示を投げる
 - 簡略化した分，大量に載せることで性能を上げる



代表的な GPU のアーキテクチャ

- GPU は同じ処理をするスレッドを多数走らせることに特化
 - これを効率的に実現するのが SIMD と呼ばれる方式
 - SIMD は GPU のハードウェア方式やプログラミング・モデルの根幹に関わる
- 大きく分けて 2 つの方式
 1. NVIDIA : Single Instruction Multiple Thread (SIMT) 方式
 2. AMD/Intel : Single Instruction stream Multiple Data (SIMD) 方式
- SIMD 方式から順に説明
 - SIMT 方式は SIMD を発展させた概念であるため

GPU の基本的な構造

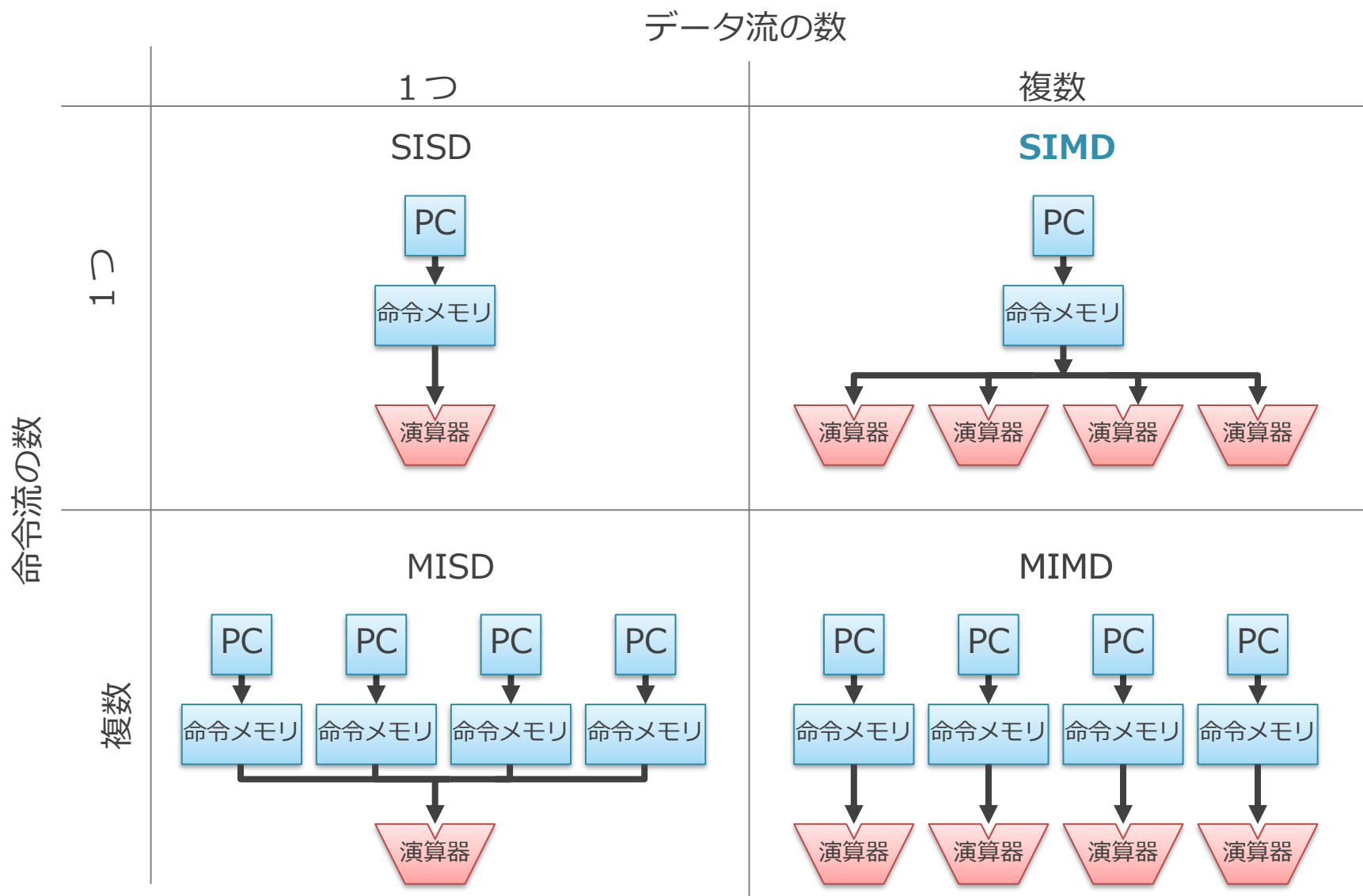
1. フリンの分類と SIMD
2. GPU における回路量当たりの性能向上
3. SIMD プロセッサのプログラミング・モデル

フリンの分類と SIMD

- Single Instruction stream/Multiple Data stream (SIMD)
 - コンピュータ・ハードウェアの構成方法の分類の 1 つ
 - あいだの「stream」は省略されることもある
- フリンによる分類に由来
 - M. J. Flynn, "Very high-speed computing systems," in Proceedings of the IEEE, vol. 54, no. 12, pp. 1901-1909, Dec. 1966
- プログラムを処理する際の
 - 「命令の流れ」と「データの流れ」に着目
 - それぞれが同時に 1 つあるか、複数あるかで分類

フリンの分類

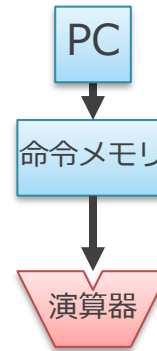
混乱したらこの図を見返そう



■ カレー屋さんとの対応

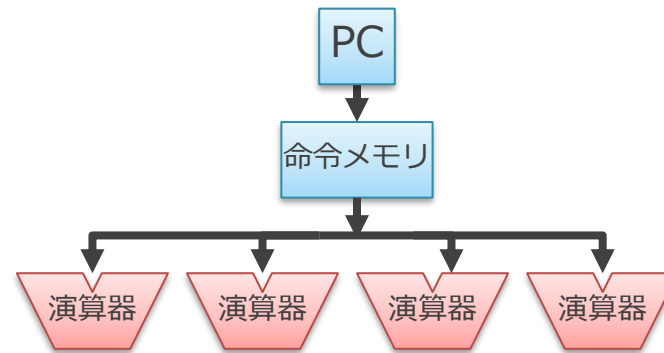
- PC: レシピ上の今やってるところ（しおり）
- 命令メモリ: レシピ
- 演算器: シェフ

SISD (Single Instruction stream/Single Data stream)



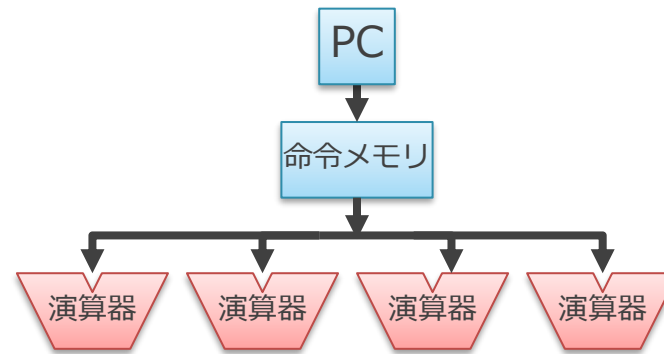
- 1つの命令流が1つのデータ流を処理する方式
 - 動作：
 - 同時に1つのプログラム（命令の流れ）を実行
 - それぞれの命令は1つのデータを演算
 - 典型的にはシングルコア CPU がこれに該当

SIMD (Single Instruction stream/Multiple Data stream)



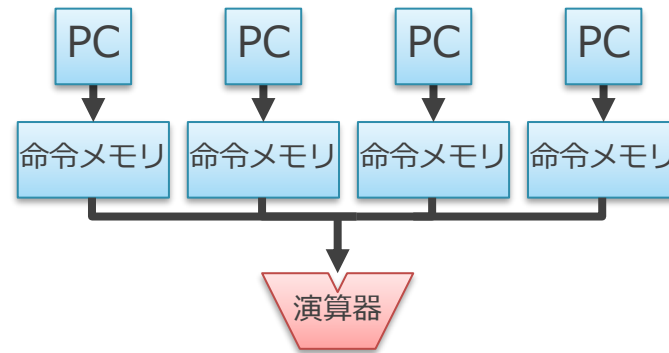
- 1つの命令の流れが複数のデータの流を処理する方式
 - SIMDは同時に1つのプログラム（命令の流れ）を実行
 - それぞれの命令は複数のデータに対して同じ演算を行う
- 例：4つの演算を同時に行う SIMD 方式
 - 解釈された命令は4つの演算器に送信
 - それぞれの演算器は異なる4つのデータに対して同じ演算を行う
 - たとえば加算命令であれば、1つの加算命令が4つの加算を行う

SIMD (Single Instruction stream/Multiple Data stream)



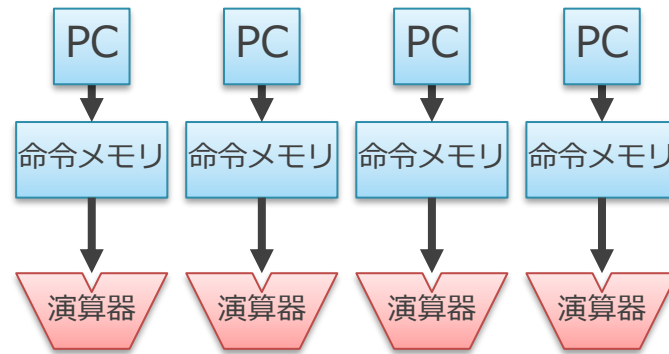
- 現在の主だった GPU はこの SIMD 方式をベースにしている
 - 一部, CPU にもこの考えを取り入れた命令がある
- GPU は同じ処理を多数走らせることに特化
 - PC や命令メモリは共有して, 各演算器に配ればよい

MISD (Multiple Instruction stream/Single Data stream)



- 複数の命令の流れが単一のデータの流を処理する方式
 - 分類の都合上存在していると言ってもよい
 - この方式の実用的なコンピュータは現在一般的ではない
- 「みんなで違うレシピを読んで、1人のシェフに同時に指示を与える」みたいなことに相当し、意味がわからない

MIMD (Multiple Instruction stream/Multiple Data stream)



- 複数の命令の流れが複数のデータの流を処理する方式
 - 典型的にはマルチコア化された CPU がこれに該当

フリンの分類と実際

- 現代のコンピュータをこのフリンの分類にそのまま厳密に当てはめようとするやや無理がある
 - 通常は複数の要素を同時に併せ持つことが多い
- たとえば…
 - GPU は SIMD 方式のコアを複数備えていることが一般的
 - SIMD と MIMD の要素を併せ持つ
 - CPU では一部の命令でのみ複数のデータを同時に処理する
 - SISD と SIMD の要素を併せ持つ
 - マルチコア化されると MIMD の要素もある
- フリンの分類は、あくまでコンピュータをある特定の軸から見た際の分類であると考え

余談：SIMD の発音

- 業界では「シムディー/sim-dee」と発音する
 - シムド と読んじゃだめ

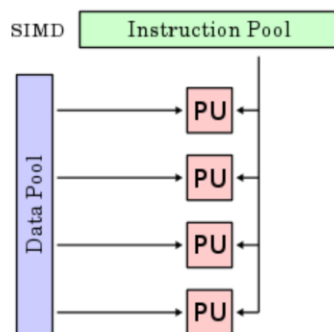
[mdn web docs](#) [References](#) [Guides](#) [MDN Plus](#)

MDN Web Docs Glossary: Definitions of Web-related terms > SIMD



SIMD

Single-Instruction/Multiple-Data Stream
(SIMD or “sim-dee”)



- SIMD computer exploits multiple data streams against a single instruction stream to operations that may be naturally parallelized, e.g., Intel SIMD instruction extensions or NVIDIA Graphics Processing Unit (GPU)

SIMD (pronounced "sim-dee") is short for **Single Instruction/Multiple Data** which is one [classification of computer architectures](#). SIMD allows one same operation to be performed on multiple data points resulting in data level parallelism and thus performance gains — for example, for 3D graphics and video processing, physics simulations or cryptography, and other domains.

それぞれ以下より

Mozilla の開発者ページ

<https://developer.mozilla.org/en-US/docs/Glossary/SIMD>

UCSB の講義資料

<https://sites.cs.ucsb.edu/~tyang/class/240a17/slides/SIMD.pdf>

イラスト

村上太一@TIER IV

もくじ

1. フリンの分類と SIMD
2. GPU における回路量当たりの性能向上
 1. 演算器と制御部
 2. SIMD による制御部の共有
 3. 制御部自体の小型化
3. SIMD プロセッサのプログラミング・モデル

回路量当たりの性能の背景：演算器と制御部

■ CPU や GPU などを構成する回路を以下に分けて考える

● 制御部（青）

- 命令を解釈しそれに基づいて演算器を制御する回路
- 右の図だと PC や命令メモリ

● 演算器（赤）

- 加算や乗算などの演算そのものを行うための回路

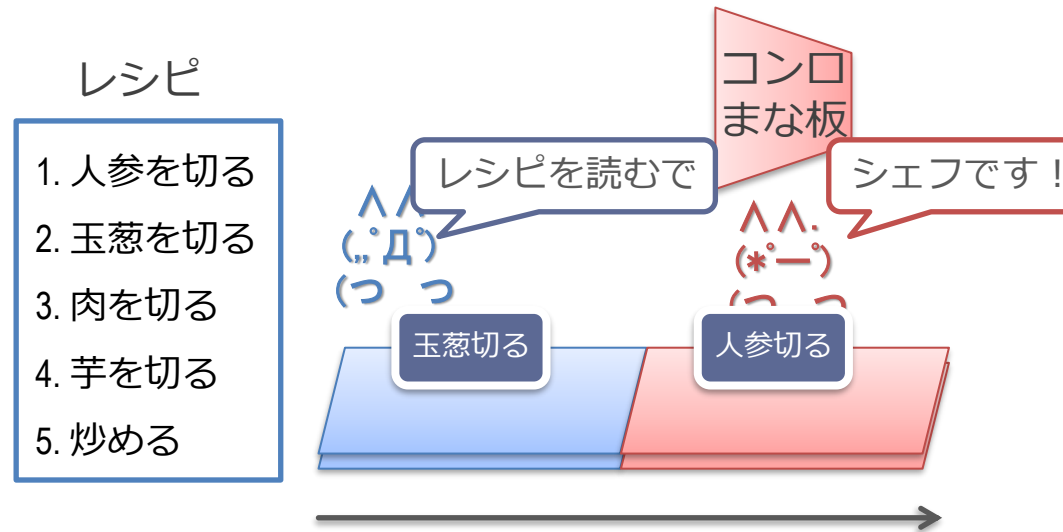


■ これに基づいて回路量あたりの性能を議論

■ ポイント：演算器部分自体は CPU でも GPU でも基本的に変わらない

- GPU が効率が良い演算器を持っているなら、それは CPU にも持っていけるはず
 - AI に特化した演算器は CPU にも取り入れられている
- それ以外をどう減らすかが要点

またカレー屋さんで考えていく

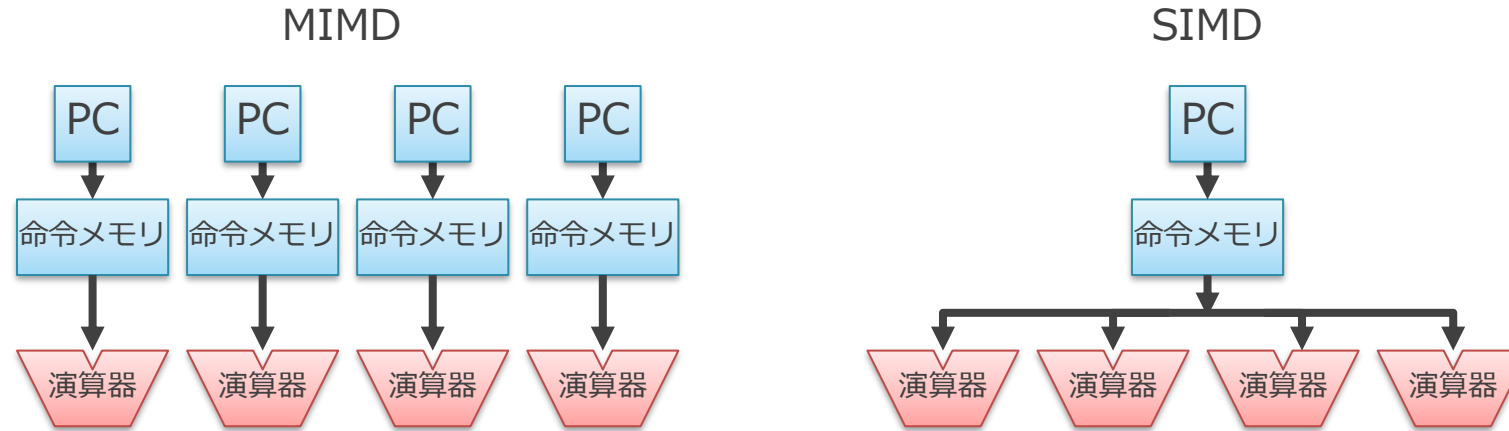


- 制御部（青）：PC と命令メモリ
 - レシピと読む係の人
- 演算器（赤）：そこより後ろで実際に乗算や加算をしている部分
 - シェフの人
 - キッチン台や関連するスタッフは一旦ここでは忘れる
(まだクビになったわけではないが、後で合理化のためにリストラする予定)

GPU における回路量あたりの性能向上

1. 制御部の共有
2. 制御部の小型化

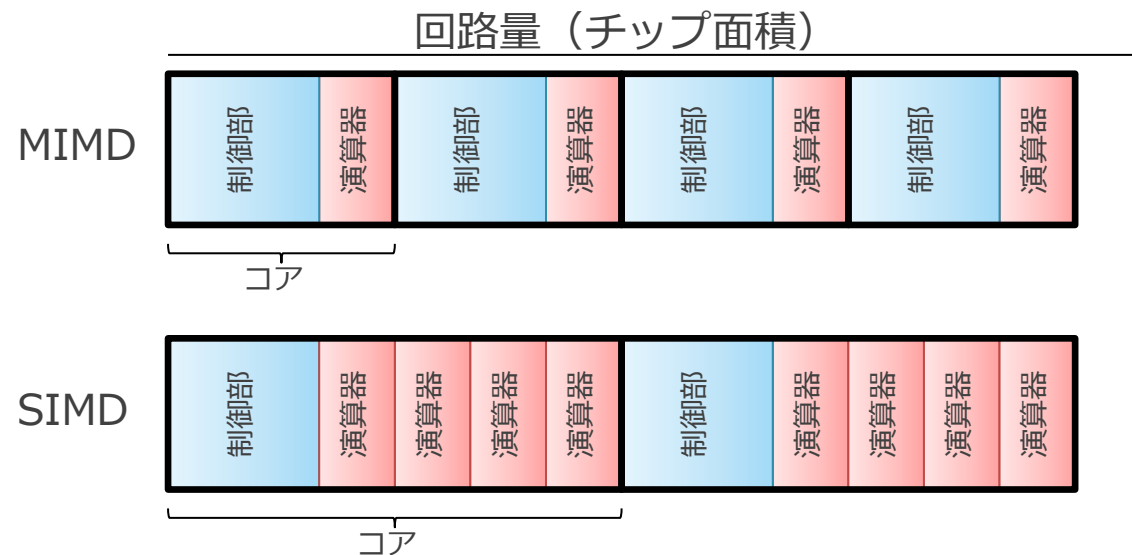
SIMD では複数の演算器間で制御部を共有できる



- たとえば, 上の図の MIMD と SIMD の場合
 - 4つの演算器を持ち最大で同時に4並列で演算ができる
 - = 同じ最大性能を持つ
 - SIMD では 1つの制御部が演算器間で共有されている
 - 実際の GPU では 16 個程度の演算器が共有されていることが多い

SIMD は同じ回路量で高い性能を実現できる

- SIMD は MIMD と比べて全体をより小さく作ることができる
 - 同じ総面積であれば、SIMD の方がより多くの演算器を搭載できる
 - 下の図だと演算器は **MIMD 4 個** vs. **SIMD 8 個**
 - つまり、同じ回路資源量あたりで、高い性能を出せる
 - 払える人件費が一定の場合、シェフ 4 人あたりレシピ読み係を 1 人にして、シェフの人数を増やす事に相当
- これが、どの GPU も基本的には SIMD を採用している理由



GPU における回路量あたりの性能向上

1. 制御部の共有

2. 制御部自体の小型化

- （この話は SIMD の話とは直交しているが、関わるのでここで

GPU では制御部自体を簡素化

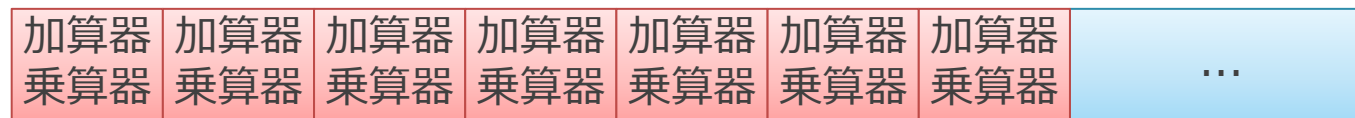
■ CPU

- 単一のスレッドの実行時間を短くすることに特化
- 各種投機実行や、動的命令スケジューリングに回路資源を投入



■ GPU

- 大量のスレッドの実行スループットを上げることに特化
- 演算器以外の部分をそぎ落とし、なるべく大量の演算器を積む

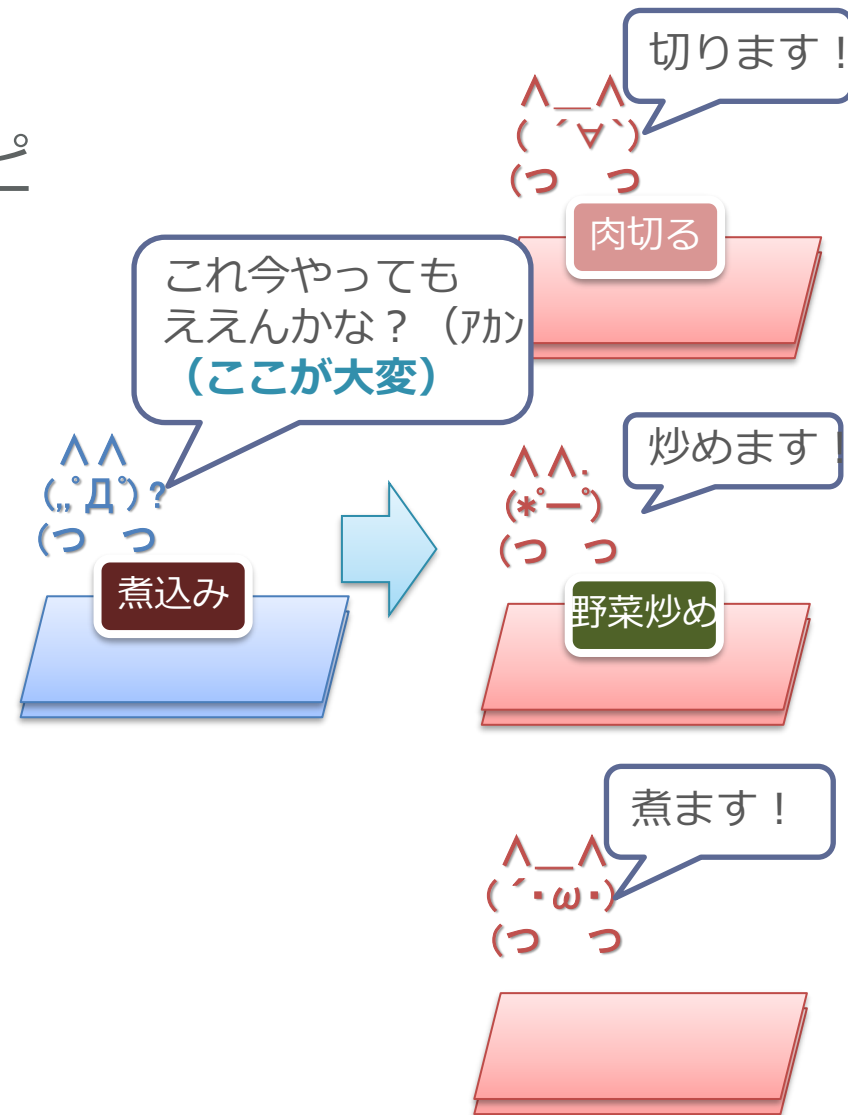


仕事の割り振りは超大変

- 並列にできる部分を探す回路は、演算そのものよりも遥かに複雑
- 現代の高性能 CPU は、ここに大半の回路を割いている

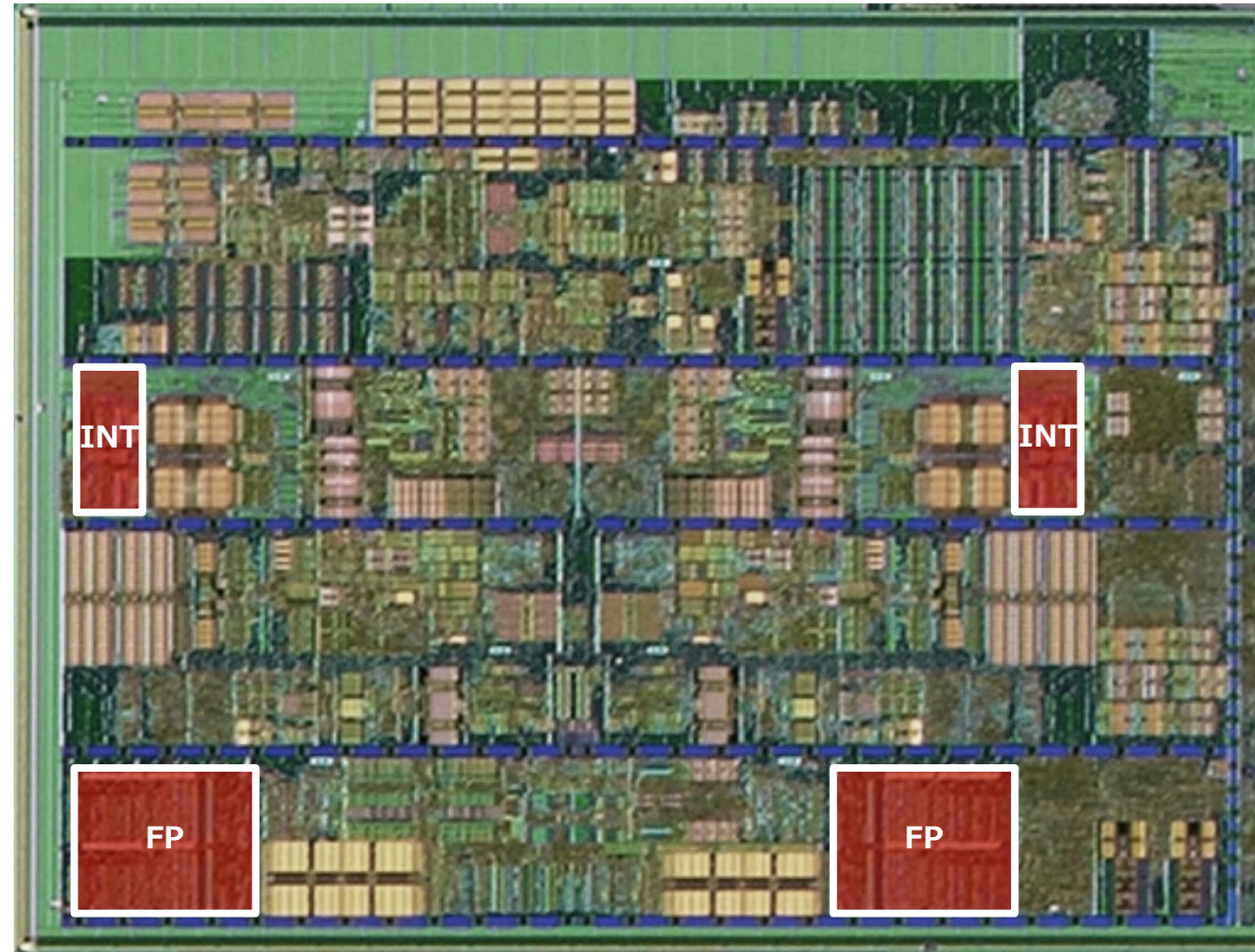
カレーのレシピ

- 1.野菜を切る
- 2.野菜を炒める
- 3.肉を切る
- 4.肉を炒める
- 5.煮込む
- 6.ルーを入れる
- 7.コメを研ぐ
- 8.コメを炊く
- 9.もりつける



AMD Bulldozer のコア部分に占める演算器部分

チップ写真は https://en.wikichip.org/wiki/File:Bulldozer_Die_Shot.jpg より



- 演算器（INT/FP）が CPU コア全体に占める割合はわずか

Apple A18/M4 CPU の場合

Apple - 2024

A18 P

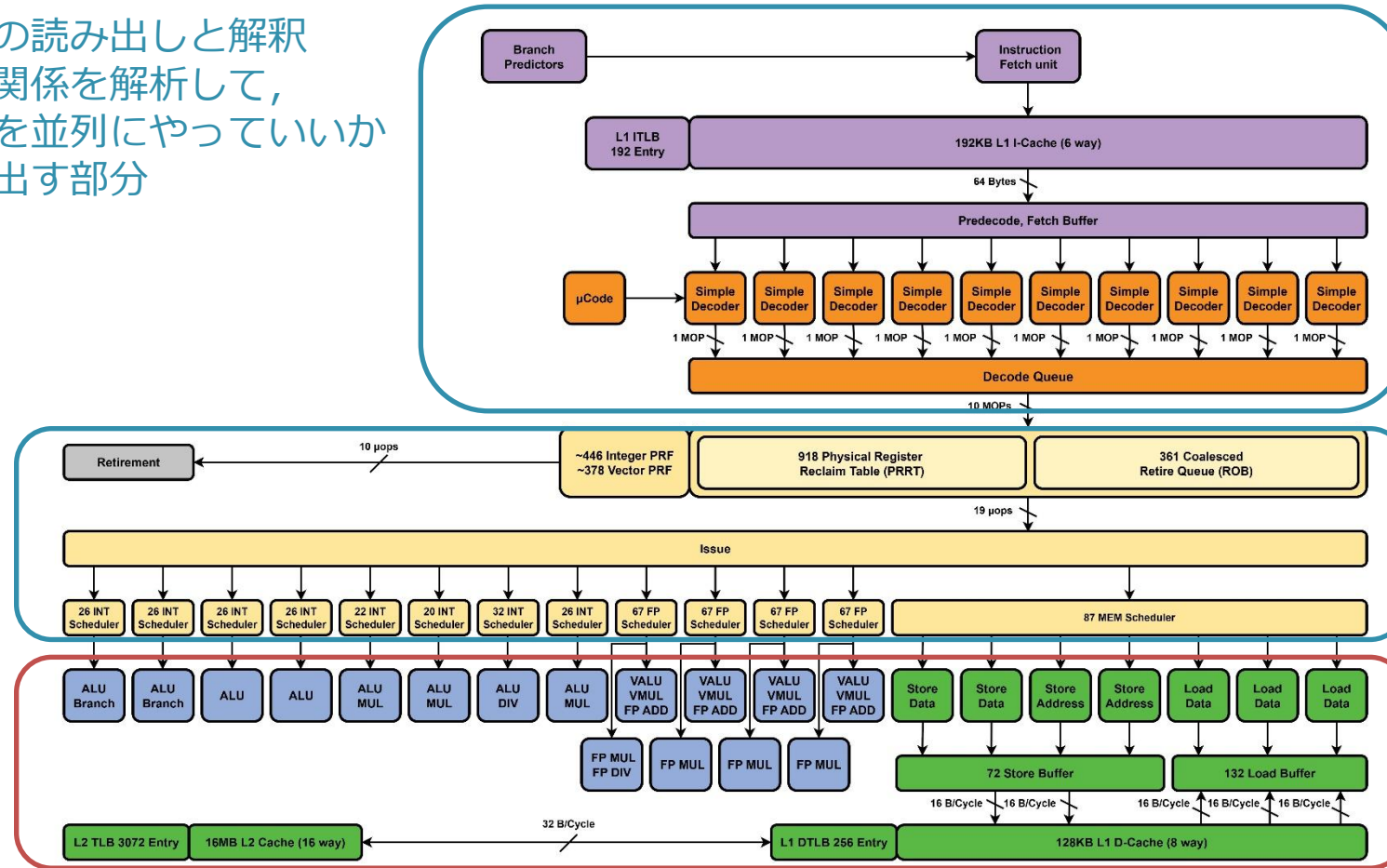
By Cardyak

Microarchitecture Block Diagram

命令の読み出しと解釈
依存関係を解析して、
どこを並列にやっていいか
割り出す部分

依存した処理が
終わっているかを監視して、
演算器に送り出す部分

演算器



■ こういう複雑な部分（青）をなくす

NVIDIA GPU の場合

- 階層型の構造をしている
 - 命令供給系（右上）
 - 4つのサブコアが上記を共有
- サブコア
 - 命令供給系
 - 16～32個の演算器で共有

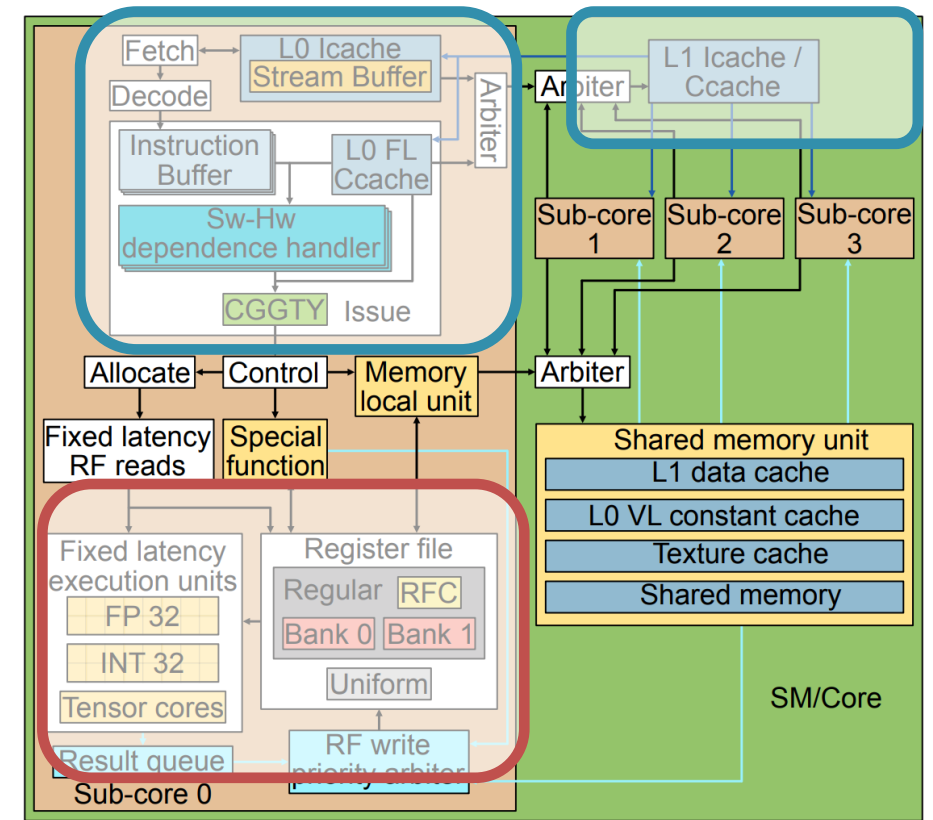
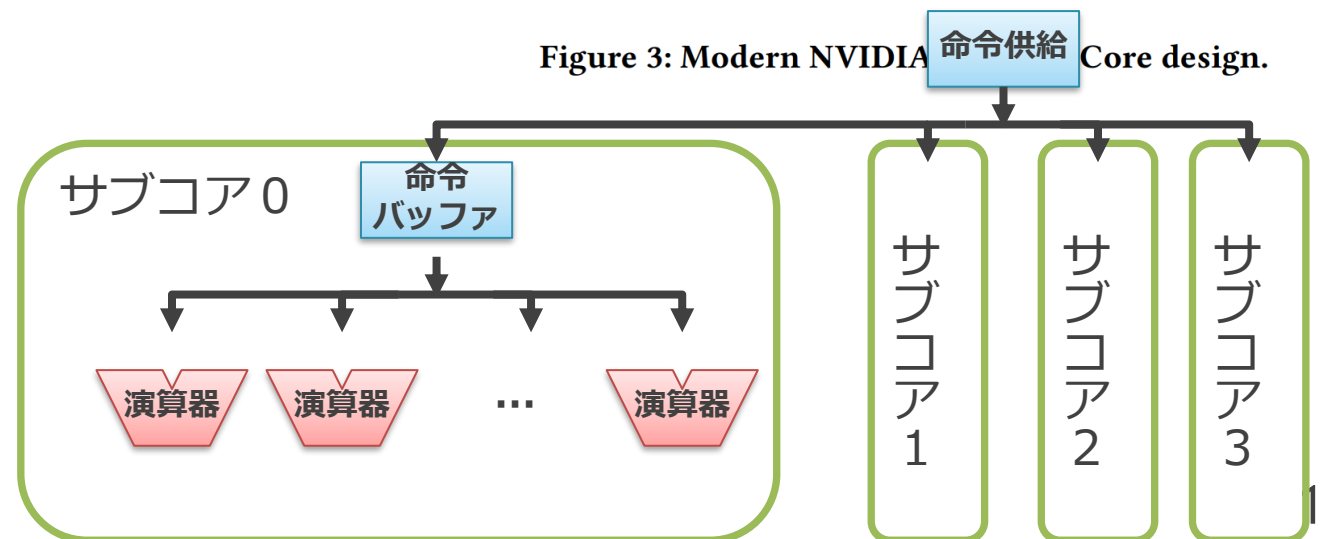


Figure 3: Modern NVIDIA Core design.



GPU における回路量当たりの性能向上

■ GPU における回路量あたりの性能向上

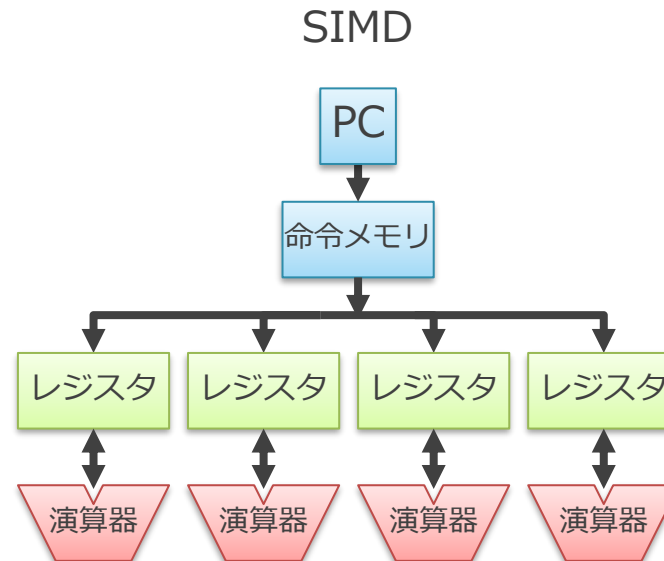
1. SIMD による制御部の共有
2. 制御部自体の小型化

■ たとえば先ほどのチップ写真の場合...

- 赤色の部分のみを N 倍に増やしても、全体は大きくは増えない
 - まず赤色自体が占める面積がかなり小さいから
- しかし非赤色の部分を減らすと、かなりの数の赤色が積める

補足：データの供給について

- ここまでは簡単のためデータの供給についての議論を無視している
 - メモリやレジスタ・ファイルは演算器間で共有できない
- ここをさらになんとかするのが、行列積に特化したハードウェア
 - TPU や Tensor Core
 - これらについては後述



もくじ

1. フリンの分類と SIMD
2. GPU における回路量当たりの性能向上
3. SIMD プロセッサのプログラミング・モデル

プログラミング・モデル

- ここまでは SIMD のハードウェアについて説明
 - ここからはその上でどのようなプログラムを走らせるのかを説明
- プログラミング・モデル
 - どのような命令によりプログラムを記述するかを規定
 - これまでに説明したハードウェアの構成方式とはある程度独立した概念
- カレー屋さんでいうと…
 - ハードウェア = お店の人員と器具をどう配置して使うか
 - プログラミング・モデル = (器具や配置を考慮した) レシピをどう表現するか

ハードウェア方式とプログラミング・モデル

■ 異なるハードウェア方式とプログラミング・モデルの組み合わせ

● NVIDIA の GPU

- ハードウェア : SIMD (を拡張した SIMT)
- プログラミング・モデル : Single Program Multiple Data (SPMD)

● AMD や Intel の GPU

- ハードウェア : SIMD
- プログラミング・モデル : 「SIMD命令」と呼ぶ形式の命令 + SPMD

- (AMD/Intel でも SPMD モデルにより書かれたプログラムをコンパイラによって SIMD 命令に変換して実行することが行われる)

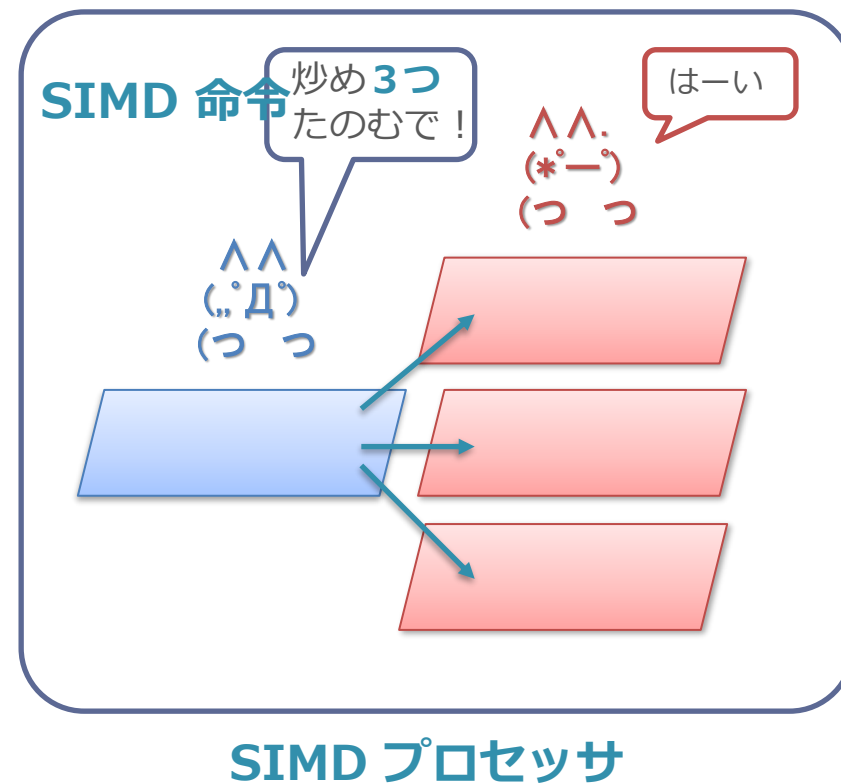
	ハードウェアの方式	プログラミング・モデル
NVIDIA (Volta)	SIMT	SPMD
AMD (GCN,RDNA)	SIMD	SPMD/SIMD 命令
Intel (GEN)	SIMD	SPMD/SIMD 命令

もくじ

1. フリンの分類と SIMD
2. GPU における回路量当たりの性能向上
3. SIMD プロセッサのプログラミング・モデル
 1. SIMD 命令方式
 2. SPMD 方式

用語の定義

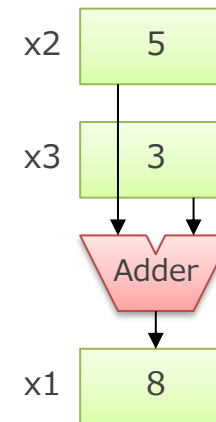
- SIMD プロセッサ :
 - 1つの命令を解釈し、複数のデータに対して同じ演算を同時に行うハードウェア
- SIMD 命令 :
 - SIMD プロセッサ上で複数のデータを対象に同時に演算を行う命令
- まぎらわしい :
 - 「SIMD (プロセッサ)」はハードウェアの構成方式だが,
 - 「SIMD 命令」は命令の表現方式を意味することに注意



SIMD 命令：複数のデータを対象に同時に演算を行う

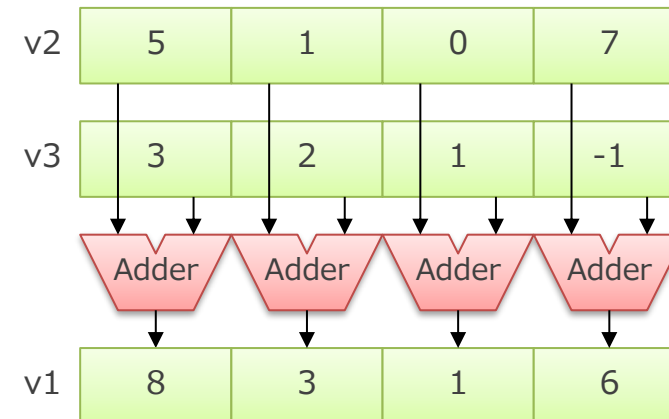
■ 通常の命令（スカラー命令）

- `add x1 ← x2 + x3`
- 「人参を 1 つ切る」



■ SIMD 命令

- `add_x4 v1 ← v2 + v3`
- v1~v3 は 1 つのレジスタに 4 つの値が入ったレジスタ
- SIMD レジスタやベクトルレジスタと呼ばれる
- 「人参を 4 つ切る」



SIMD 命令を使う方法

■ SIMD 命令を使う方法：

1. コンパイラに頑張ってもらおう
 - 複数同時に計算しても大丈夫なところを自動で検出する
 - 後述の SPMD で書かれたプログラムならある程度できる
2. 組み込み関数を使って人間が直接書く
 - 自動でうまくいかない場合は人間が頑張る（つらい）

(余談) 組み込み関数 (intrinsic)

// 連続した加算を4つ行う SIMD 命令に相当する関数

// コンパイル時にこの関数と等価な結果が得られる SIMD 命令に変換される

```
void add_x4(int* dst, int* src1, int* src2) {  
    dst[0] = src1[0] + src2[0];  
    dst[1] = src1[1] + src2[1];  
    dst[2] = src1[2] + src2[2];  
    dst[3] = src1[3] + src2[3];  
}
```

// 連続した乗算を4つ行う SIMD 命令に相当する関数

```
void mul_x4(int* dst, int* src1, int* src2) {  
    dst[0] = src1[0] * src2[0];  
    dst[1] = src1[1] * src2[1];  
    dst[2] = src1[2] * src2[2];  
    dst[3] = src1[3] * src2[3];  
}
```

(余談) 組み込み関数の使用例

```
■ // src1 と src2 から n個 のベクトルの
// 内積を行い, 返り値として返す関数
int dot_product(int* s1, int* s2, int n) {
    int r = 0;
    for (int i = 0; i < n; i++) {
        r += s1[i] * s2[i];
    }
    return r;
}
```

```
■ // ベクトルの内積を行う関数の SIMD 命令版
// データの個数は4の倍数であるとする
int dot_product_x4(int* s1, int* s2, int n) {

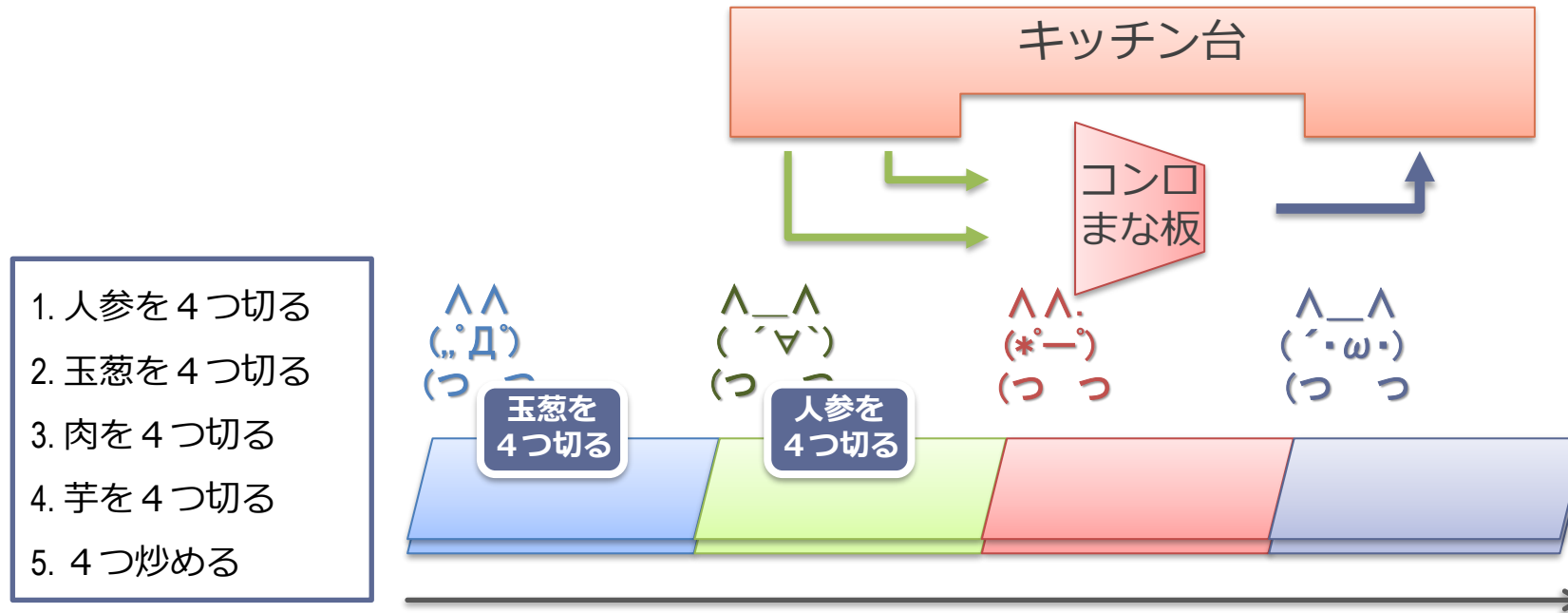
    // 4並列で内積の処理を行う
    int r_x4[4] = {0, 0, 0, 0};
    for (int i = 0; i < n; i+=4) {
        int tmp_x4[4];
        mul_x4(tmp_x4, s1 + i, s2 + i);
        add_x4(r_x4, r_x4, tmp_x4);
    }

    // 4つの中間結果をまとめる
    int r = 0;
    for (int i = 0; i < 4; i++) {
        r += r_x4[i];
    }
    return r;
}
```

■ mul_x4 や add_x4 の部分が SIMD 命令に置き換わる形でコンパイルされる

◇ これを人手でやるのは結構しんどい

カレー屋さんの場合の SIMD 命令や組み込み関数



- SIMD 命令や組み込み関数：
 - レシピに「4つ切る」と最初から書いてある
- プログラミング（レシピを書くとき）なにが難しいのか？
 - 都合よく4つかレーの注文が来ない時の対応
 - 材料を全部4つセットで並べなて、一気に取れるようにしないといけない

2つのプログラミング・モデル

1. SIMD 命令方式
2. SPMD方式

SIMD 命令と SPMD

■ SIMD 命令

- SIMD のハードウェアを直接命令に見せている
- 命令の形：「人参を 4 つ切って」

■ Single Program Multiple Data (SPMD) :

- マルチスレッド方式によってプログラムを記述
 - Single Program = 同じことをするスレッド
 - 本日冒頭の行列積の例そのもの
- SIMD のハードウェアはプログラマからは直接見えないよう隠蔽される
- 命令の形：「人参を 1 つ切って」
(具体的にいくつ切られるかはハードで決まる)

SPMD による並列化の例

- ループで書いた行列積のコード

```
for (k = 0; k < 1024; k++)  
  for (j = 0; j < 1024; j++)  
    for (i = 0; i < 1024; i++)  
      a[j][i] += b[j][k] * c[k][i];
```

- j の部分のループを複数のスレッドとしてマルチスレッド化

// func が 1024 個起動されて並列に実行される

// 0-1023 がスレッド ID として渡される

```
func(thread_id) {  
  j = thread_id; // k だと同じ場所を書いてしまう  
  for (k = 0; k < 1024; k++)  
    for (i = 0; i < 1024; i++)  
      a[j][i] += b[j][k] * c[k][i];  
}
```

SPMD で書かれたプログラムの実行

■ 以下の 2 通りの方法がある

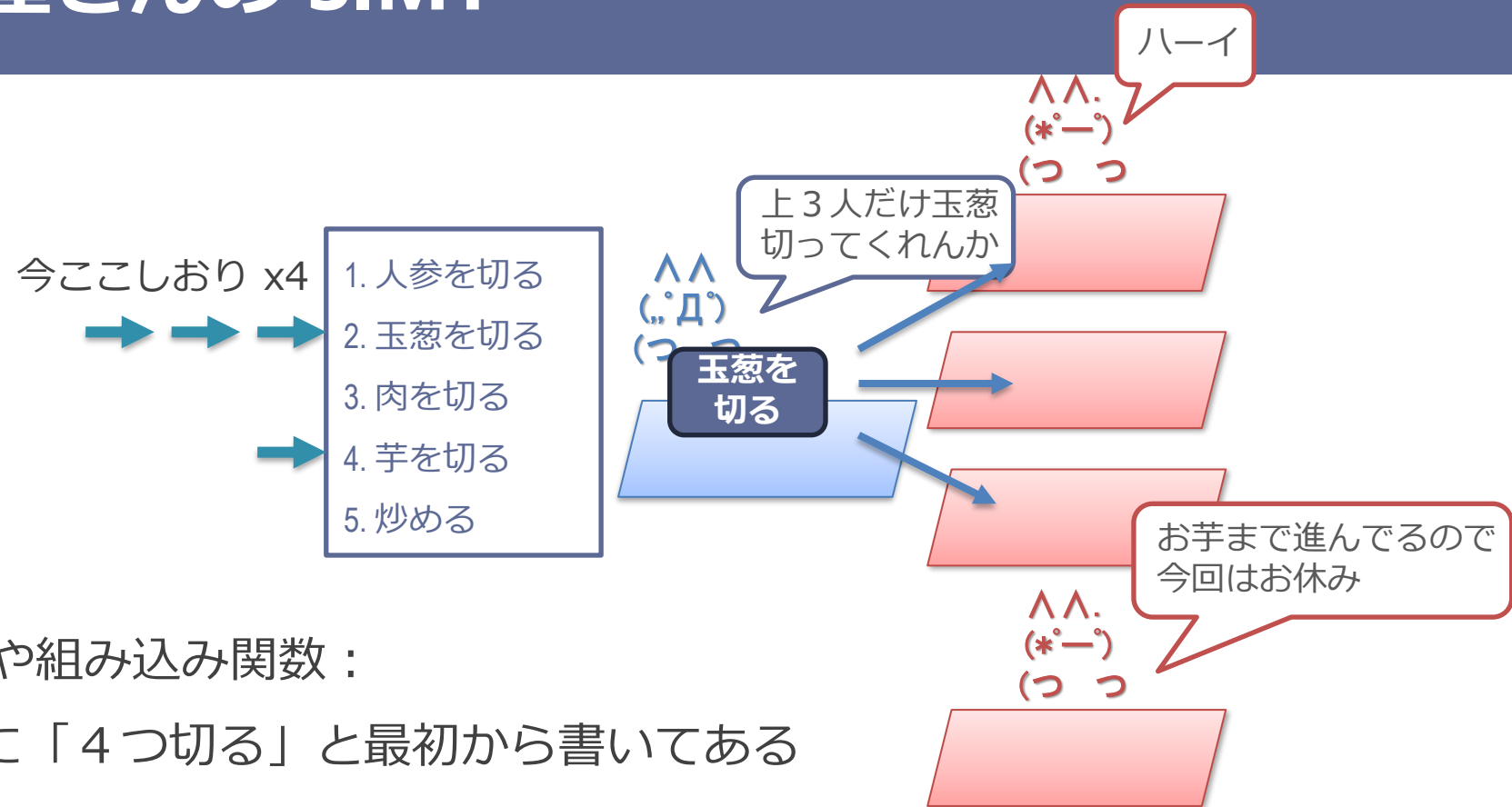
1. SIMD 命令への変換

- コンパイラにより自動で変換
- 変換された SIMD 命令を実行
- AMD/Intel の方式

2. Single Instruction Multiple Thread (SIMT) ハードウェアによる実行

- 命令列自体はスカラー命令で記述
- マルチスレッド実行をハードウェアで SIMD にマップする
- NVIDIA の方式

カレー屋さんの SIMT



■ SIMD 命令や組み込み関数：

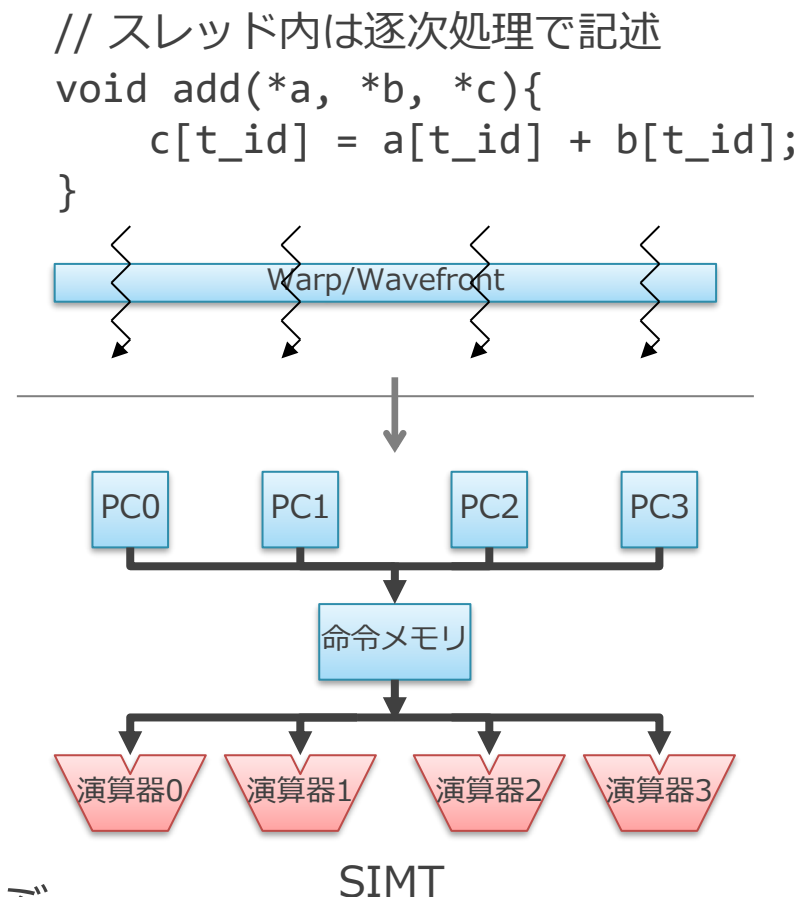
- レシピに「4つ切る」と最初から書いてある

■ SIMT：

- レシピ自体は「1つ切る」とだけ書いてある
- かわりに「今ここ」を4つ持ち、都度いくつ同時にできるか考える
- 「今ここ」が同じところにある人達の分は、一括して出来る

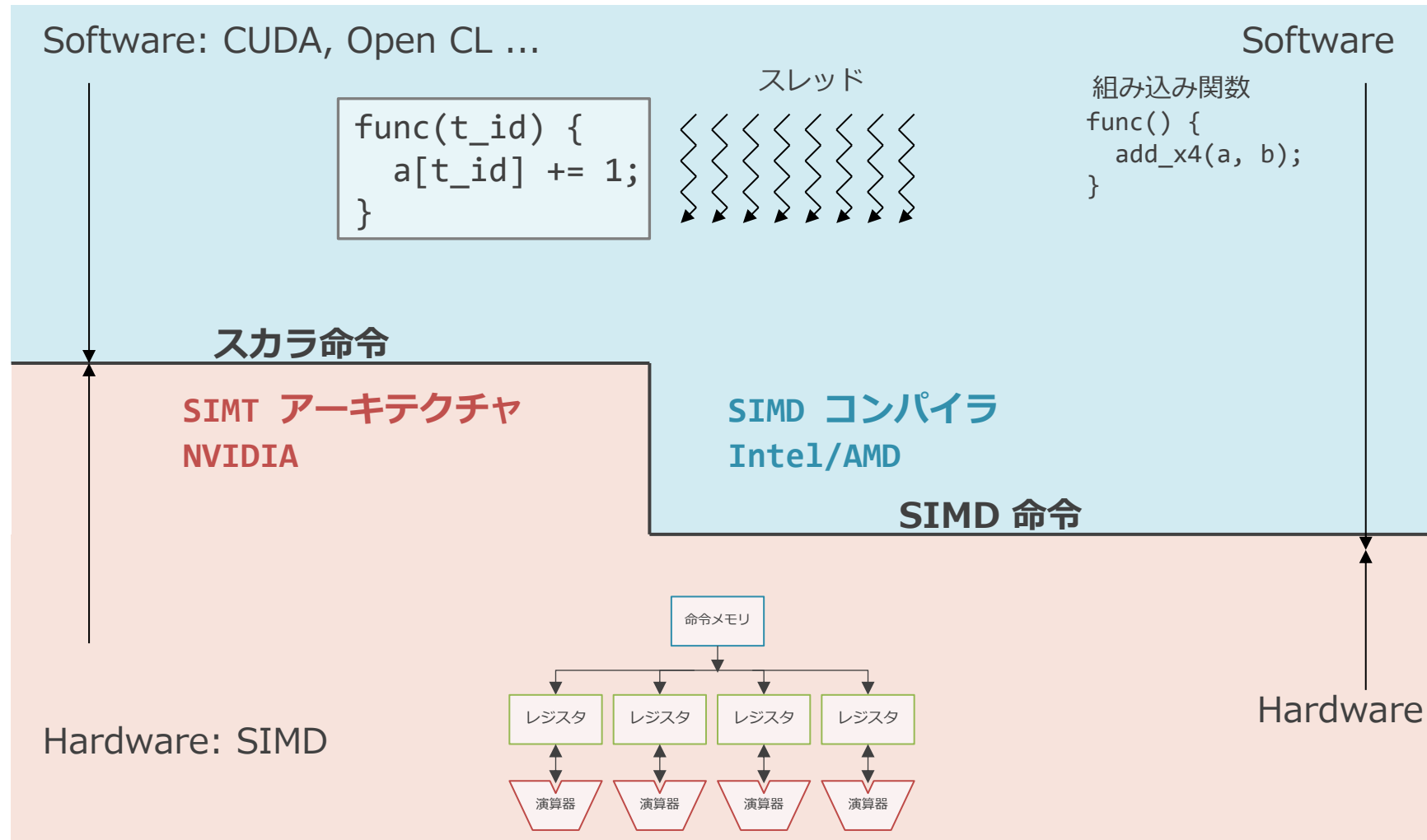
SIMT の構造

- PC を演算器の分だけ持つ
 - 命令メモリは 1 つのまま
- SPMD の各スレッドを PC に割り当てる
 - 基本的にはスレッドを時分割でそれぞれ実行
- ポイント：
 - 全スレッドの PC が同じアドレスを指している場合は SIMD プロセッサとして振る舞う
 - このまとまりを Warp or Wavefront と呼ぶ
 - 分岐しなければこの状態が保たれる

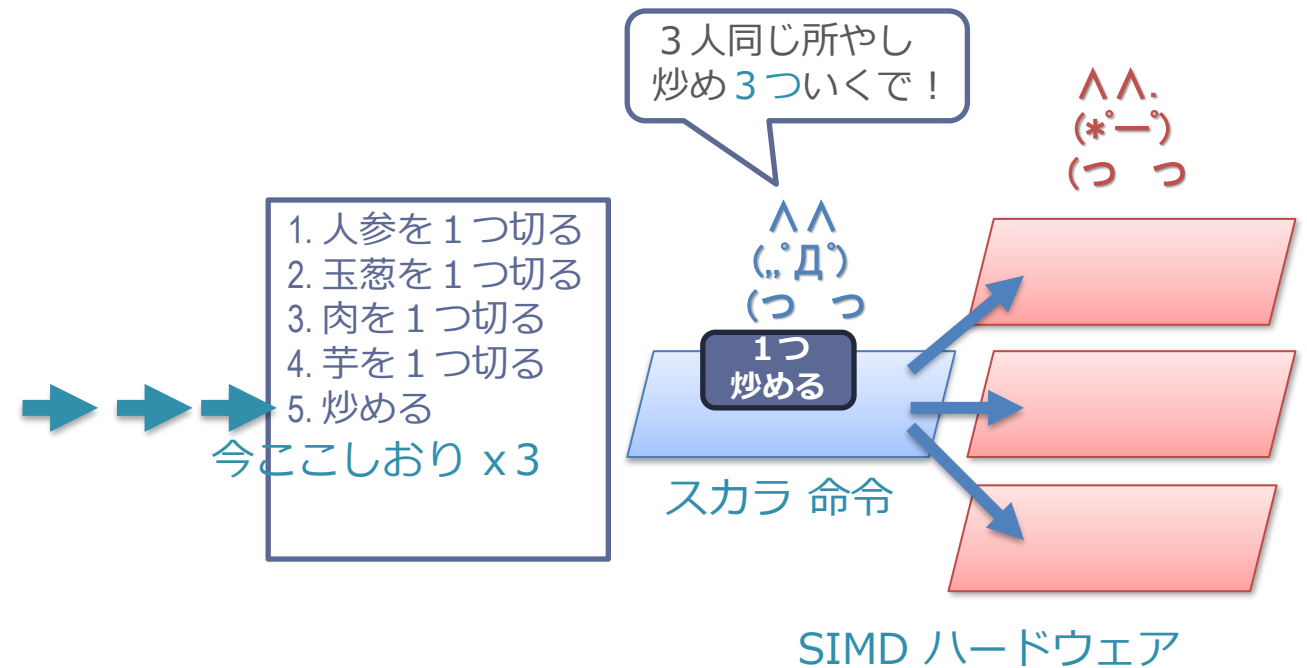
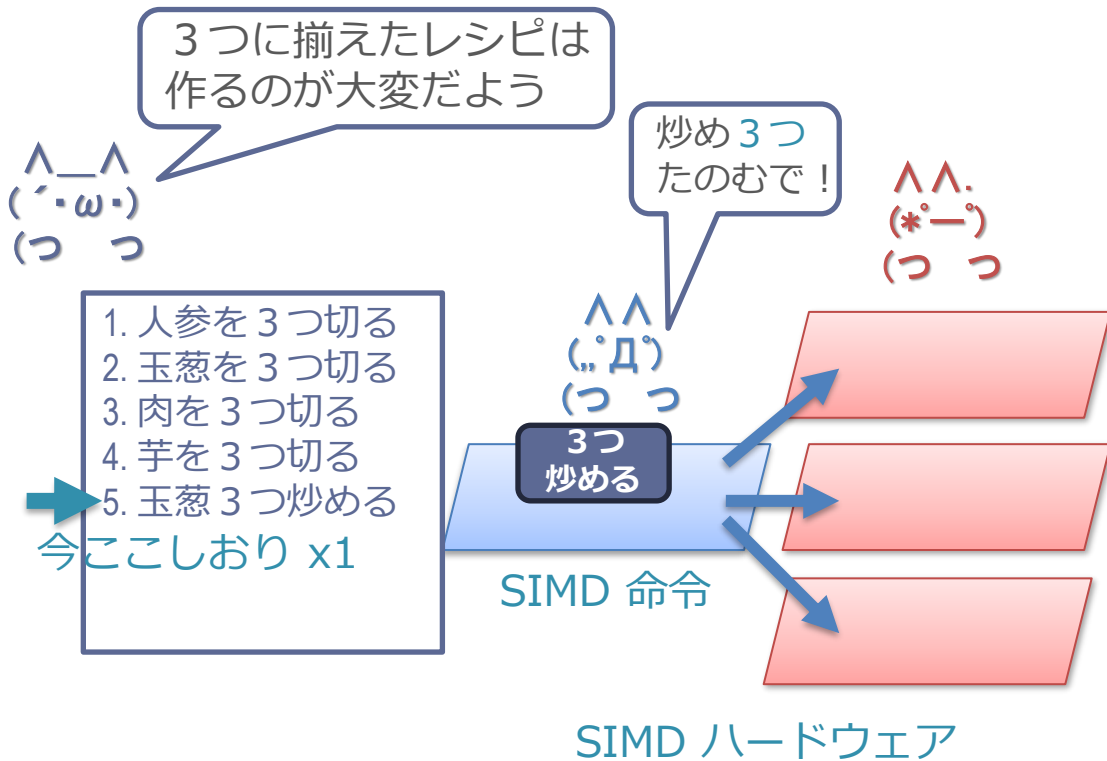


GPU におけるソフトとハードの関係

図は Caroline Collange “GPU architecture part 2: SIMT control flow management” より



カレー屋さんにおける SIMD 命令と SIMT



同じところを処理している場合に、指示をブロードキャストするのが SIMT.
ハードの形としては SIMD ハードウェアであり、この方式のことを SIMT と言う。

SIMT の利点と問題点

■ 利点：

- マルチスレッドで書かれたプログラムが SIMD プロセッサでそのまま実行できる
- = 難しいコンパイラがいらない
 - CPU 命令セットである RISC-V ベースの SIMT プロセッサも提案されている

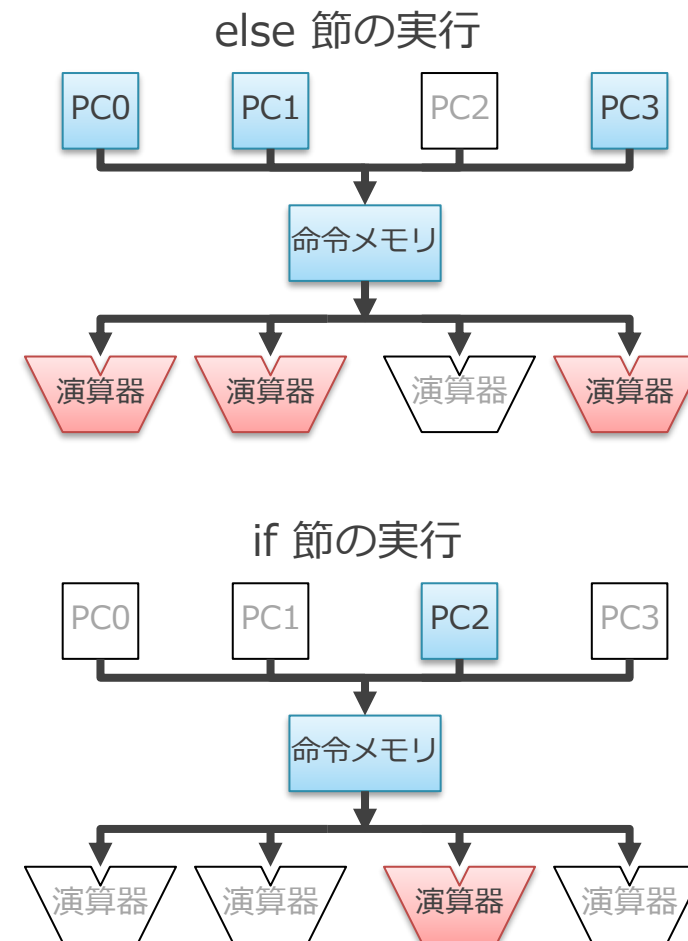
■ 問題点：

- スレッド間で違う方向に分岐すると実行速度が落ちる
 - branch divergence と呼ぶ
- 水平方向の演算が自然には表現できない

問題 1 : スレッド間で違う方向に分岐すると実行速度が落ちる (Branch divergence)

```
int branch_func(int t_id, int i) {  
    if (t_id == 2)  
        i++;  
    else  
        i--;  
    return i;  
}
```

- else と if の 2 回に分けて実行される
 - この例では実行速度が $\frac{1}{2}$ に



カレー屋さんにおける branch divergence

■ レシピに分岐があるとする

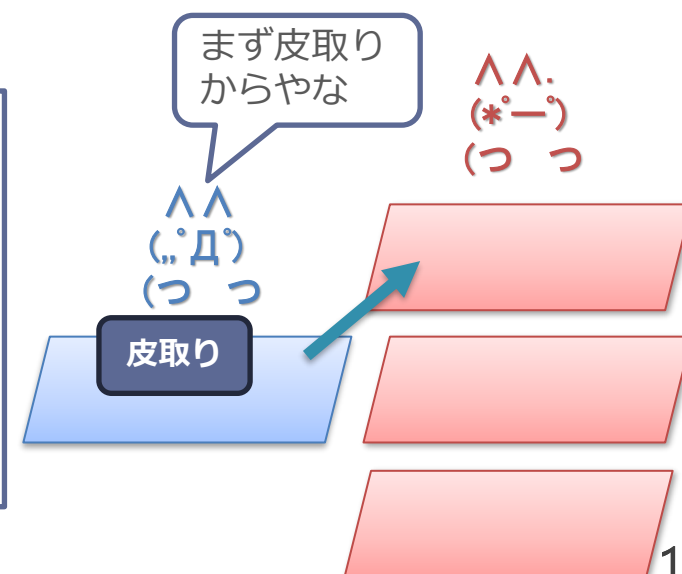
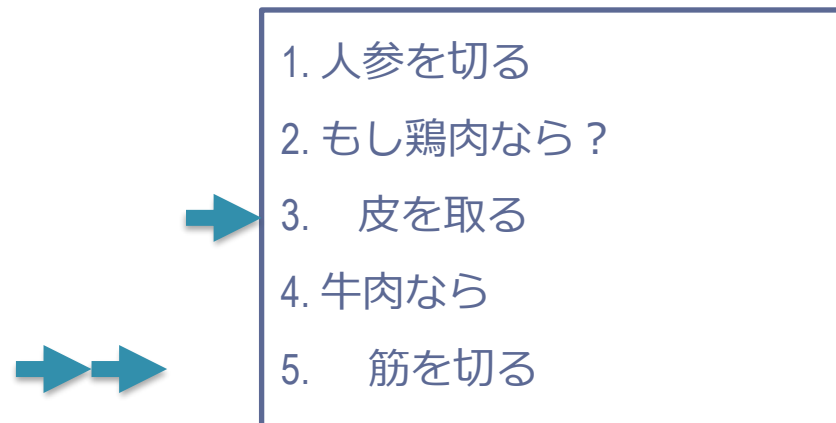
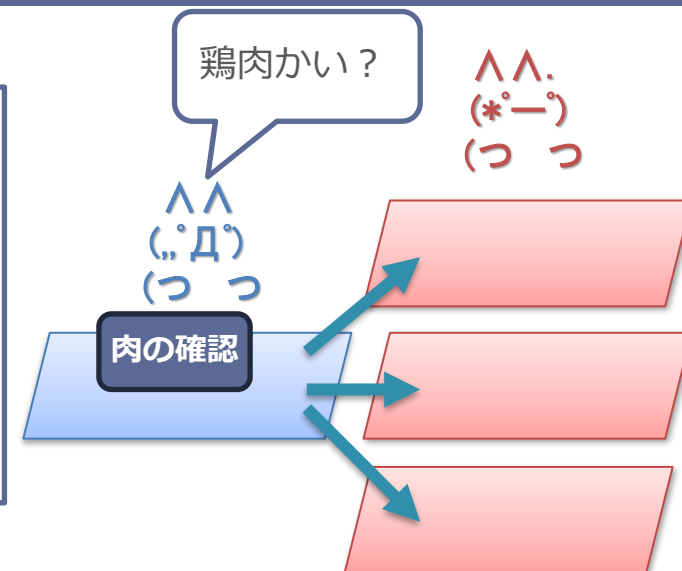
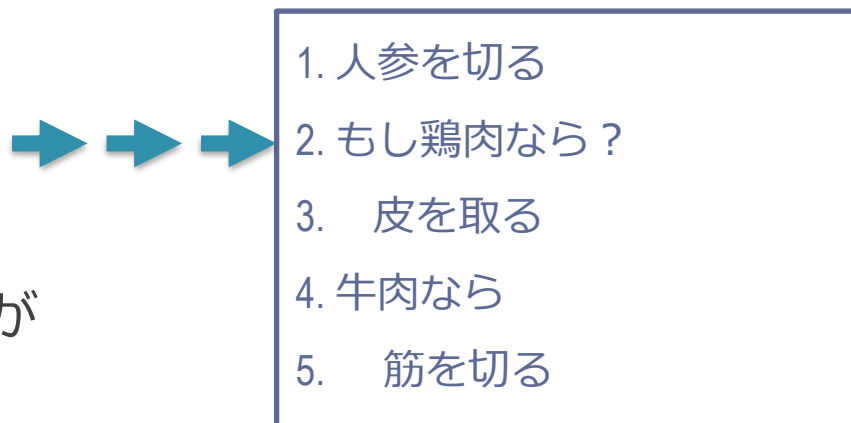
- 鶏肉か？牛肉か？

■ レシピの「今ここやってます」が分散する

- レシピ読み上げ指示の一人は一箇所しか読めない

■ 鶏肉の処理をやってから、牛肉の処理を時分割でやる

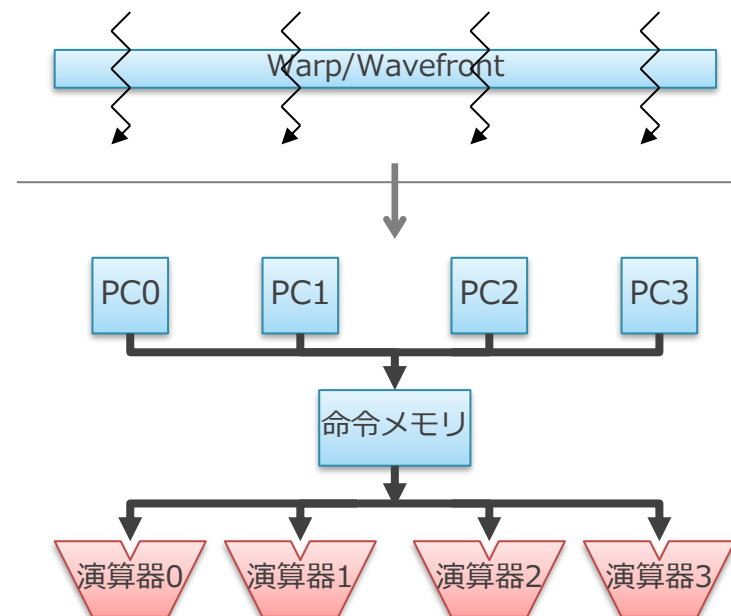
- 鶏肉の処理をしている間は下の2本は何もせず無駄



問題 2 水平方向の演算が自然には表現できない

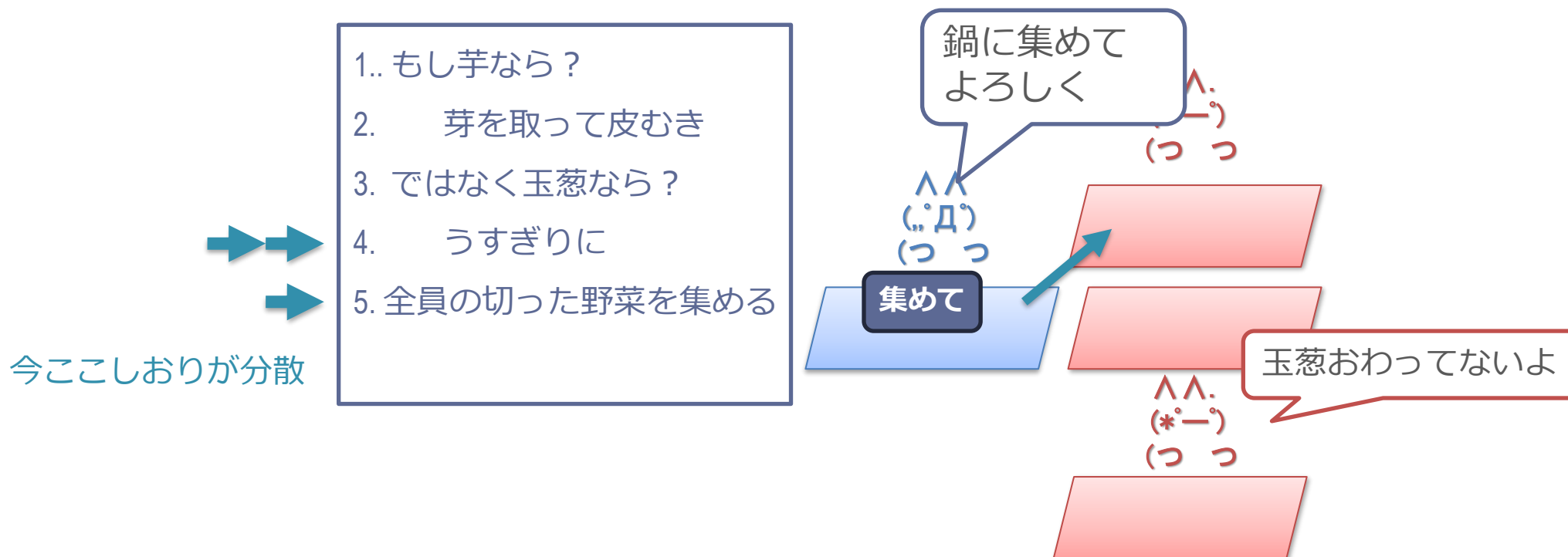
- 各演算器は独立したスレッドに割り当てられる
- 横にある演算器間では直接データのやり取りができない
 - 横で何が行われているかは保証できないため、そのようなプログラムが記述できない
 - SIMD 命令であれば、そのようなプログラムが書ける

```
// スレッド内は逐次処理で記述  
void add(*a, *b, *c){  
    c[t_id] = a[t_id] + b[t_id];  
}
```



カレー屋さんにおける 水平演算

- 「全員の野菜を集めて鍋にいれる」みたいな操作が難しい
 - 全員がそこに揃ってる事が保証できないため
 - 分岐があると時分割で処理される
 - 玉葱の薄切りが終わってないかもしれない
 - 「ここで全員を待つ」みたいな指示が必要



SIMD 命令と SIMT のまとめ

- SIMD プロセッサをどのように使うか
 - SIMD 命令モデル vs. SIMT モデル
 - (モデルの名前が適切ではない気がする)
 - 異なる層で、複数演算器の同時処理の対応を行っている
 - SIMD 命令： コンパイラ or 人手
 - SIMT： ハードウェア

GPU のアーキテクチャのまとめ

- 下記の性質を利用

1. 並列に動作する大量のスレッドがある
2. 各スレッドは基本的に同じことをしている

- 演算器以外の回路を削減

- 制御部を共有してハード量当たりの性能を向上
- 制御部自体の小型化

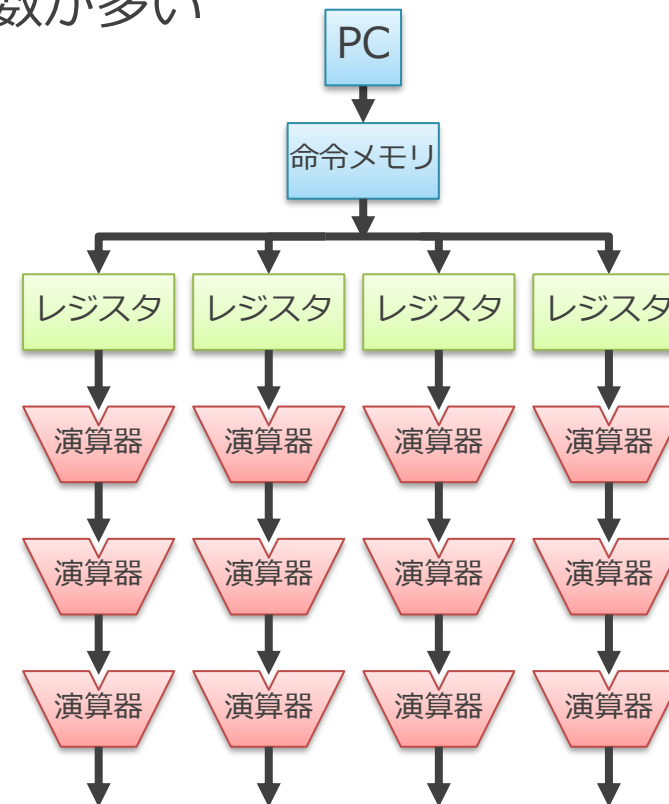
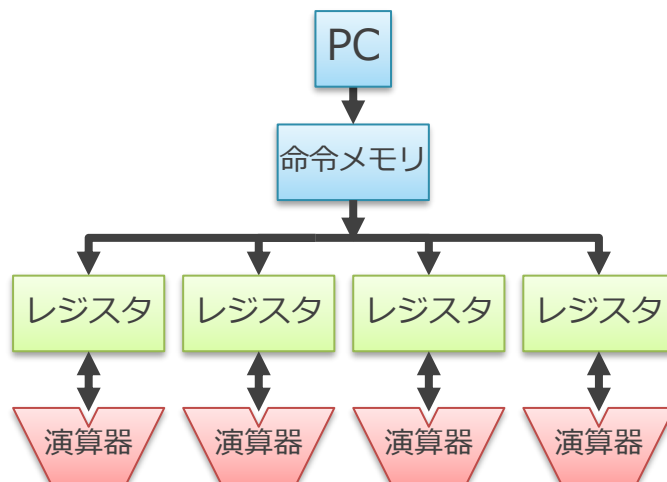
アクセラレータについて

アクセラレータや GPU は何故 CPU より速いのか？

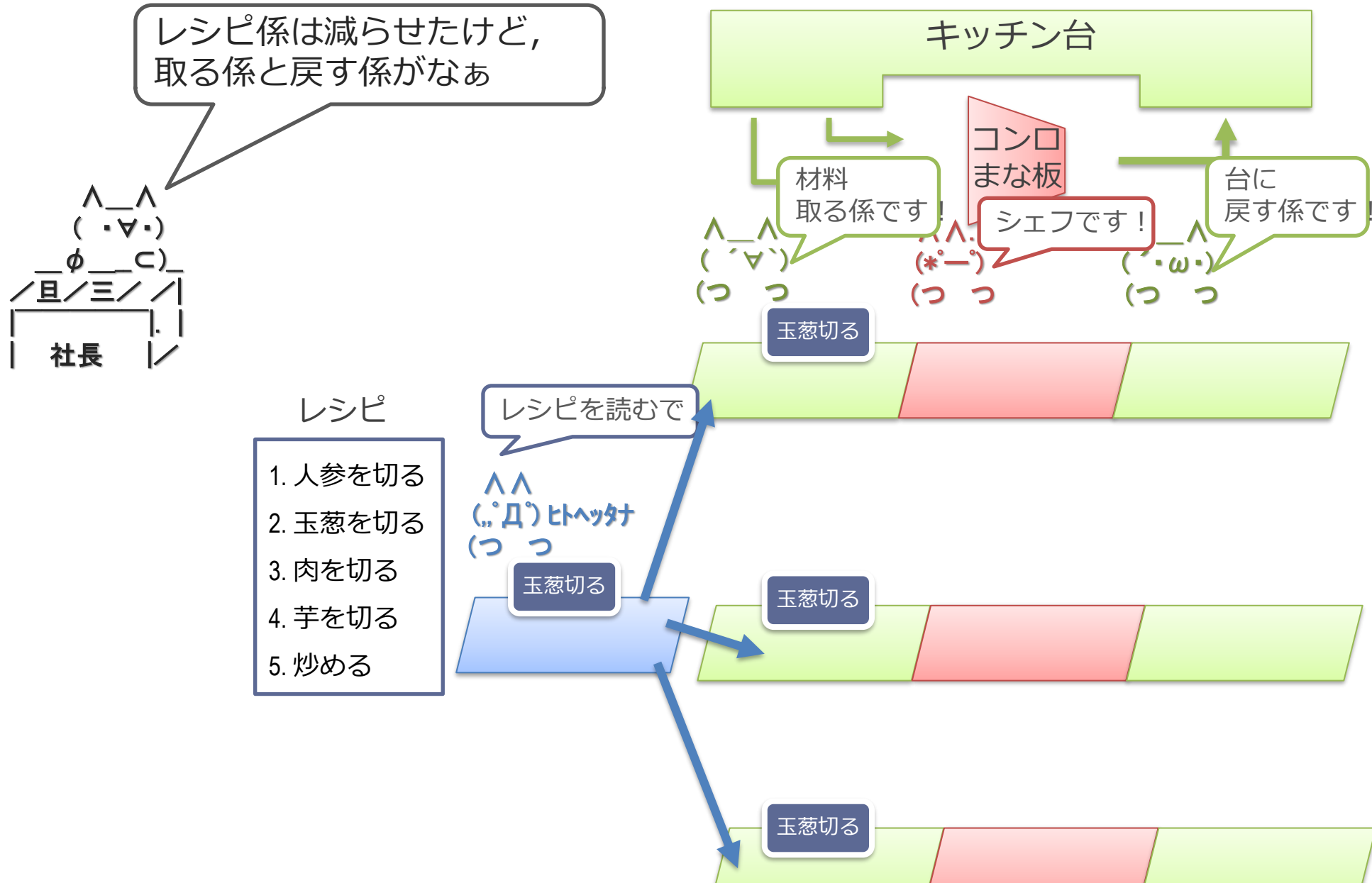
- アクセラレータ
 - 特定の問題に特化したハードウェア
- 演算器そのものは一緒
 - 個々の加算器や乗算器などはどのアーキテクチャでも共通
- 制御部やデータのやりとりをいかに減らすかにより決まる
 - 対象のプログラムの性質に特化することで、機能を削っても性能が落ちないようにする
 - 減らした分だけよりたくさん演算器がつめる

行列積アクセラレータの場合

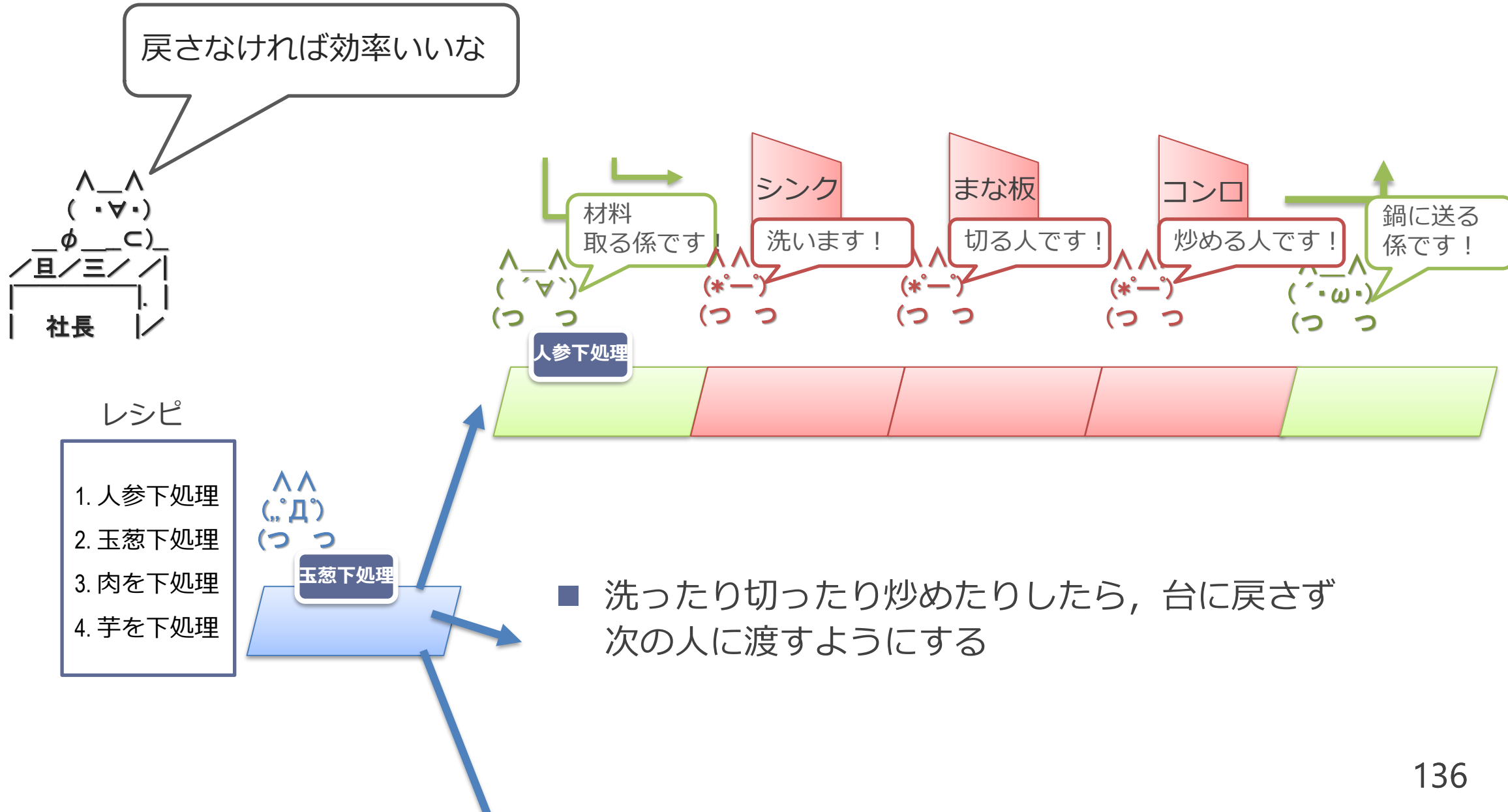
- 行列積では、単純な GPU はまだ無駄がある
 - 演算1回ごとに命令を読んだり、レジスタ・ファイルにアクセス
- 演算器を直接繋いでそこにデータを流せば良い
 - 行列積は入力データ数と比べて演算数が多い
 - $O(N^2)$ vs. $O(N^3)$
 - Google TPU や NVIDIA Tensor Core



カレー屋さんの場合：キッチン台に毎回戻すのが無駄



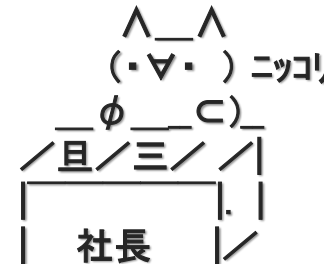
カレー屋さんの場合：戻さず流れ作業に



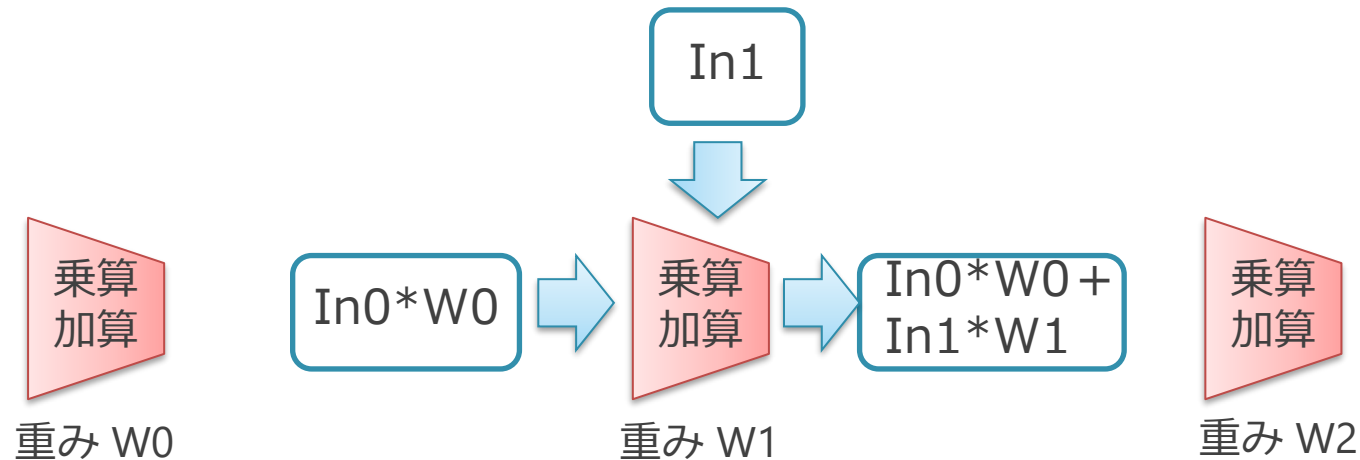
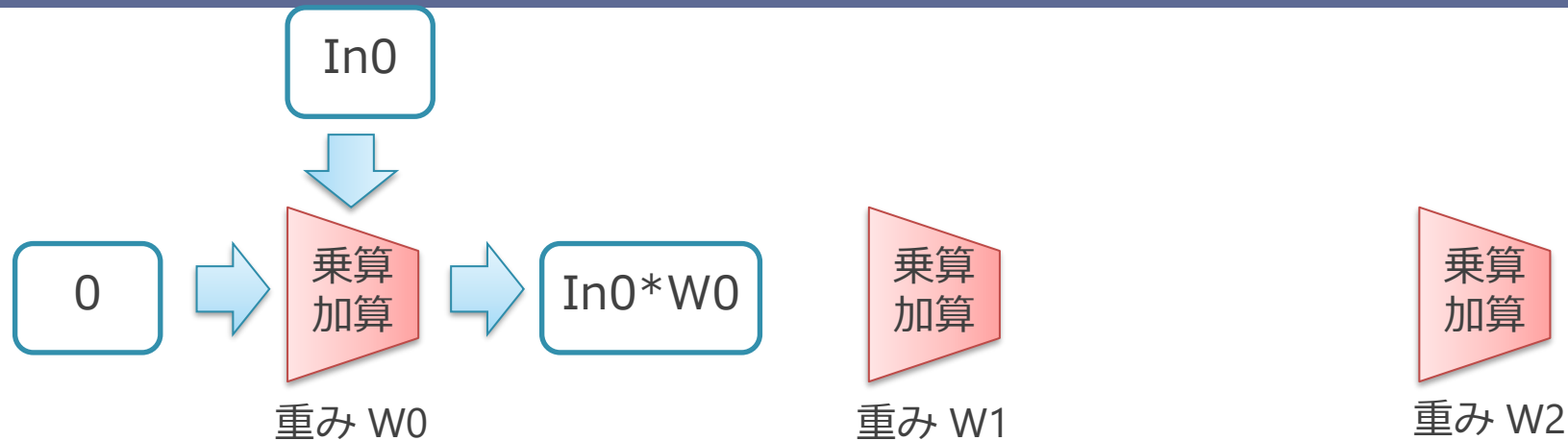
カレー屋さんの場合



- 全体の数が変わらず，赤い部分（演算器）の数が増えている
 - Before: 洗う or 切る or 炒める，が合計 5 つできる
 - After: 洗う and 切る and 炒める，が 3 セットできる
 - 炒め終わったものが 3 つ毎回できる
 - 処理としては合計 9 つできている



行列積の場合：シストリックアレイ

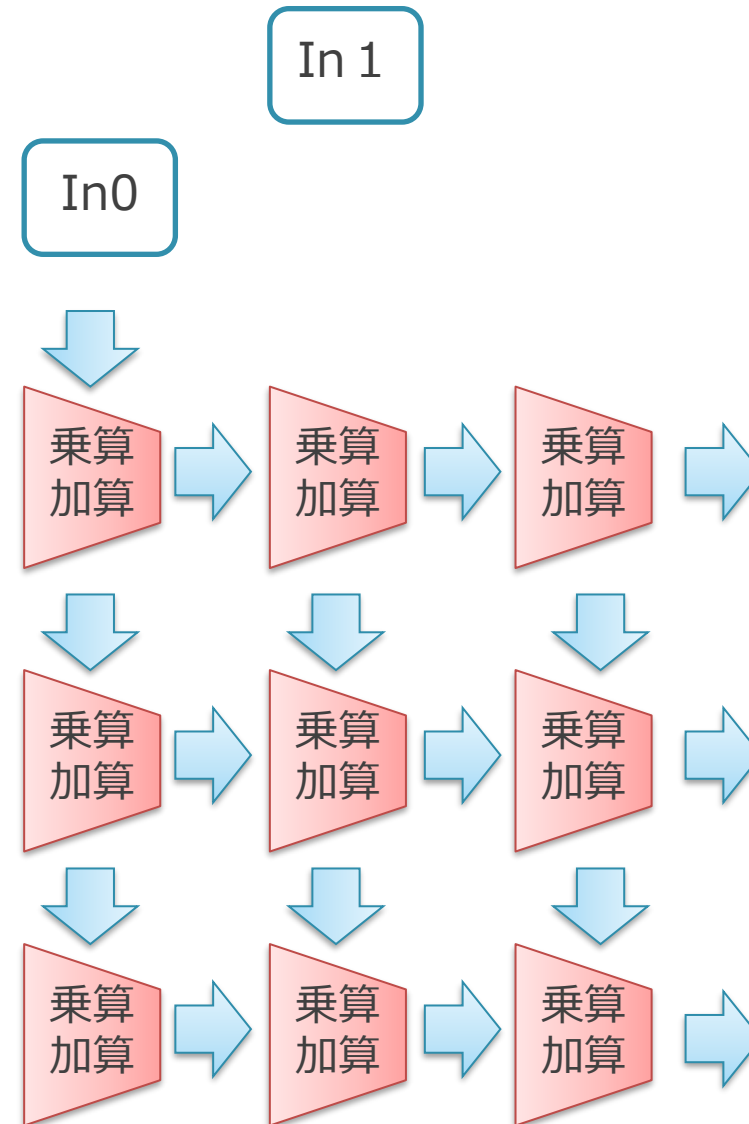


- 上から来た Input に Weight をかけて、左から来たものに足し、右に流す
- 右端からドット積の結果がでてくる

行列積の場合：シストリックアレイ

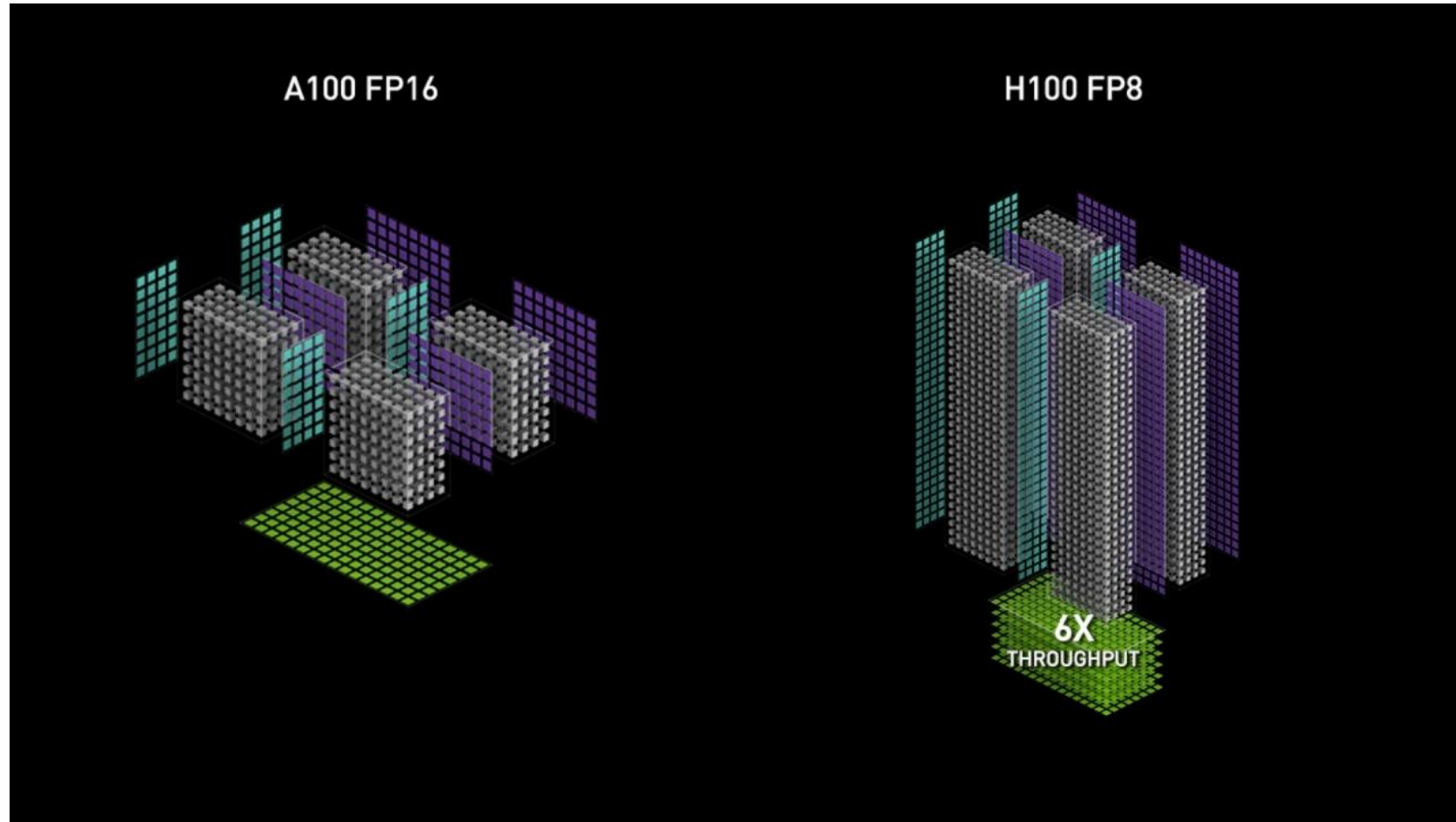
In2

- 2次元的に同様にデータを流していく
 - 上からいれるデータは，流れてくるものとタイミングを合わせてずらす
 - $In0 * W0 + In1 * W1$
- シストリック：心臓の脈打ち
 - ドクン ドクンとデータが処理されて流れていく
- Input/Output/Weight のどれを演算器に貼り付けるかで，いくつかやり方がある



Tensor Core (NVIDIA Ampere と Hopper のもの)

<https://www.nvidia.com/ja-jp/data-center/tensor-cores/> より



- NVIDIA GPU の Tensor Core は、より直接に行列積を一気に行う

Google TPU v1 の構造

Norman P. Jouppi et al., In-Datcenter Performance Analysis of a Tensor Processing Unit (ISCA 2017)より

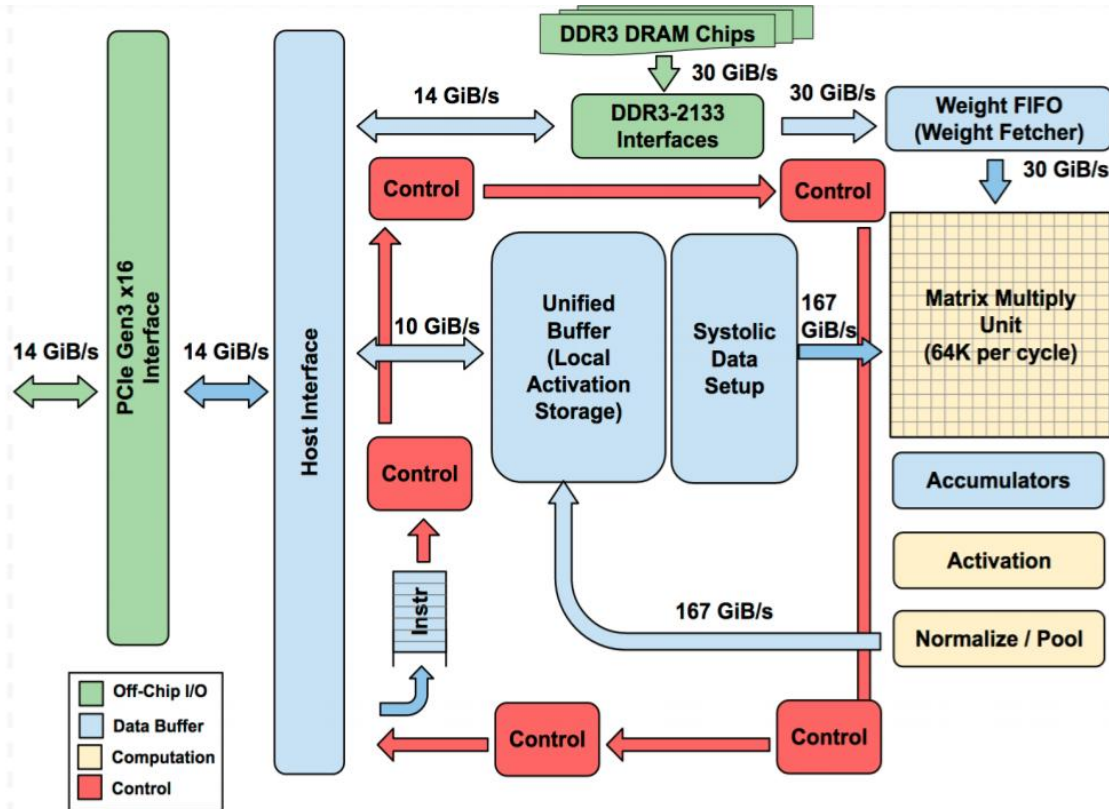


Figure 1. TPU Block Diagram. The main computation is the yellow Matrix Multiply unit. Its inputs are the blue Weight FIFO and the blue Unified Buffer and its output is the blue Accumulators. The yellow Activation Unit performs the nonlinear functions on the Accumulators, which go to the Unified Buffer.

■ 構成：

- 行列積を行うシストリックアレイ
- 活性化関数进行处理する SIMD

■ この構成はよくある

META (旧 Facebook) MITA v2 の構造

Joel Coburn et al. Meta's Second Generation AI Chip: Model-Chip Co-Design and Productionization Experiences (ISCA 2025) より

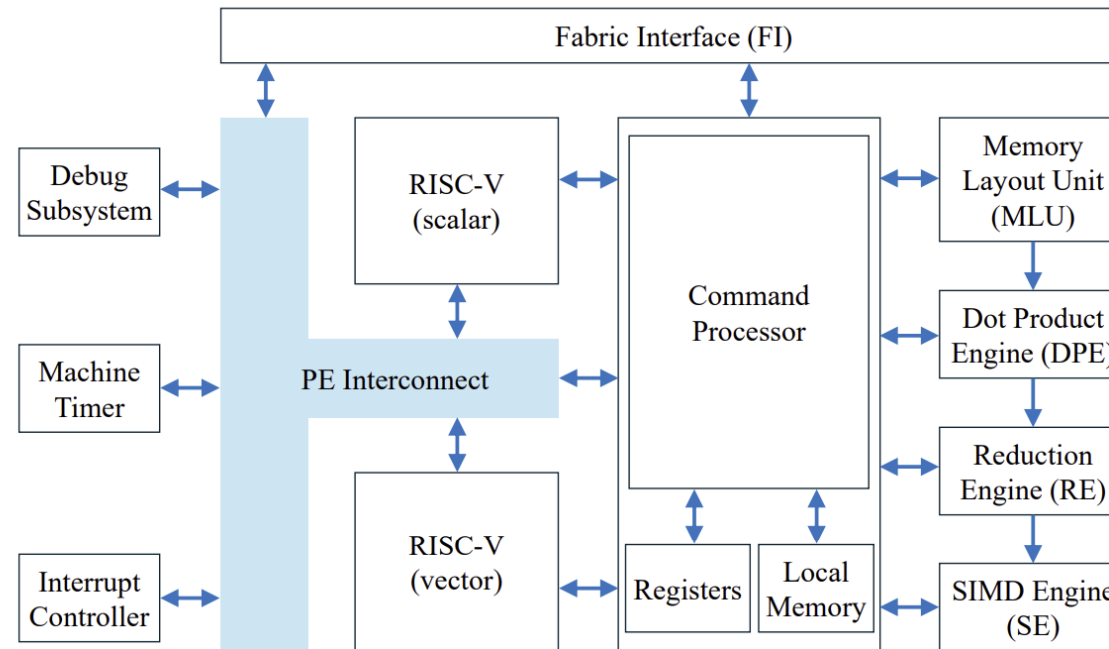


Figure 2: Internal architecture of Processing Element (PE).

■ 先ほどと同様の構造

- Dot Product Engine が行列積
- Reduction Engine と SIMD Engine が活性化関数処理

■ AI フレームワーク対応のために、柔軟性がある RISC-V CPU がのっている

アクセラレータのまとめ

- 行列積では、演算器を並べて効率を向上
 - 命令の制御だけでなく、レジスタの読み書きも省略できる
- AI アクセラレータの場合、以下の構成が典型的
 - 行列積エンジン
 - 活性化関数処理する SIMD エンジン
 - 活性化関数の処理は、あまり数珠つなぎにできないため、SIMD になっていることが多い

まとめ

まとめ：半導体回路の消費電力

- 半導体回路の消費電力：
 - 回路の量と動作回数に比例する
 - 流れる電荷の総量と電圧により決まる
 - 流れ込む水流のイメージに近い
 - タンクやパイプにたまる水の量と高さで説明

まとめ：カレー屋さんの場合

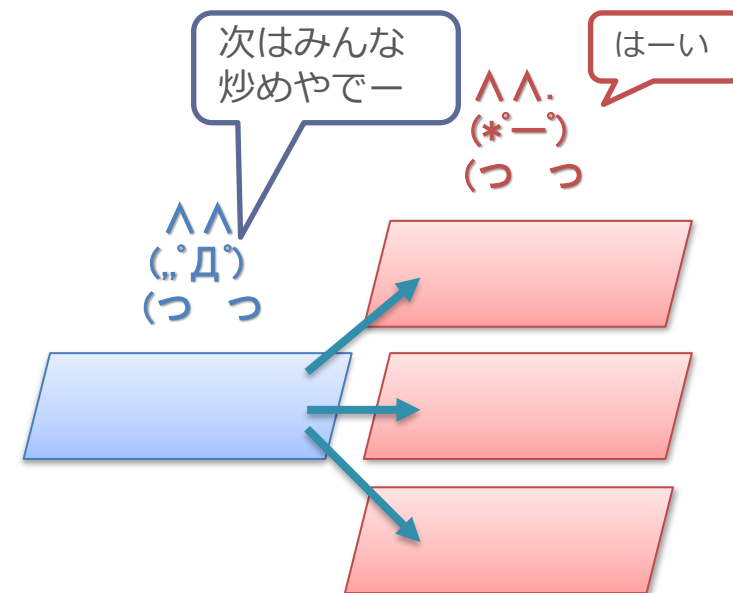
■ 前提：

- 1つのカレーを作るのではなく、大量の同じカレーを作るのが目的
- レシピを読む人やキッチン台担当の人を減らしても、同じカレーなら大量に作れる

■ 効率の上げ方：

- レシピを読む人の数を減らし、シェフ人数を増やす
- レシピを読む人に難しいことをさせず、人件費を下げる
 - 難しい事 = レシピの中で並列に出来ることを探す

■ 根っこの部分では GPU は同じことをしている



まとめ : GPU と CPU の違い

- GPU は CPU とは何が違うのか？
 - 制御部を共有して回路全体に占める演算器比率をあげている (SIMD)
 - 制御部で難しいことをさせず, 小型化しその分演算器比率を上げる
- なぜ GPU やアクセラレータは, AI で高速で効率が良いのか？
 - 行列積や活性化関数は, 同じ処理を複数のデータに対して処理している
 - 演算器アレイや SIMD 型の構造で, 余計なことをせずに効率よく処理できる

補足：本日触れなかった GPU に関する話題

- マルチスレッド実行によるレイテンシの隠蔽
 - GPU では非常に多くのスレッドを同時に実行することで、メモリアクセスのレイテンシを隠蔽している
 - SIMT や SPMD においてスレッドが並列に実行されているのとは直交した概念
- 演算精度の低精度化
 - 付録で少しだけ補足

補足：GPU についての他所の講義資料について

- Caroline Collange (フランス国立情報学自動制御研究所),
"GPU architecture",
<https://team.inria.fr/pacap/members/collange/>
- Juan Gómez Luna and Onur Mutlu (チューリッヒ工科大学),
"Computer Architecture Lecture 7: SIMD Processors and GPUs",
<https://safari.ethz.ch/architecture/fall2018/lib/exe/fetch.php?media=comparch-fall2018-lecture7-simdandgpu-afterlecture.pdf>

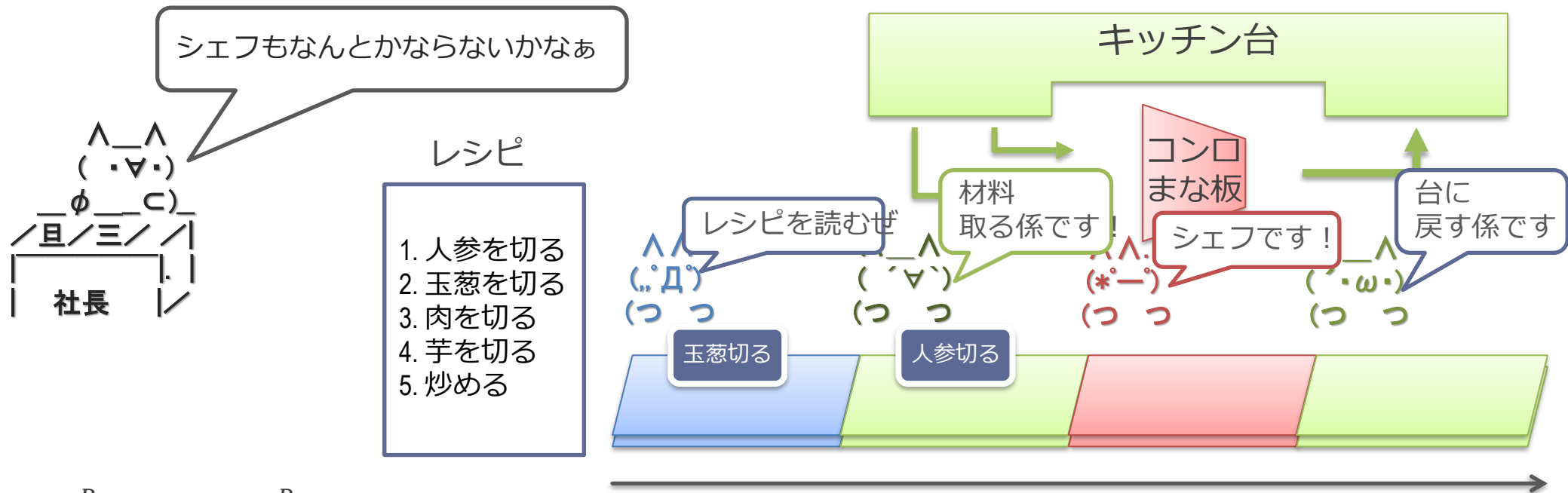
より GPU の事を勉強したい人はたとえば上記の講義資料がおすすめです

今回触れなかった部分の補足

- 演算の低精度化
 - GPU では演算の精度も低下させて消費エネルギーを削減している
- マイクロスケーリング (MX)
 - 複数の低精度のデータと、共有された指数部から構成
 - 個々の乗算は低精度となり、回路が簡略化される
 - 推論だけでなく、学習でも使用する取り組みが進んでいる
 - なぜ指数を共有できるのか？
 - * 行列演算などでドット積を取ると、最も指数が大きいものが結果に対して支配的になる
 - * 指数が小さいものは精度が低下しても結果への影響が小さい
 - MS/ARM/Meta/Intel/AMD/Qualcomm による標準化
 - OCP Microscaling Formats (MX) Specification Version 1.0
 - <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf>
 - NVIDIA GPU でも Blackwell 世代から採用

(時間がたりないので省略)

カレー屋さんのコストカット



■ 効率 $\eta = \frac{P_{min}}{P_{actual}} = \frac{P_{min}}{P_{min} + P_{overhead}}$

- P_{min} = 材料の原価とシェフの人件費（絶対カレーを作るのに必要）
- $P_{overhead}$ = キッチン台, 他の係の人（必ずしもなくても）

■ トータルコストを下げる：

- P_{min} を減らす = 切り方を雑にして質を下げて（演算の精度を下げる）トータルコストを下げる
- 見た目の効率 η はこの場合は改善されない（ $P_{overhead}$ の相対的な割合が増す）